



UNIVERSITÀ DEGLI STUDI
DI NAPOLI FEDERICO II

NaTour

PROGETTAZIONE E SVILUPPO DI UN SISTEMA COMPLESSO E
DISTRIBUITO FINALIZZATO AD OFFRIRE UN SOCIAL NETWORK PER
APPASSIONATI DI ESCURSIONI.



Documentazione progetto Ingegneria del Software

a.a. 2021 - 2022

INGSW₂₁₂₂_N_o6

Marotta	Vincenzo	N86003005
Muto	Emanuele Ciro	N86003440
Salernitano	Mattia	N86003385



Sommario

Descrizione del progetto.....	7
1.1 Analisi e funzionalità del problema	7
Modello Funzionale	9
2.1 Introduzione.....	9
2.2 Modellazione dei casi d'uso	9
2.2.1 Utente non registrato	9
2.2.2 Utente registrato.....	10
2.2.3 Crea Itinerario	11
2.2.4 Amministratore.....	11
2.3 Tabelle di Cockburn	12
2.3.1 Creazione di un itinerario	12
2.3.2 Invio di un messaggio	31
2.4 Prototipazione Visuale	34
2.4.1 Mock-up creazione itinerario	34
MOCKUP_1_0	34
MOCKUP_1_1.....	35
MOCKUP_1_2	36
MOCKUP_1_3	37
MOCKUP_1_4.....	38
MOCKUP_1_5	39
MOCKUP_1_6	40
MOCKUP_1_7	41
MOCKUP_1_8	42
MOCKUP_1_9	43
MOCKUP_1_10.....	44
MOCKUP_1_11.....	45
MOCKUP_1_12	46
MOCKUP_1_13	47
2.4.2 Mock-up invio messaggio.....	48
MOCKUP_2_1	48
MOCKUP_2_2.....	49



MOCKUP_2_3.....	50
MOCKUP_2_4.....	51
MOCKUP_2_5.....	52
MOCKUP_2_6	53
MOCKUP_2_7.....	54
2.5 Presentazione dell'idea progettuale.....	55
2.6 Target d'utenti	56
2.7 Valutazione usabilità a priori.....	59
2.8 Prototipazione funzionale interfaccia grafica	60
2.8.1 Statechart creazione itinerario	60
2.8.2 Statechart invio messaggio privato	61
2.9 Glossario	62
Modello di dominio	64
3.1 Introduzione.....	64
3.2 Classi di dominio	64
3.2.1 Registrazione	65
3.2.2 Accesso	66
3.2.3 Home	67
3.2.4 Creazione itinerario	68
3.2.5 Visualizzazione itinerario	69
3.2.6 Ricerca	70
3.2.7 Messaggistica.....	71
3.3 Sequence Diagram	72
3.3.1 Ricerca.....	72
3.3.2 Accesso	73
3.4 Activity Diagram.....	74
3.4.1 Registrazione	74
3.4.2 Accesso	75
3.4.3 Creazione itinerario	76
3.4.4 Visualizzazione itinerario.....	77
3.4.5 Aggiunta lista preferiti.....	78
3.4.6 Aggiunta lista da visitare	79
3.4.7 Condivisione posizione.....	80



3.4.8 Ricerca.....	81
3.4.9 Messaggistica	82
3.4.10 Promozione ad admin	83
3.4.11 Modifica o cancellazione di un itinerario.....	84
Design del sistema	85
4.1 Introduzione.....	85
4.2 Analisi dell'architettura.....	85
4.2.1 Architettura server	86
4.2.2 Architettura Client.....	89
4.3 Servizi cloud utilizzati.....	89
4.4 Progettazione Database	91
4.5 Diagrammi classi di design	92
4.5.1 Registrazione	92
4.5.2 Accesso	93
4.5.3 Creazione itinerario	94
4.5.4 Visualizzazione itinerario.....	95
4.5.5 Home	96
4.5.6 Ricerca.....	97
4.5.7 Messaggistica	98
4.6 Sequence Diagram	99
4.6.1 Accesso	99
4.6.2 Visualizzazione itinerario	100
4.7 Gerarchia funzionale	101
Testing.....	102
5.1 Introduzione.....	102
5.2 Metodo buildLocationString.....	102
5.2.1 Metodo	103
5.2.2 Strategia Black Box	104
5.2.3 Metodi di test Black Box	105
5.2.4 Strategia White Box	108
5.2.5 Metodi di test White Box.....	109
5.3 Metodo createFilterQuery.....	111
5.3.1 Metodo	112



5.3.2 Strategia Black Box.....	113
5.3.3 Metodi di test	114
5.4 Metodo checkPassword.....	121
5.4.1 Metodo.....	121
5.4.2 Strategia Black Box	122
5.4.3 Metodi di test Black Box.....	124
5.4.4 Strategia White Box.....	127
5.4.5 Metodi di test White Box	128
5.5 Valutazione usabilità.....	129





Capitolo 1

Descrizione del progetto

1.1 ANALISI E FUNZIONALITÀ DEL PROBLEMA

Si progetterà ed implementerà un sistema complesso e distribuito finalizzato ad offrire un moderno social network multiplatforma per appassionati di escursioni.

Il sistema sarà diviso in due parti:

- Una parte di back-end sicuro, performabile e scalabile;
- Un client mobile attraverso il quale gli utenti potranno usufruire delle funzionalità del sistema in modo intuitivo, piacevole e veloce.

Il sistema offrirà un ampio numero di funzionalità, di cui l'utente potrà usufruire solo se registrato correttamente.

Le funzionalità offerte da NaTour sono le seguenti:

- Inserimento di nuovi itinerari:

L'itinerario sarà caratterizzato da un nome, una durata, il livello di difficoltà, un punto di inizio, una descrizione ed un tracciato geografico.

L'utente potrà creare itinerari pubblici o privati.

Sarà possibile registrare l'itinerario attraverso inserimento di punti manualmente, usando un file .gpx oppure registrando una sequenza di posizioni GPS con il proprio dispositivo mobile.

- Ricerca di itinerari:

L'utente potrà cercare itinerari presenti in piattaforma con possibilità di filtrare i risultati per area geografica, livello di difficoltà, durata e accessibilità ai disabili.

- Visualizzare gli itinerari:

L'utente sarà in grado di visualizzare gli itinerari in una schermata con una mappa interattiva che mostrerà i dettagli dell'itinerario e il tracciato.

In questa schermata sarà possibile condividere la propria posizione sulla mappa con altri utenti, se il dispositivo mobile supporta la localizzazione GPS.



- Gestione di liste:

L'utente sarà in grado di gestire e visualizzare la lista dei preferiti e la lista di itinerari da visitare, aggiungendo e rimuovendo itinerari personali e/o di altri utenti.

- Messaggi privati:

L'utente sarà in grado di scambiare messaggi in privato con un altro utente, per esempio per chiedere informazioni su un itinerario oppure per organizzare un'escursione insieme.

- Ruolo di amministratore:

Un utente amministratore è in grado di rendere amministratore un altro utente. L'amministratore può modificare o eliminare gli itinerari già creati. Gli utenti che visualizzeranno l'itinerario, visualizzeranno un'icona di warning all'interno della schermata.



Capitolo 2

Modello Funzionale

2.1 INTRODUZIONE

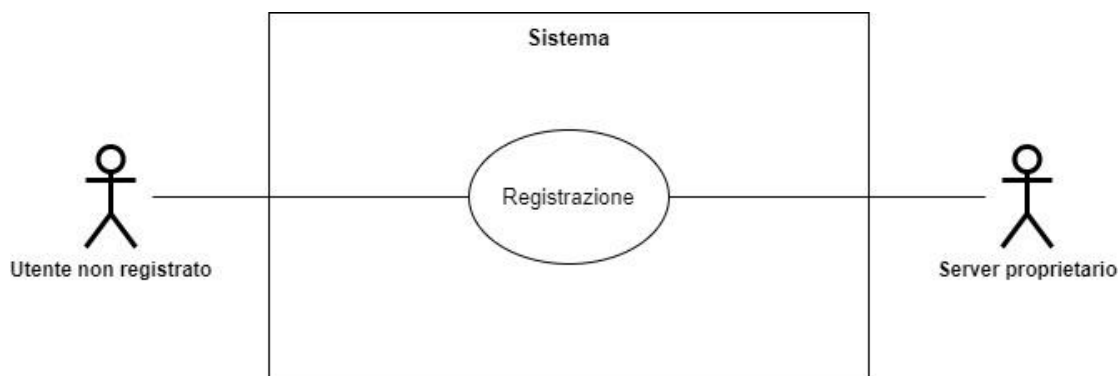
Dopo aver analizzato e descritto per sommi capi le funzionalità del sistema, si passa ad una descrizione più accurata delle funzionalità.

2.2 MODELLAZIONE DEI CASI D'USO

Di seguito viene riportato lo Use Case Diagram relativo alle funzionalità che il sistema è in grado di offrire.

L'Use Case Diagram (abbreviato UCD) verrà diviso in più parti per facilitarne la leggibilità.

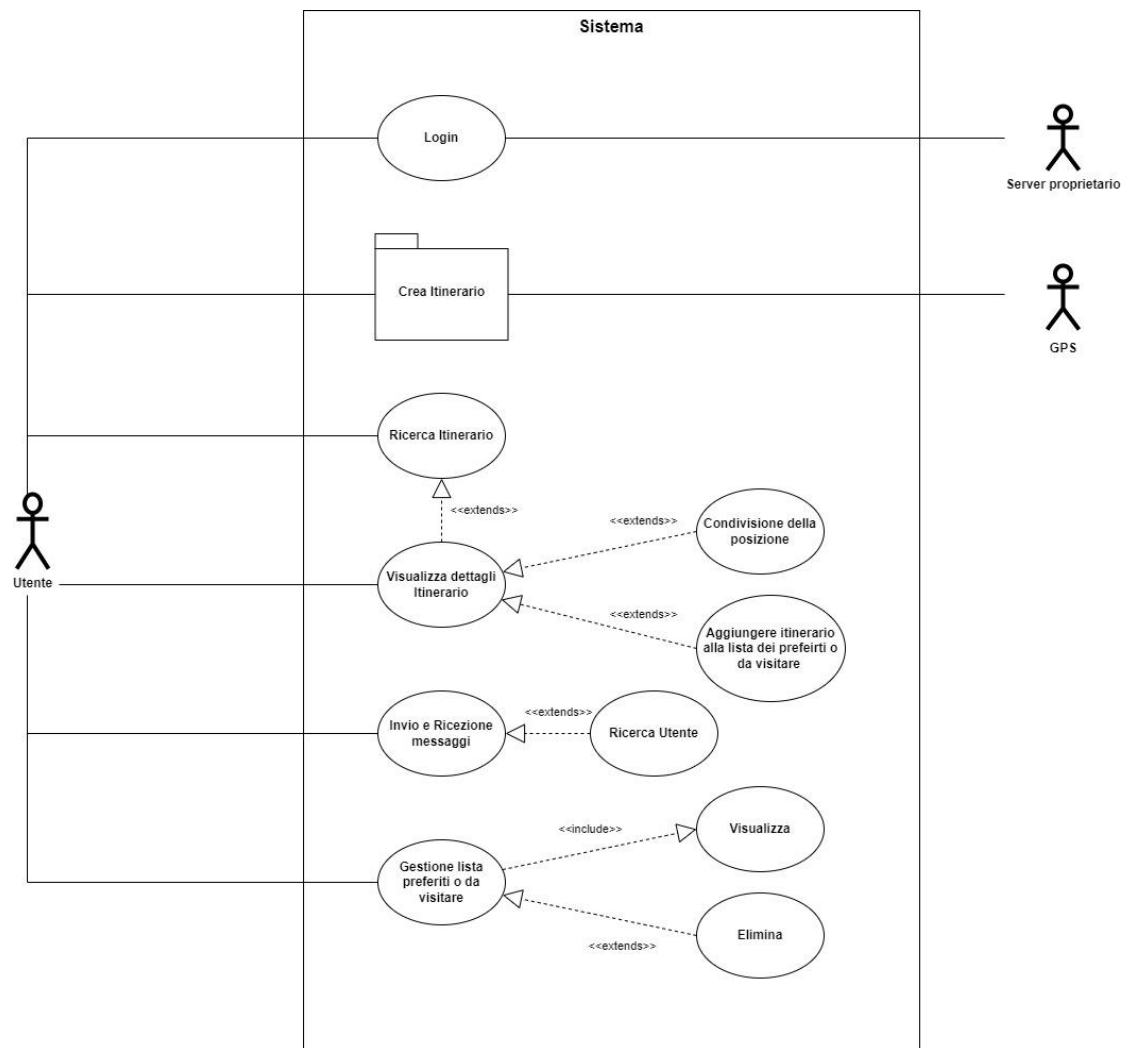
2.2.1 UTENTE NON REGISTRATO



UCD 1 - Utente non registrato



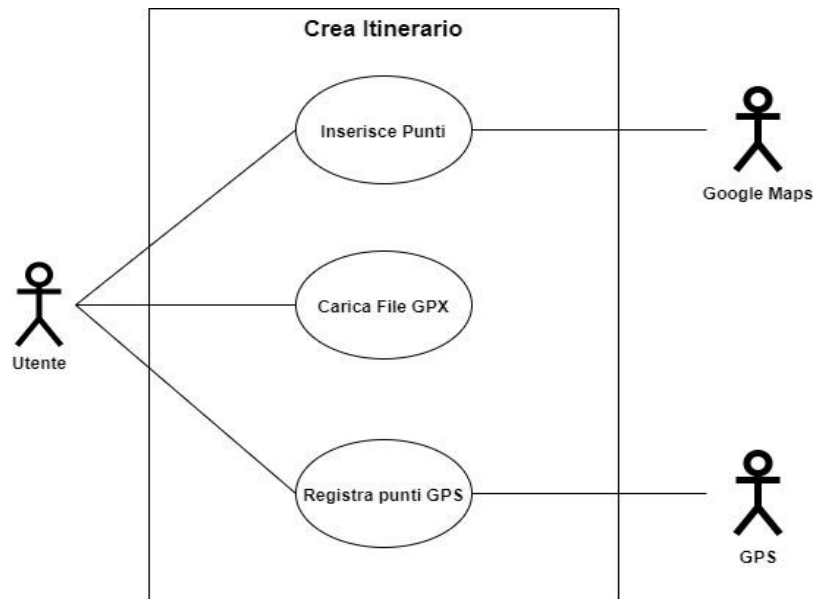
2.2.2 UTENTE REGISTRATO



UCD 2 - Utente registrato



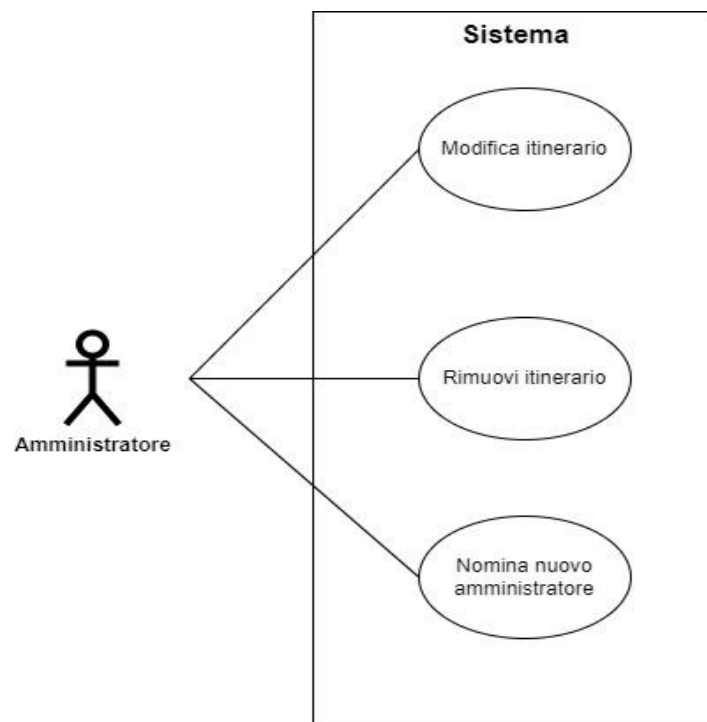
2.2.3 CREA ITINERARIO



UCD 3 - Package di creazione itinerario

2.2.4 AMMINISTRATORE

L'amministratore può svolgere tutte le funzionalità di un normale utente.



UCD 4 - Amministratore



2.3 TABELLE DI COCKBURN

Di seguito verranno riportate le tabelle di Cockburn relative a due casi d'uso significativi presenti nel precedente UCD.

I casi d'uso scelti sono la creazione di un itinerario nei tre differenti metodi e l'invio di un messaggio privato.

2.3.1 CREAZIONE DI UN ITINERARIO

USE CASE #1	Creazione itinerario		
<i>Obiettivo</i>	L'utente vuole creare un itinerario e salvarlo in remoto.		
<i>Precondizioni</i>	L'utente è iscritto alla piattaforma NaTour, ha effettuato il login e ha concesso i permessi di localizzazione.		
<i>Condizione dello stato in caso di successo</i>	L'utente ha creato un itinerario correttamente e l'ha salvato in remoto.		
<i>Condizione dello stato in caso di fallimento</i>	Il sistema non riesce a salvare l'itinerario in remoto.		
<i>Attore primario</i>	Utente.		
<i>Evento che innesca l'use case</i>	L'utente preme il pulsante di creazione nuovo itinerario nella schermata principale dell'app.		
DESCRIZIONE	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	Preme il pulsante relativo al metodo di inserimento scelto per l'itinerario.	
	3		Il sistema visualizza la schermata MOCKUP_1_1.
	4	L'utente inserisce, modifica e visualizza il percorso dell'itinerario. Quando ha finito preme il pulsante "Next".	
	5		Il sistema visualizza la schermata MOCKUP_1_2.
	6	L'utente inserisce, modifica e visualizza i dati dell'itinerario. Quando ha	



		finito preme il pulsante "Save".	
	7		Il sistema visualizza la dialog MOCKUP_1_5.
	8	L'utente preme il pulsante "Ok".	
	9		Ritorna alla schermata principale dell'app.
ESTENSIONE #1: <i>L'utente inserisce manualmente l'itinerario.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "Manually".	
	3		Il sistema visualizza la schermata MOCKUP_1_1.
	4	L'utente inserisce manualmente i punti sulla mappa. Quando ha finito preme il pulsante "Next".	
	5		Il sistema visualizza la schermata MOCKUP_1_2.
	6	L'utente inserisce manualmente i punti sulla mappa. Quando ha finito preme il pulsante "Next".	
	7		Il sistema visualizza la dialog MOCKUP_1_5.
	8	L'utente preme il pulsante "Ok".	
	9		Ritorna alla schermata principale dell'app.
SOTTO VARIAZIONE #1.1: <i>L'utente non concede i permessi di localizzazione.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.



	2	L'utente preme il pulsante "Manually".	
	3		Il sistema visualizza la schermata MOCKUP_1_1.
	4	Non viene visualizzato il pulsante di centratura sulla posizione attuale. L'utente inserisce manualmente i punti sulla mappa. Quando ha finito preme il pulsante "Next".	
	5		Il sistema visualizza la schermata MOCKUP_1_2.
	6	L'utente inserisce manualmente i punti sulla mappa. Quando ha finito preme il pulsante "Next".	
	7		Il sistema visualizza la dialog MOCKUP_1_5.
	8	L'utente preme il pulsante "Ok".	
	9		Ritorna alla schermata principale dell'app.
SOTTO VARIAZIONE #1.2: <i>L'utente non inserisce almeno un punto sulla mappa.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "Manually".	
	3		Il sistema visualizza la schermata MOCKUP_1_1.
	4	L'utente non inserisce punti sulla mappa; preme il pulsante "Next".	
	5		Il sistema mostra la dialog MOCKUP_1_3 finchè l'utente non inserisce almeno un punto.



SOTTO VARIAZIONE #1.3: <i>L'utente recupera i dati di localizzazione dell'itinerario da internet.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "Manually".	
	3		Il sistema visualizza la schermata MOCKUP_1_1.
	4	L'utente inserisce manualmente i punti sulla mappa. Quando ha finito preme il pulsante "Next".	
	5		Il sistema visualizza la schermata MOCKUP_1_2.
	6	L'utente preme il pulsante "Get Info From Internet" e vengono visualizzati i dati disponibili negli appositi campi. L'utente inserisce gli altri dati e gli eventuali dati di localizzazione mancanti. Quando ha finito preme "Save".	
	7		Il sistema visualizza la dialog MOCKUP_1_5.
	8	L'utente preme il pulsante "Ok".	
	9		Ritorna alla schermata principale dell'app.
SOTTO VARIAZIONE #1.4: <i>L'utente recupera i dati di localizzazione dell'itinerario da internet.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "Manually".	
	3		Il sistema visualizza la schermata MOCKUP_1_1.
	4	L'utente inserisce manualmente i punti sulla	



		mappa. Quando ha finito preme il pulsante "Next".	
	5		Il sistema visualizza la schermata MOCKUP_1_2.
	6	L'utente preme sul pulsante "Get Info From Internet".	
	7		Il sistema non riesce a recuperare le informazioni di localizzazione e visualizza la dialog MOCKUP_1_6.
	8	L'utente preme il pulsante "Ok".	
	9		Il sistema visualizza la schermata MOCKUP_1_2.
	10	L'utente inserisce i dati dell'itinerario. Quando ha finito preme il pulsante "Save".	
	11		Il sistema visualizza la dialog MOCKUP_1_5.
	12	L'utente preme il pulsante "Ok".	
	13		Ritorna alla schermata principale dell'app.
SOTTO VARIAZIONE #1.5: <i>Il sistema non riesce a salvare l'itinerario in remoto.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "Manually".	
	3		Il sistema visualizza la schermata MOCKUP_1_1.
	4	L'utente inserisce manualmente i punti sulla	



		mappa. Quando ha finito preme il pulsante “Next”.	
	5		Il sistema visualizza la schermata MOCKUP_1_2.
	6	L'utente inserisce i dati dell'itinerario. Quando ha finito preme il pulsante “Save”.	
	7		Il sistema non riesce a salvare l'itinerario in remoto, viene visualizzata la dialog MOCKUP_1_4.
ESTENSIONE #2: <i>L'utente inserisce l'itinerario da file .gpx.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante “From .gpx file”.	
	3		Il sistema visualizza la schermata MOCKUP_1_7.
	4	L'utente seleziona il file .gpx dalla memoria.	
	5		Il sistema visualizza la schermata MOCKUP_1_1.
	6	L'utente visualizza il percorso contenuto nel file .gpx sulla mappa. L'utente può modificare il percorso. L'utente preme il pulsante “Next”.	
	7		Il sistema visualizza la schermata MOCKUP_1_2 contenente i dati disponibili da file.
	8	L'utente visualizza i dati disponibili, li può modificare e aggiunge i dati mancanti. Preme il bottone “Save”.	



	9		Il sistema visualizza la dialog MOCKUP_1_5.
	10	L'utente preme il pulsante "Ok".	
	11		Ritorna alla schermata principale dell'app.
SOTTO VARIAZIONE #2.1: <i>L'utente seleziona un file .gpx non valido.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "From .gpx file".	
	3		Il sistema mostra la schermata MOCKUP_1_7.
	4	L'utente seleziona il file .gpx dalla memoria.	
	5		Il file ha un formato non valido, il sistema visualizza la schermata MOCKUP_1_8.
	6	L'utente preme "Ok".	
SOTTO VARIAZIONE #2.2: <i>Il file .gpx contiene più itinerari.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "From .gpx file".	
	3		Il sistema visualizza la schermata MOCKUP_1_7.
	4	L'utente seleziona il file .gpx dalla memoria.	
	5		Il sistema visualizza la schermata MOCKUP_1_13.



	6	L'utente seleziona l'itinerario tra quelli disponibili e preme il pulsante "Next".	
	7		Il sistema visualizza la schermata MOCKUP_1_1.
	8	L'utente visualizza il percorso contenuto nel file .gpx sulla mappa. L'utente può modificare il percorso. L'utente preme il pulsante "Next".	
	9		Il sistema visualizza la schermata MOCKUP_1_2 contenente i dati disponibili da file.
	10	L'utente visualizza i dati disponibili, li può modificare e aggiunge i dati mancanti. Preme il bottone "Save".	
	11		Il sistema visualizza la dialog MOCKUP_1_5.
	12	L'utente preme il pulsante "Ok".	
	13		Il sistema ritorna alla schermata principale dell'app.
SOTTO VARIAZIONE #2.3: <i>L'utente seleziona un file .gpx con un formato XML non valido.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "From .gpx file".	
	3		Il sistema visualizza la schermata MOCKUP_1_7.
	4	L'utente seleziona il file .gpx dalla memoria.	
	5		Il file ha un formato non valido. Il sistema visualizza la schermata MOCKUP_1_9.



	6	L'utente preme "Ok".	
SOTTO VARIAZIONE #2.4: <i>L'utente cancella tutti i punti sulla mappa.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "From .gpx file".	
	3		Il sistema visualizza la schermata MOCKUP_1_7.
	4	L'utente seleziona il file .gpx dalla memoria.	
	5		Il sistema visualizza la schermata MOCKUP_1_1.
	6	L'utente visualizza il percorso contenuto nel file .gpx sulla mappa. L'utente cancella tutti i punti del percorso. L'utente preme il pulsante "Next".	
	7		Il sistema visualizza la dialog MOCKUP_1_3 finché l'utente non inserisce sulla mappa almeno un punto.
SOTTO VARIAZIONE #2.5: <i>L'utente recupera i dati di localizzazione dell'itinerario da internet.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "From .gpx file".	
	3		Il sistema visualizza la schermata MOCKUP_1_7.
	4	L'utente seleziona il file .gpx dalla memoria.	
	5		Il sistema visualizza la schermata MOCKUP_1_1.



	6	L'utente visualizza il percorso contenuto nel file .gpx sulla mappa. L'utente può modificare il percorso. L'utente preme il pulsante "Next".	
	7		Il sistema visualizza la schermata MOCKUP_1_2 contenente i dati disponibili da file.
	8	L'utente preme il pulsante "Get info from internet" e vengono visualizzati i dati disponibili negli appositi campi. L'utente inserisce gli altri dati e gli eventuali dati di localizzazione mancanti. Quando ha finito preme il pulsante "Save".	
	9		Il sistema visualizza la dialog MOCKUP_1_5.
	10	L'utente preme il pulsante "Ok".	
	11		Il sistema ritorna alla schermata principale dell'app.
SOTTO VARIAZIONE #2.6: <i>Il sistema non riesce a recuperare i dati di localizzazione su internet.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "From .gpx file".	
	3		Il sistema visualizza la schermata MOCKUP_1_7.
	4	L'utente seleziona il file .gpx dalla memoria.	
	5		Il sistema visualizza la schermata MOCKUP_1_1.
	6	L'utente visualizza il percorso contenuto nel file .gpx sulla mappa. L'utente può modificare il percorso.	



		L'utente preme il pulsante "Next".	
	7		Il sistema visualizza la schermata MOCKUP_1_2 contenente i dati disponibili da file.
	8	L'utente visualizza i dati disponibili, modifica i dati di localizzazione premendo il tasto "Get info from internet".	
	9		Il sistema non riesce a recuperare le informazioni di localizzazione e visualizza la dialog MOKCUP_1_6.
	10	L'utente preme il pulsante "Ok".	
	11		Il sistema visualizza la schermata MOCKUP_1_2.
	12	L'utente inserisce i dati dell'itinerario. Quando ha finito preme il pulsante "Save".	
	13		Il sistema visualizza la dialog MOCKUP_1_5.
	14	L'utente preme il pulsante "Ok".	
	15		Il sistema ritorna alla schermata principale dell'app.
SOTTO VARIAZIONE #2.7: <i>Il sistema non riesce a salvare in remoto l'itinerario.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "From .gpx file".	
	3		Il sistema visualizza la schermata MOCKUP_1_7.



	4	L'utente seleziona il file .gpx dalla memoria.	
	5		Il sistema visualizza la schermata MOCKUP_1_1.
	6	L'utente visualizza il percorso contenuto nel file .gpx sulla mappa. L'utente può modificare il percorso. L'utente preme il pulsante "Next".	
	7		Il sistema visualizza la schermata MOCKUP_1_2 contenente i dati disponibili da file.
	8	L'utente visualizza i dati disponibili, li può modificare e aggiunge i dati mancanti. Preme il bottone "Save".	
	9		Il sistema non riesce a salvare l'itinerario in remoto, viene visualizzata la dialog MOCKUP_1_4.
ESTENSIONE #3: <i>L'utente inserisce tramite tracking GPS.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "By GPS tracking".	
	3		Il sistema visualizza il MOCKUP_1_1 con visualizzazione del pulsante per il comando della registrazione dei punti.
	4	L'utente preme il pulsante per la registrazione.	
	5		Il sistema inizia la registrazione dei punti e mostra la notifica MOCKUP_1_10.



	6	L'utente preme il tasto per fermare la registrazione.	
	7		Il sistema ferma la registrazione dei punti e mostra la notifica MOCKUP_1_10.
	8	L'utente preme il tasto "Next".	
	9		Il sistema visualizza la schermata MOCKUP_1_2.
	10	L'utente inserisce i dati dell'itinerario. Quando ha finito preme il pulsante "Save".	
	11		Il sistema visualizza la dialog MOCKUP_1_5.
	12	L'utente preme il pulsante "Ok".	
	13		Il sistema ritorna alla schermata principale dell'app.
SOTTO VARIAZIONE #3.1: <i>L'utente non concede i permessi di localizzazione.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "By GPS tracking".	
	3		Il sistema visualizza la dialog MOCKUP_1_11.
	4	L'utente preme il pulsante "Yes".	
	5		Il sistema mostra la dialog standard per la richiesta dei permessi di localizzazione.
	6	L'utente non concede i permessi di localizzazione.	
	7		Il sistema visualizza il MOCKUP_1_1 con visualizzazione



			del pulsante per il comando della registrazione dei punti.
	8	L'utente preme il pulsante per iniziare la registrazione.	
	9		Il sistema inizia la registrazione dei punti e mostra la notifica MOCKUP_1_10.
	10	L'utente preme il pulsante per fermare la registrazione.	
	11		Il sistema ferma la registrazione dei punti e nasconde la notifica MOCKUP_1_10.
	12	L'utente preme il tasto "Next".	
	13		Il sistema visualizza la schermata MOCKUP_1_2.
	14	L'utente inserisce i dati dell'itinerario. Quando ha finito preme il pulsante "Save".	
	15		Il sistema visualizza la dialog MOCKUP_1_5.
	16	L'utente preme il pulsante "Ok".	
	17		Il sistema ritorna alla schermata principale dell'app.
<i>SOTTO VARIAZIONE #3.1.1: L'utente non concede i permessi di localizzazione.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "By GPS tracking".	
	3		Il sistema visualizza la dialog MOCKUP_1_11.



	4	L'utente preme il tasto "Return".	
	5		Il sistema visualizza nuovamente la schermata MOCKUP_1_o.
SOTTO VARIAZIONE #3.1.2: <i>L'utente non concede i permessi di localizzazione.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_o.
	2	L'utente preme il pulsante "By GPS tracking".	
	3		Il sistema visualizza la dialog MOCKUP_1_11.
	4	L'utente preme il pulsante "Yes".	
	5		Il sistema mostra la dialog standard per la richiesta dei permessi di localizzazione.
	6	L'utente nega i permessi di localizzazione.	
	7		Il sistema visualizza la dialog MOCKUP_1_12.
	8	L'utente preme il pulsante "Ok".	
	9		Il sistema mostra nuovamente la schermata MOCKUP_1_o.
SOTTO VARIAZIONE #3.2: <i>L'utente non inserisce, non registra o cancella tutti i punti registrati.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_o.
	2	L'utente preme il pulsante "By GPS tracking".	
	3		Il sistema visualizza il MOCKUP_1_1 con visualizzazione del pulsante per il comando della



			registrazione dei punti.
	4	L'utente non registra punti o cancella tutti i punti registrati. Preme il tasto "Next".	
	5		Il sistema visualizza la dialog MOCKUP_1_3 finchè sulla mappa non è presente almeno un punto.
SOTTO VARIAZIONE #3.3: <i>L'utente recupera i dati di localizzazione dell'itinerario da internet.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "By GPS tracking".	
	3		Il sistema visualizza il MOCKUP_1_1 con visualizzazione del pulsante per il comando della registrazione dei punti.
	4	L'utente preme il pulsante per la registrazione.	
	5		Il sistema inizia la registrazione dei punti e mostra la notifica MOCKUP_1_10.
	6	L'utente preme il pulsante per fermare la registrazione.	
	7		Il sistema ferma la registrazione dei punti e nasconde la notifica MOCKUP_1_10.
	8	L'utente preme il tasto "Next".	
	9		Il sistema visualizza la schermata MOCKUP_1_2.
	10	L'utente preme il pulsante "Get info from internet" e vengono visualizzati i dati	



		disponibili negli appositi campi. L'utente inserisce gli altri dati e gli eventuali dati di localizzazione mancanti. Quando ha finito preme il pulsante "Save".	
	11		Il sistema visualizza la dialog MOCKUP_1_5.
	12	L'utente preme il pulsante "Ok".	
	13		Il sistema ritorna alla schermata principale dell'app.
SOTTO VARIAZIONE #3.4: <i>Il sistema non riesce a recuperare i dati di localizzazione da internet.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "By GPS tracking".	
	3		Il sistema visualizza il MOCKUP_1_1 con visualizzazione del pulsante per il comando della registrazione dei punti.
	4	L'utente preme il pulsante per la registrazione.	
	5		Il sistema inizia la registrazione dei punti e mostra la notifica MOCKUP_1_10.
	6	L'utente preme il pulsante per fermare la registrazione.	
	7		Il sistema ferma la registrazione dei punti e nasconde la notifica MOCKUP_1_10.
	8	L'utente preme il tasto "Next".	
	9		Il sistema visualizza la schermata MOCKUP_1_2.



	10	L'utente preme il pulsante "Get info from internet".	
	11		Il sistema non riesce a recuperare le informazioni di localizzazione e visualizza la dialog MOCKUP_1_6.
	12	L'utente preme il pulsante "Ok".	
	13		Il sistema riporta in primo piano MOCKUP_1_2.
	14	L'utente inserisce i dati dell'itinerario. Quando ha finito preme il pulsante "Save".	
	15		Il sistema visualizza la dialog MOKCUP_1_5.
	16	L'utente preme il pulsante "Ok".	
	17		Il sistema ritorna alla schermata principale dell'app.
SOTTO VARIAZIONE #3.5: <i>Il sistema non riesce a salvare in remoto l'itinerario.</i>	STEP	UTENTE	SISTEMA
	1		Il sistema visualizza la schermata MOCKUP_1_0.
	2	L'utente preme il pulsante "By GPS tracking".	
	3		Il sistema visualizza il MOCKUP_1_1 con visualizzazione del pulsante per il comando della registrazione dei punti.
	4	L'utente preme il pulsante per la registrazione.	
	5		Il sistema inizia la registrazione dei punti e mostra la notifica MOCKUP_1_10.



	6	L'utente preme il pulsante per fermare la registrazione.	
	7		Il sistema ferma la registrazione dei punti e nasconde la notifica MOCKUP_1_10.
	8	L'utente preme il tasto "Next".	
	9		Il sistema visualizza la schermata MOCKUP_1_2.
	10	L'utente inserisce i dati dell'itinerario. Quando ha finito preme il pulsante "Save".	
	11		Il sistema non riesce a salvare l'itinerario in remoto, viene visualizzata la dialog MOCKUP_1_4.



2.3.2 INVIO DI UN MESSAGGIO

USE CASE #2	Invio di un messaggio ad un utente.		
<i>Obiettivo</i>	L'utente o l'admin vuole inviare un messaggio ad un determinato utente		
<i>Precondizioni</i>	L'utente o l'admin deve aver effettuato l'accesso all'applicazione.		
<i>Condizione dello stato in caso di successo</i>	L'utente o l'admin ha inviato un messaggio all'utente scelto.		
<i>Condizione dello stato in caso di fallimento</i>	L'utente e/o l'admin non ha inviato un messaggio all'utente scelto.		
<i>Attore primario</i>	Utente/Admin		
<i>Evento che innesca l'UseCase</i>	L'utente o admin clicca sulla tab dei messaggi		
DESCRIZIONE	STEP	UTENTE/ADMIN	SISTEMA
	1		Il sistema mostra MOCKUP_2_07
	2	L'utente scorre e seleziona tra gli elementi della lista di utenti con cui ha già avuto una conversazione in precedenza	
	3		Il sistema mostra MOCKUP_2_05
	4	L'utente inserisce nella barra d'inserimento il messaggio che vuole mandare	
	5	L'utente clicca sul bottone per inviare un messaggio	
	6		Il sistema aggiunge il messaggio alla conversazione del MOCKUP_2_05 e termina l'Use Case.
ESTENSIONI	STEP	UTENTE/ADMIN	SISTEMA
ESTENSIONE #1: <i>L'utente che non ha conversazioni passate.</i>	3.1		Il sistema mostra il messaggio che indica la non presenza di conversazioni già avute in passato



<i>ESTENSIONE #2:</i> <i>Errore nell'invio del messaggio.</i>	6.2		Il sistema mostra MOCKUP_2_o6
	7.2	L'utente clicca sul bottone OK	
	8.2		Il sistema torna a step 3 dello scenario primario
<i>ESTENSIONE #3:</i> <i>Controllo messaggio vuoto.</i>	5.3	L'utente clicca sul bottone per inviare un messaggio (rimanendo barra d'inserimento vuota)	
	6.3		Il sistema controlla il messaggio vuoto e annulla l'azione dell'utente, ritorno allo step 3 dello scenario primario
<i>ESTENSIONE #4:</i> <i>Risultato non trovato nella sotto variazione 1.</i>	8.4.1		Il sistema mostra il MOCKUP_2_o9
	8.5.1	L'utente clicca sul bottone Ok	
	8.6.1		Il sistema ricollega allo Step 5 della Sotto variazione 1
<i>ESTENSIONE #5:</i> <i>Un admin che effettua la sotto variazione 1.</i>	10.4.2		Il sistema mostra il MOCKUP_2_o4
	11.4.2	L'admin clicca sul bottone Send Message	
	12.4.2		Il sistema ricollega allo Step 3 dello scenario primario.
SOTTO VARIAZIONI	STEP	UTENTE/ADMIN	SISTEMA
<i>SOTTO VARIAZIONE #1:</i> <i>Un utente normale ricerca la persona con la quale vuole comunicare.</i>	2.4	L'utente clicca sul bottone della ricerca.	
	3.4		Il sistema mostra il MOCKUP_2_o1.
	4.4	L'utente clicca sulla tab degli utenti cercati.	
	5.4		Il sistema mostra il MOCKUP_2_o2.
	6.4	L'utente inserisce nome, cognome o l'e-mail dell'utente con il quale vuole comunicare.	



	7.4	L'utente clicca sul bottone ricerca.	
	8.4		Il sistema aggiunge alla lista di utenti del MOCKUP_2_o2 la lista degli utenti trovati secondo il criterio di ricerca.
	9.4	L'utente scorre e seleziona l'utente con il quale vuole comunicare.	
	10.4		Il sistema mostra il MOCKUP_2_o8.
	11.4	L'utente clicca sul bottone Send Message.	
	12.4		Il sistema ricollega allo Step 3 dello scenario primario.



2.4 PROTOTIPAZIONE VISUALE

Di seguito verranno riportati i mock-up dell'interfaccia grafica dei casi d'uso precedenti, descritti tramite tabelle di Cockburn.

I mock-up denominati "MOCKUP_1" rappresentano lo Use Case #1, della creazione dell'itinerario.

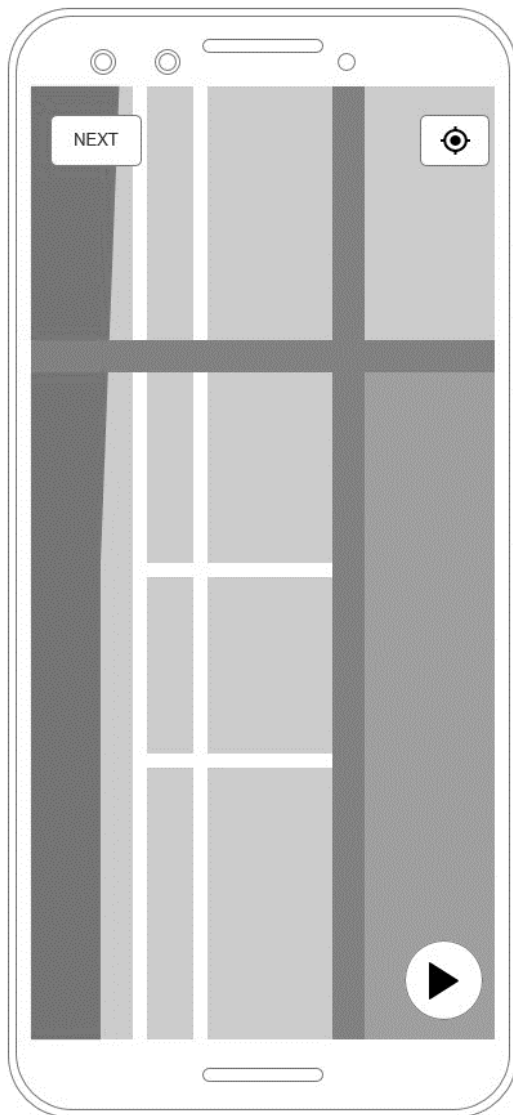
I mock-up denominati "MOCKUP_2" rappresentano lo Use Case #2, dell'invio dei messaggi.

I mock-up sono stati realizzati con il software *AxureRPro*®.

2.4.1 MOCK-UP CREAZIONE ITINERARIO

MOCKUP_1_o	
	In questa schermata sono presenti tre pulsanti: • "Manually" per la creazione manuale dell'itinerario; • "From .gpx file" per la creazione tramite file .gpx; • "By GPS tracking" per la creazione dell'itinerario registrando la propria posizione.



MOCKUP_1_1

In questa schermata è presente:

- Il pulsante “Next” per procedere con l’inserimento dei dati;
- Il pulsante di centro posizione situato in alto a destra mostra sulla mappa la posizione dell’utente in maniera più precisa;
- Il pulsante di registrazione per iniziare la sessione di tracking GPS e inserire i punti usando l’aggiornamento della posizione.



MOCKUP_1_2

Title

State

Region

City

GET INFO FROM INTERNET

Accessibility for disabled people ☒

Private Itinerary ☒

Estimated time:

Route length:

Difficulty

★★★★★

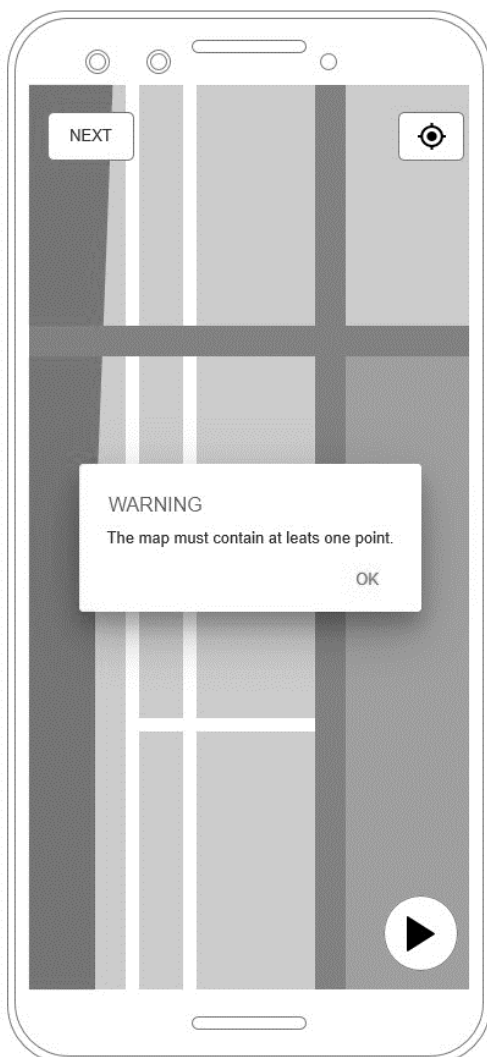
Description

SAVE

In questa schermata sono presenti:

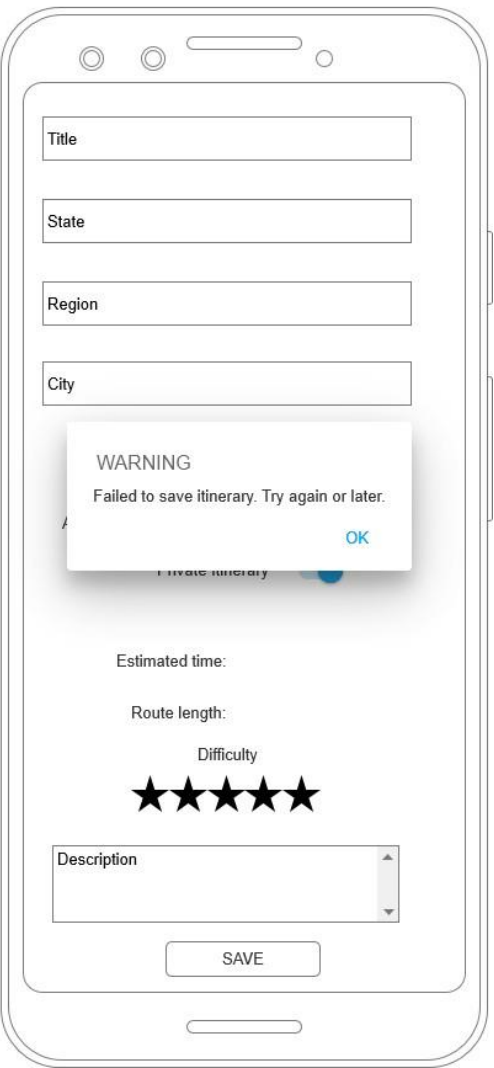
- Cinque campi di testo, per inserire i dettagli dell'itinerario, quali titolo, stato, regione, città e la descrizione;
- Un pulsante "Get Info From Internet" per recuperare le informazioni come State, Region e City da internet;
- Uno switch per decidere se l'itinerario è accessibile ai disabili;
- Uno switch per decidere se l'itinerario è privato o pubblico;
- Due etichette, che segnano la lunghezza del percorso e il tempo di percorrenza stimato;
- Un rating bar, per segnare la difficoltà dell'itinerario;
- Un pulsante "Save" per salvare l'itinerario in remoto.



MOCKUP_1_3

Questa schermata contiene la dialog di errore che appare quando l'utente non inserisce nessun punto sulla mappa, e clicca sul pulsante "Next".



MOCKUP_1_4

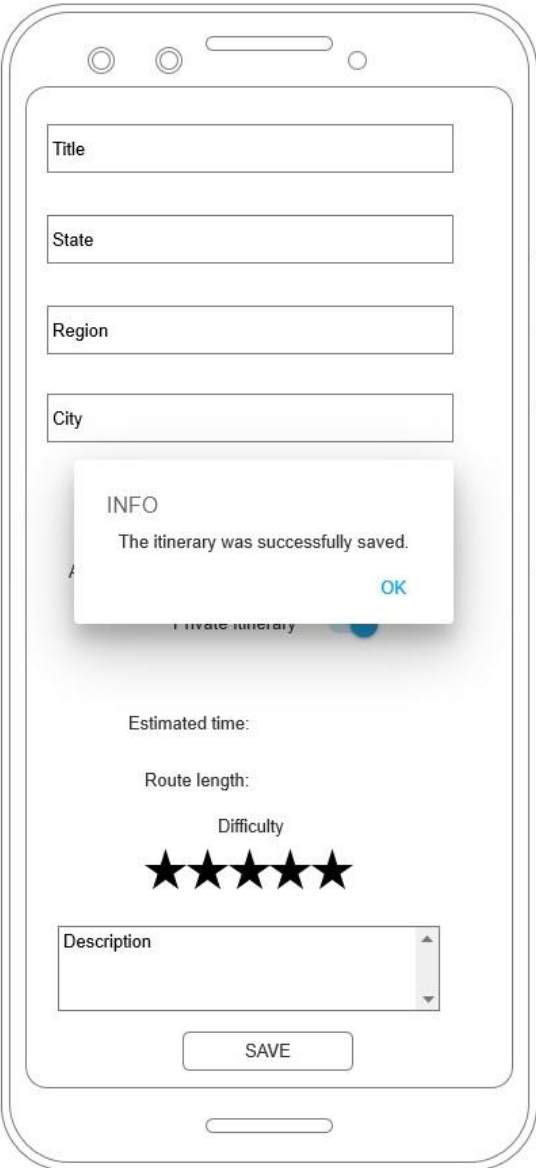
The mockup shows a mobile app interface with a warning dialog box. The dialog box is titled "WARNING" and contains the text "Failed to save itinerary. Try again or later." with an "OK" button. The background form has the following elements:

- Title
- State
- Region
- City
- Estimated time:
- Route length:
- Difficulty
- Five stars rating
- Description
- SAVE button

Questa schermata contiene la dialog di errore che appare quando il salvataggio in remoto dell'itinerario fallisce.

La dialog appare dopo la pressione del pulsante "Save".



MOCKUP_1_5

This mockup illustrates a mobile application interface for saving an itinerary. The screen features a form with the following elements:

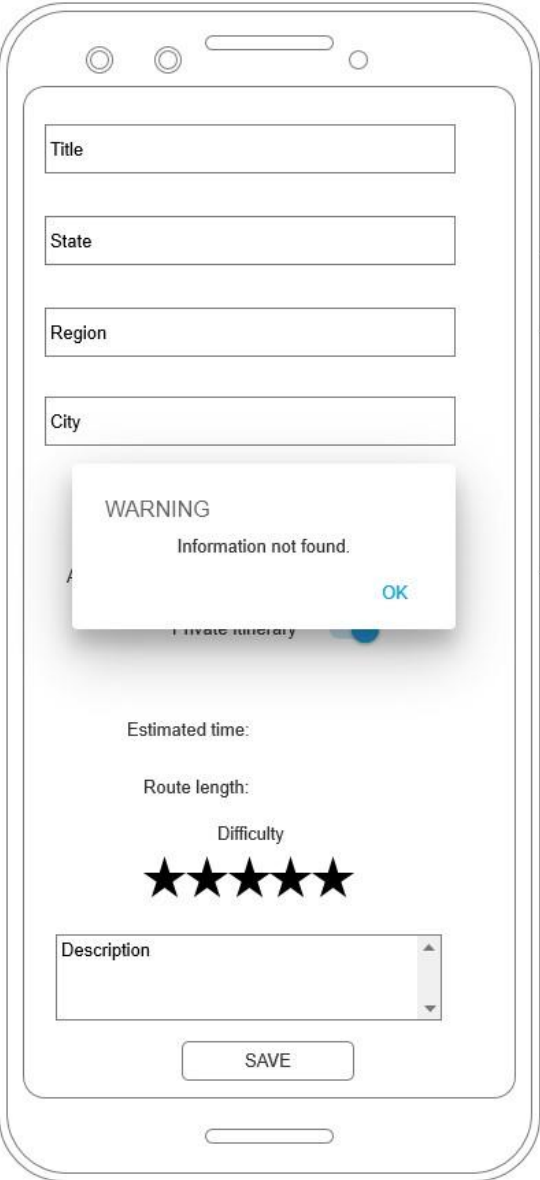
- Title**: A text input field.
- State**: A text input field.
- Region**: A text input field.
- City**: A text input field.
- Estimated time**: A label.
- Route length**: A label.
- Difficulty**: A label above a row of five stars.
- Description**: A text area with a vertical scrollbar.
- SAVE**: A button at the bottom of the form.

An **INFO** dialog box is shown in the center, indicating that the itinerary was successfully saved. The dialog contains the text "The itinerary was successfully saved." and an **OK** button.

Questa schermata contiene la dialog che informa l'utente che il salvataggio in remoto dell'itinerario è riuscito correttamente.

La dialog appare dopo la pressione del pulsante "Save".



MOCKUP_1_6

The mockup shows a mobile app interface with a warning dialog box. The dialog box is titled "WARNING" and contains the text "Information not found." with an "OK" button. The background interface includes input fields for "Title", "State", "Region", and "City". Below these fields, there is a "SAVE" button. Further down, there are labels for "Estimated time:", "Route length:", and "Difficulty", followed by five stars. At the bottom, there is a "Description" text area.

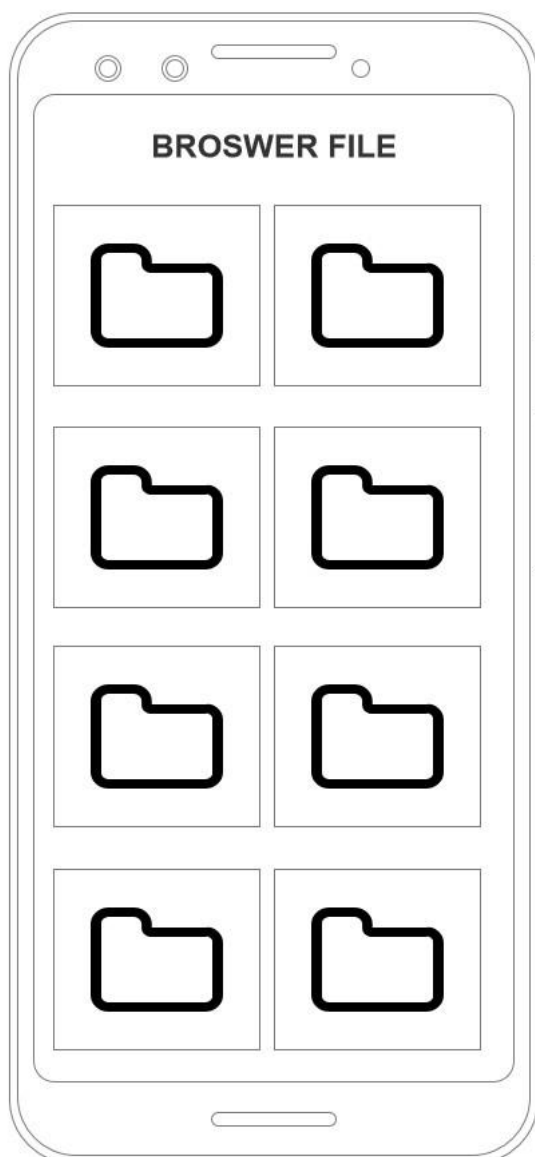
WARNING
Information not found.
OK

Estimated time:
Route length:
Difficulty
★★★★★
Description
SAVE

Questa schermata contiene la dialog che informa l'utente che non sono stati trovati i dati di stato, regione e città da internet.

La dialog appare dopo la pressione del pulsante "Get Info From Internet".



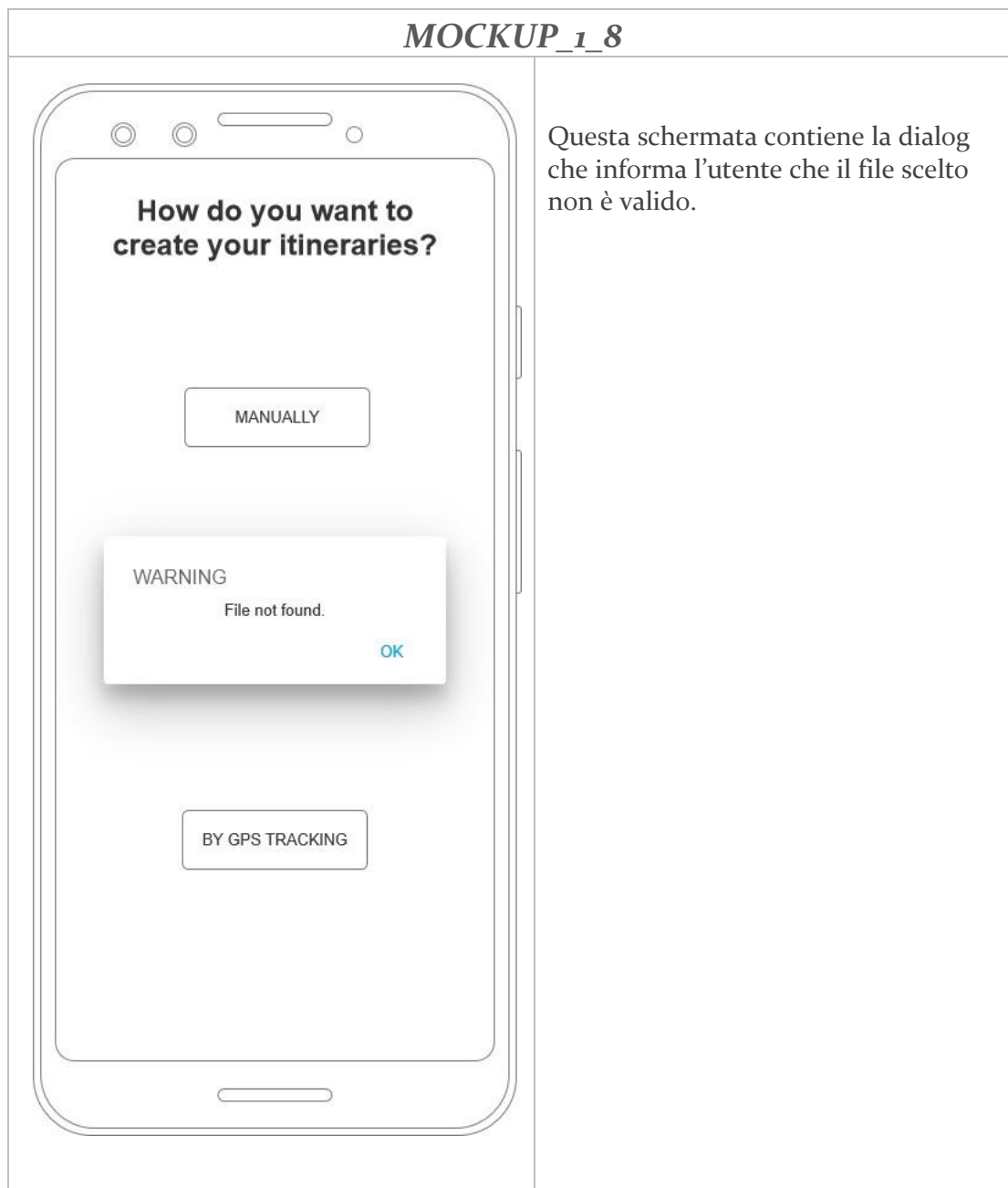
MOCKUP_1_7

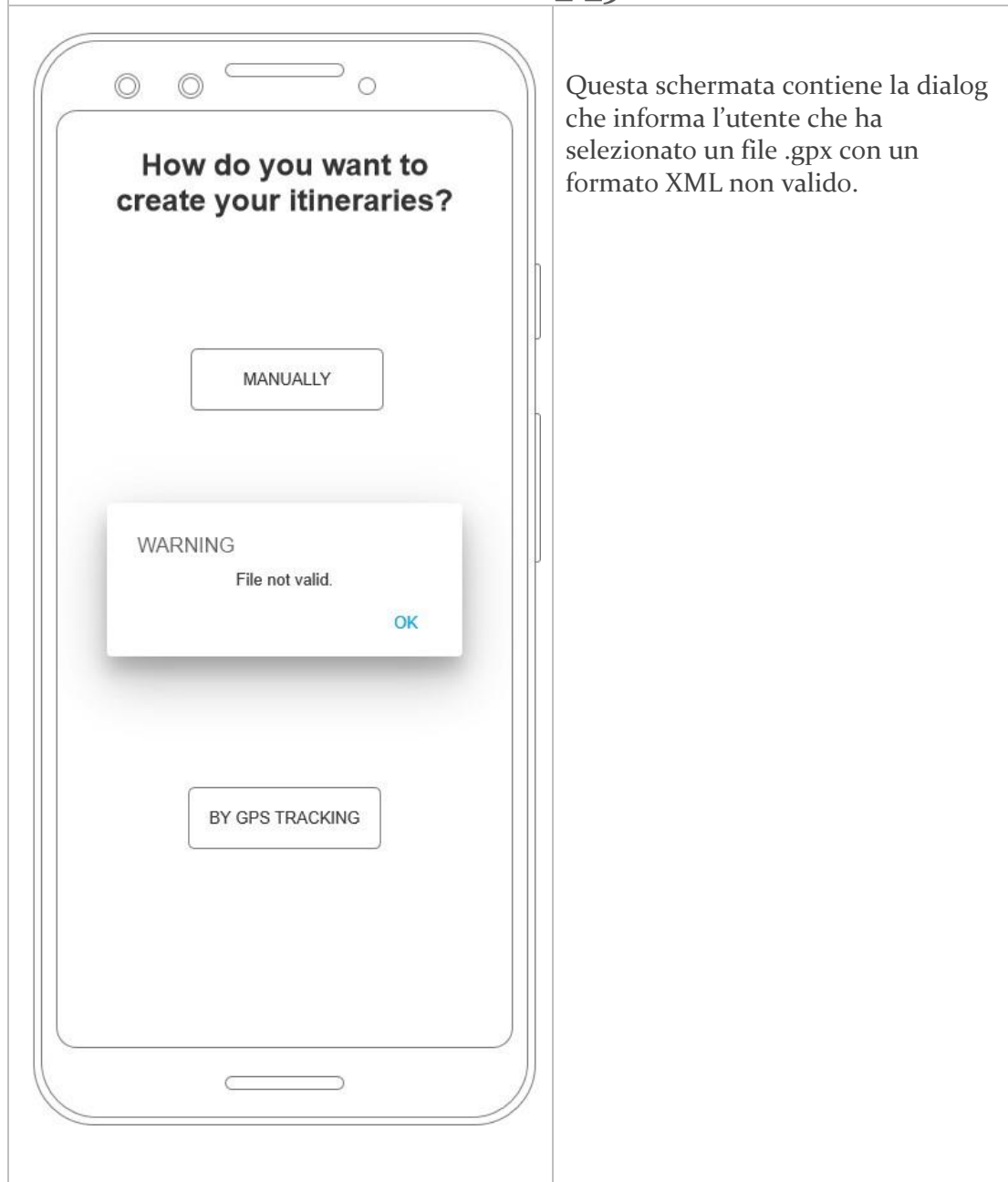
Questa schermata rappresenta il Browser File di uno smartphone Android.

Questa schermata viene visualizzata quando l'utente decide di creare un itinerario tramite file .gpx.

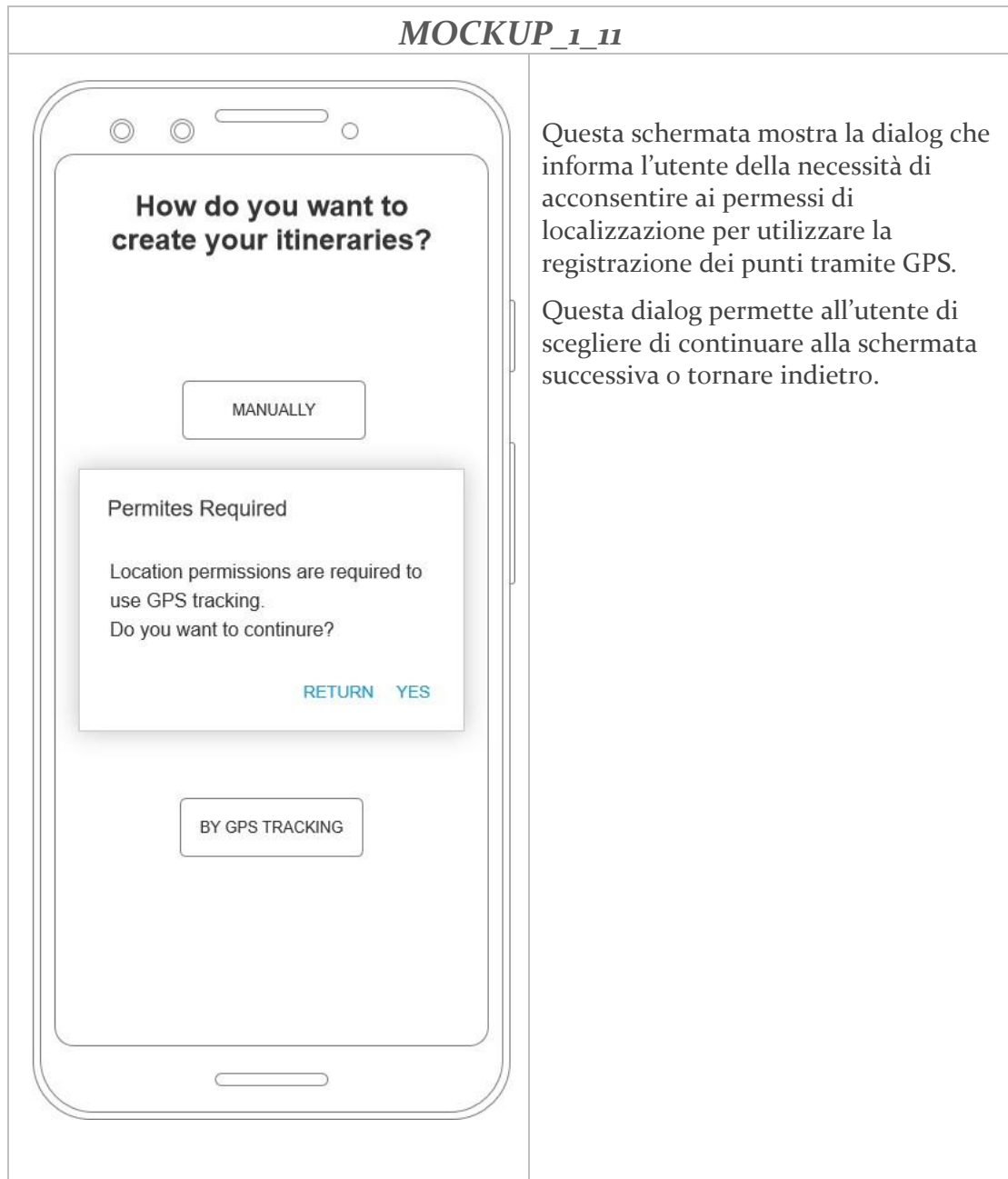


MOCKUP_1_8



MOCKUP_1_9

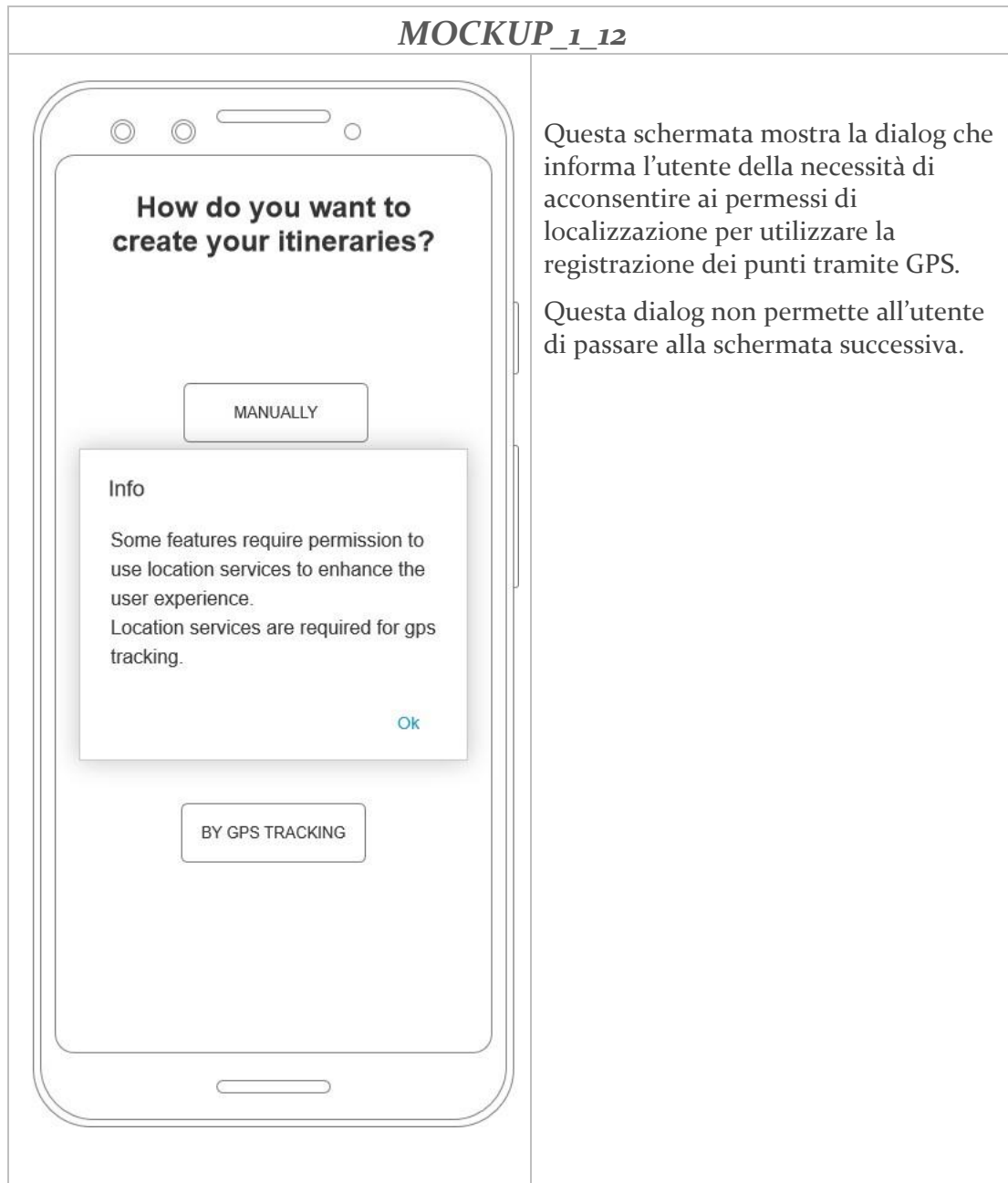


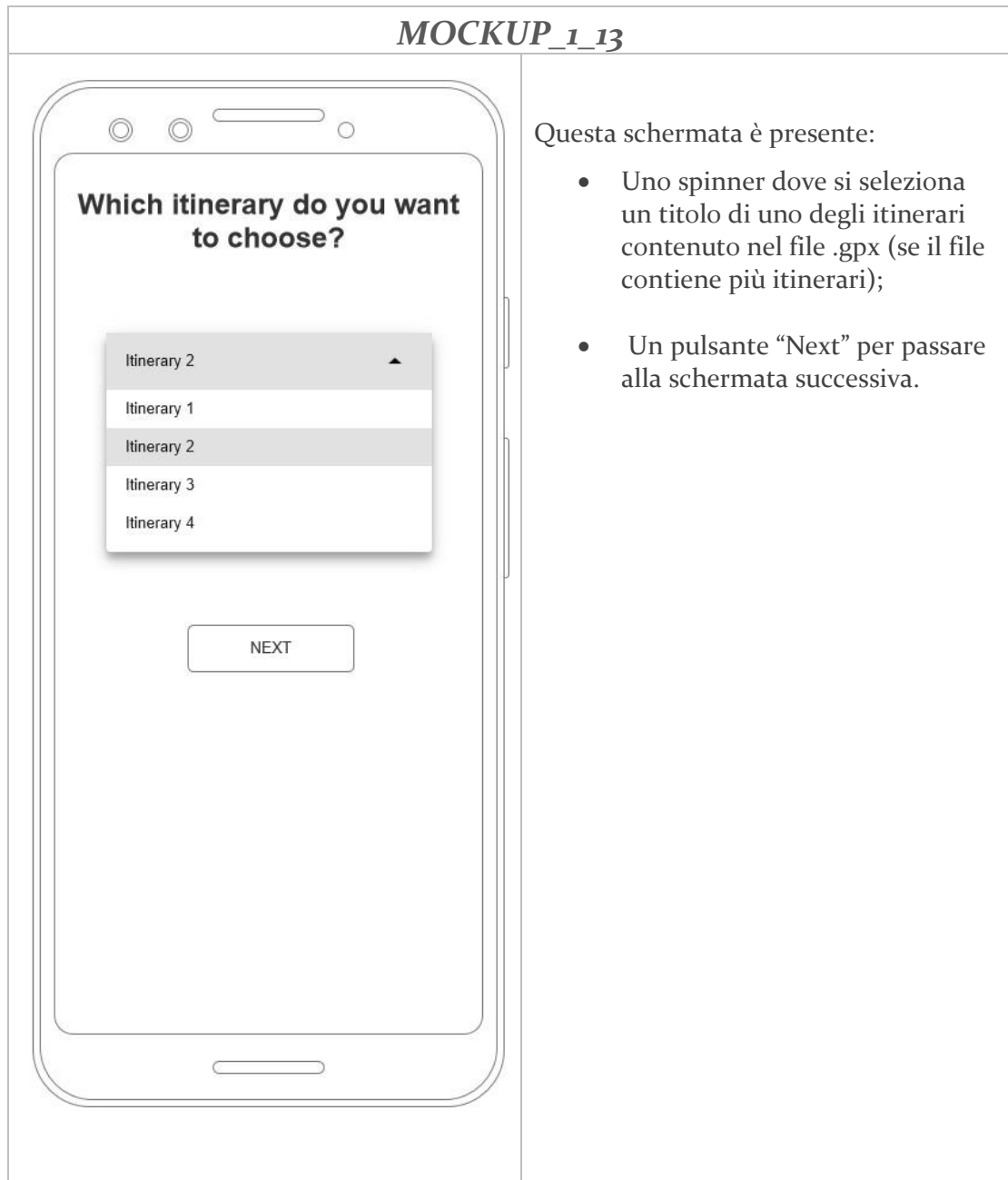
MOCKUP_1_11

Questa schermata mostra la dialog che informa l'utente della necessità di acconsentire ai permessi di localizzazione per utilizzare la registrazione dei punti tramite GPS.

Questa dialog permette all'utente di scegliere di continuare alla schermata successiva o tornare indietro.



MOCKUP_1_12

MOCKUP_1_13

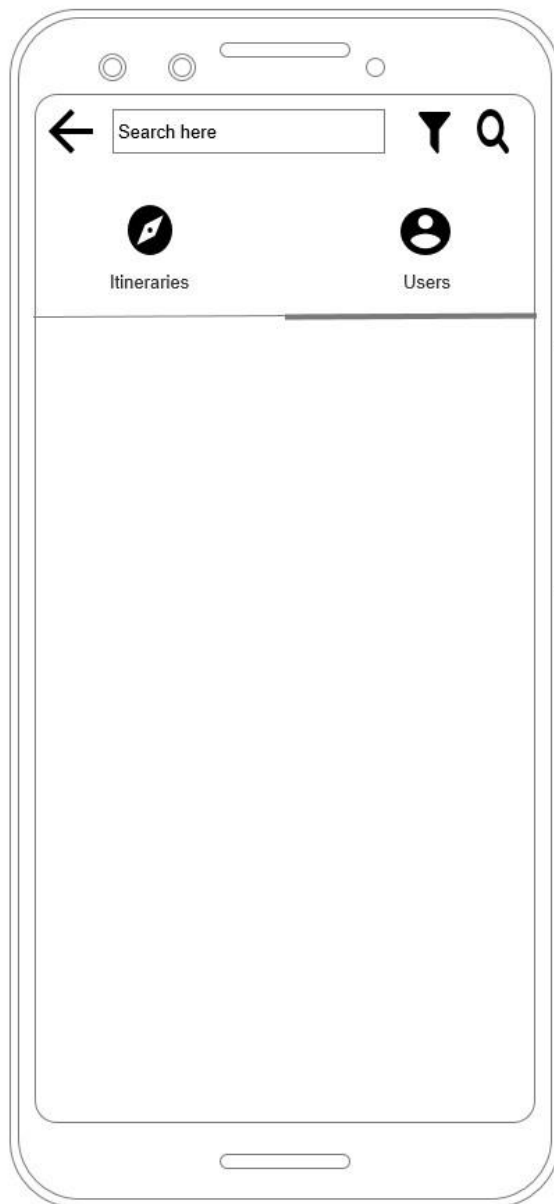
Questa schermata è presente:

- Uno spinner dove si seleziona un titolo di uno degli itinerari contenuto nel file .gpx (se il file contiene più itinerari);
- Un pulsante “Next” per passare alla schermata successiva.



2.4.2 MOCK-UP INVIO MESSAGGIO



MOCKUP_2_2

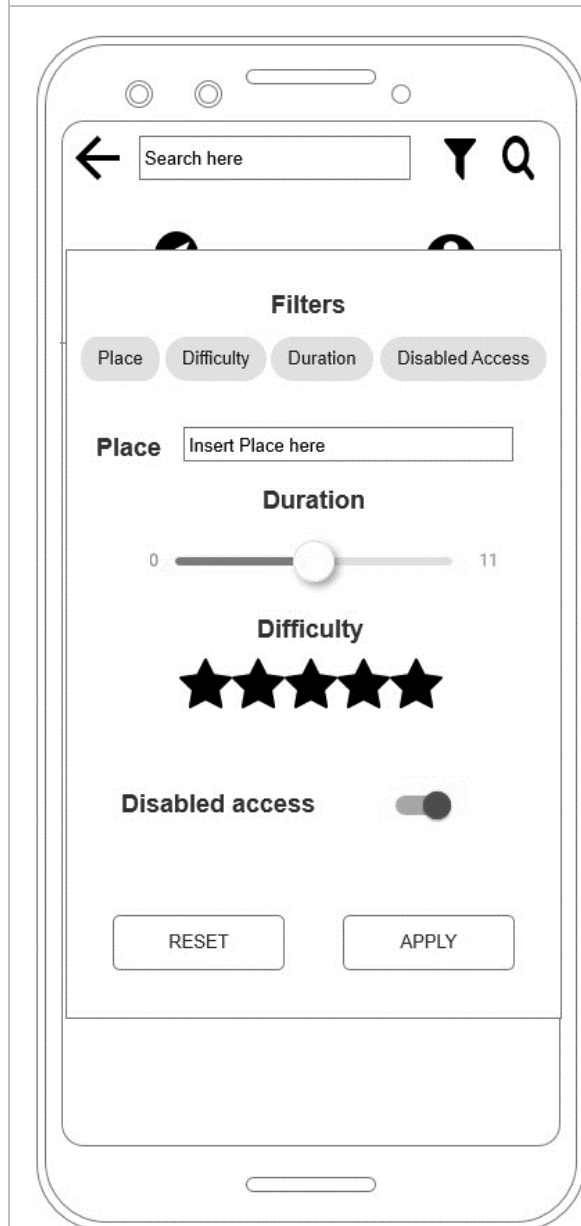
Questa schermata rappresenta la ricerca, divisa per itinerari e utenti.

In questa schermata è presente:

- Un campo di testo, dove inserire la stringa da ricercare;
- L'icona di filtro, che mostra la schermata dei filtri disponibili;
- L'icona di ricerca, che inizia la ricerca.

La schermata mostra in primo piano gli utenti, come si può notare dall'indicatore presente sotto la scritta "Users".



MOCKUP_2_3

Questa schermata mostra i filtri disponibili all'utente.

In questa schermata è presente:

- Dei chip di selezione filtro, con i quali l'utente può decidere su cosa basare la propria ricerca;
- Un campo di testo dove inserire il posto che si vuole cercare;
- Uno slider per decidere la durata massima del percorso;
- Una rating bar per decidere la difficoltà massima del percorso;
- Uno switch per decidere se il percorso che si cerca dev'essere accessibile ai disabili.

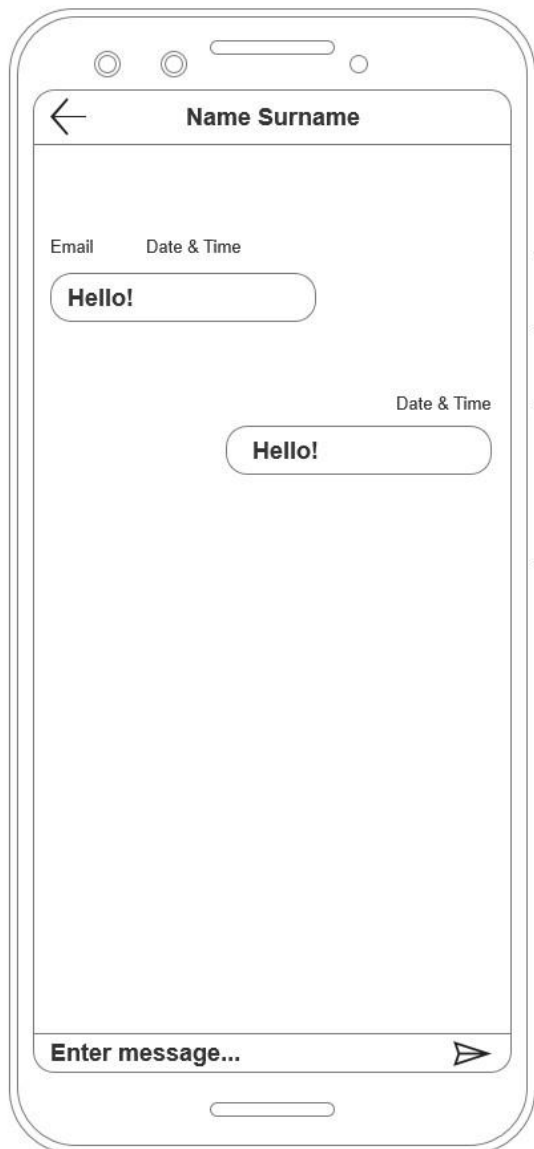


MOCKUP_2_4

Questa schermata mostra la dialog che un amministratore vede quando, dopo aver effettuato la ricerca di un utente, preme sul nome nella lista dei risultati.

L'amministratore sarà in grado di mandare un messaggio all'utente desiderato oppure di promuoverlo ad amministratore.



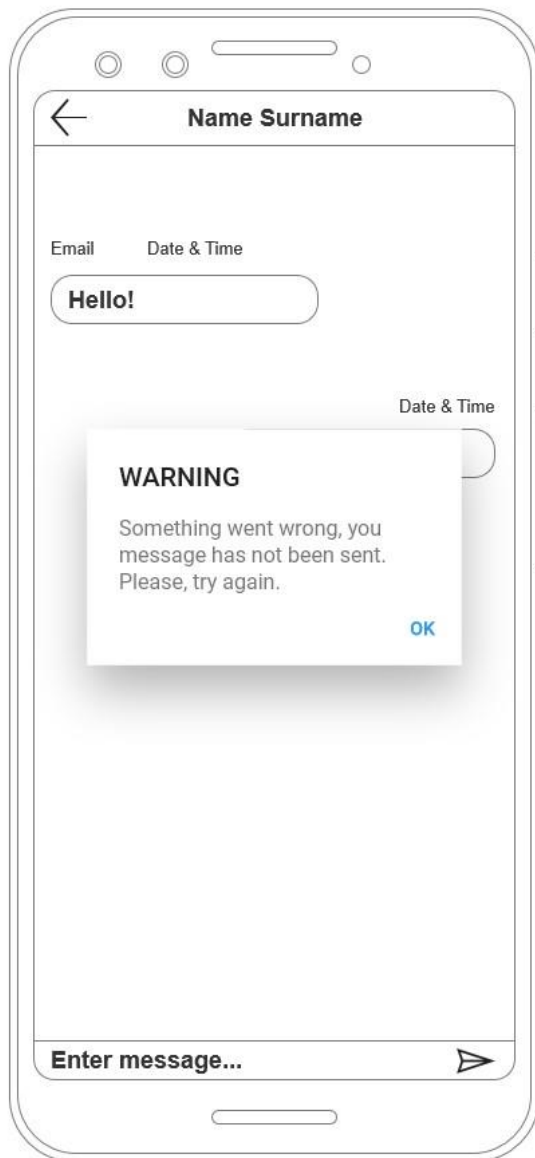
MOCKUP_2_5

Questa schermata mostra una conversazione tra due utenti.

I messaggi sono caratterizzati da una data ed un orario, e sarà possibile vedere l'email dell'utente con cui si parla sopra il messaggio.

In basso alla schermata, invece, è presente un campo di testo dove l'utente inserirà il proprio messaggio e un pulsante di invio.



MOCKUP_2_6

Questa schermata mostra la dialog di errore che informa l'utente della presenza di un errore e che il messaggio non è stato correttamente inviato.



MOCKUP_2_7

Questa schermata rappresenta la lista di tutte le conversazioni di un utente.

Questa schermata si trova nella parte principale dell'applicazione.



2.5 PRESENTAZIONE DELL'IDEA PROGETTUALE

NaTour è un social network per appassionati di escursioni e per persone in cerca di avventura, basata sulla creazione di itinerari e la condivisione degli stessi.

L'idea alla base di NaTour è quella di un social network semplice da usare, che metta al centro dell'esperienza l'itinerario e tutto ciò che gli riguarda: creazione, modifica e condivisione.

NaTour ha un'interfaccia semplice ed intuitiva da utilizzare, suddivisa in sezioni di facile accesso.

L'ispirazione è quella delle moderne applicazioni di social network, cioè un prodotto con funzionalità e meccaniche simili a quelle adottate dai migliori competitor in circolazione, in modo da garantire all'utente un punto di riferimento su cui iniziare la propria esperienza.

Inoltre, grazie alla possibilità di inviare e ricevere messaggi NaTour offre la possibilità agli utenti di condividere, discutere e consigliare nuove avventure.

In più, il punto di forza di NaTour è sicuramente la condivisione della propria posizione su uno specifico itinerario con tutti altri utenti.

Questa funzionalità garantisce non solo un modo per socializzare e incontrare dal vivo persone che percorrono lo stesso itinerario, ma anche un modo per contattare persone nei paraggi in caso di difficoltà, infortunio o smarrimento.



2.6 TARGET D'UTENTI

NaTour è un social network destinato soprattutto a persone appassionate di escursioni, natura e passeggiate all'aria aperta.

Sono stati individuati alcuni profili di utenti potenzialmente interessati all'utilizzo di NaTour:

	Nome	Nancy
	Età	21
	Utilizzo dispositivi elettronici	Utilizza dispositivi elettronici <i>varie volte</i> al giorno, per studiare, per effettuare ricerche e per mettersi in contatto con compagni di studio ed amici.
	Hobby	<i>Viaggiare</i> , ama scoprire posti nuovi, punti di interesse e panorami. <i>Escursioni e passeggiate</i> , ama la natura e l'aria pulita e appena possibile organizza lunghe passeggiate con i suoi amici.
	Potenziale interesse	Elevato , apprezzerrebbe un social network dove confrontarsi con diverse persone e condividere con altri utenti i propri percorsi.



	Nome	Salvatore
	Età	56
	Utilizzo dispositivi elettronici	Utilizza dispositivi elettronici <i>poché volte</i> al giorno, per effettuare ricerche o mettersi in contatto con la propria famiglia.
	Hobby	<i>Escursioni e passeggiate</i> con la propria famiglia, ama portare i propri figli in posti nuovi, con aria pulita e lontani dal caos delle grandi città.
	Potenziale interesse	Elevato , apprezzerrebbe un social network dove controllare i dettagli di un itinerario prima di percorrerlo, per garantire a tutti i membri della sua famiglia una piacevole gita.

	Nome	Miriam
	Età	34
	Utilizzo dispositivi elettronici	Utilizza dispositivi elettronici <i>moderatamente</i> per effettuare ricerche, mettersi in contatto con amici e familiari e come forma di intrattenimento.
	Hobby	<i>Viaggiare</i> , scoprire posti nuovi e difficili da percorrere, ama mettersi in gioco.
	Potenziale interesse	Elevato , apprezzerrebbe la possibilità di cercare itinerari in base al posto in cui si trova e alla difficoltà. Inoltre, sarebbe di suo gradimento percorrere gli itinerari più difficili con qualche persona in più, come forma di distrazione ed aiuto.



Infine, è stato individuato anche un utente non interessato a NaTour, per via delle sue passioni poco concordi con il target d'utenti dell'applicazione.

	Nome	Fabrizio
	Età	28
	Utilizzo dispositivi elettronici	<i>Elevato</i> , utilizza i dispositivi elettronici per studiare e per svago.
	Hobby	<i>Videogiochi</i>, ama passare il tempo in compagnia dei suoi amici, per giocare online e dare sfogo alla sua creatività. <i>Cinema, film e serie tv</i>, ama spendere le sue giornate e le sue serate sul divano, facendo maratone di serie tv e riguardando i grandi classici del cinema.
	Potenziale interesse	Basso , non ama le passeggiate o le escursioni, ama visitare le grandi città e preferisce passare i propri pomeriggi con gli amici online o al cinema.



2.7 VALUTAZIONE USABILITÀ A PRIORI

Durante la fase di modellazione dei casi d'uso sono state effettuate delle prove preliminari di usabilità, tramite l'interazione con i mock-up, con un ristretto bacino di utenti in modo da ottenere un primo riscontro e per raccogliere le loro opinioni.

In particolare, è stato chiesto agli utenti un'opinione sulle seguenti schermate:

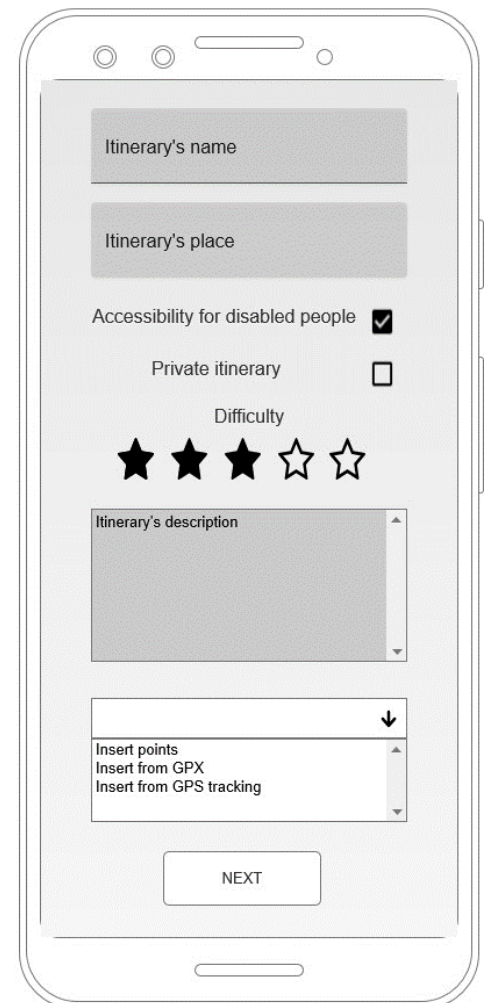
- #1 - registrazione;
- #2 - inserimento dettagli di un itinerario;
- #3 - visualizzazione di un itinerario;
- #5 - ricerca.

Gli utenti hanno trovato le schermate #1 e #3 molto semplici, intuitive e di facile utilizzo.

Le criticità sono state riscontrare nelle schermate #2 e #4. Gli utenti hanno avuto problemi nell'interagire con la schermata di inserimento dei dettagli, riportata qui di fianco, ritenendo la stessa troppo complessa e poco efficiente.

Inoltre, gli utenti hanno trovato confusionaria la schermata dei risultati di ricerca di itinerari ed utenti, dichiarando di preferire una divisione tra le due cose per velocizzare il processo di ricerca del risultato desiderato.

Le due schermate, quindi, non rispettano le euristiche di Nielsen; quindi, sono state modificate per garantire all'utente la massima esperienza possibile.



The mockup shows a mobile app interface for creating an itinerary. It includes the following elements:

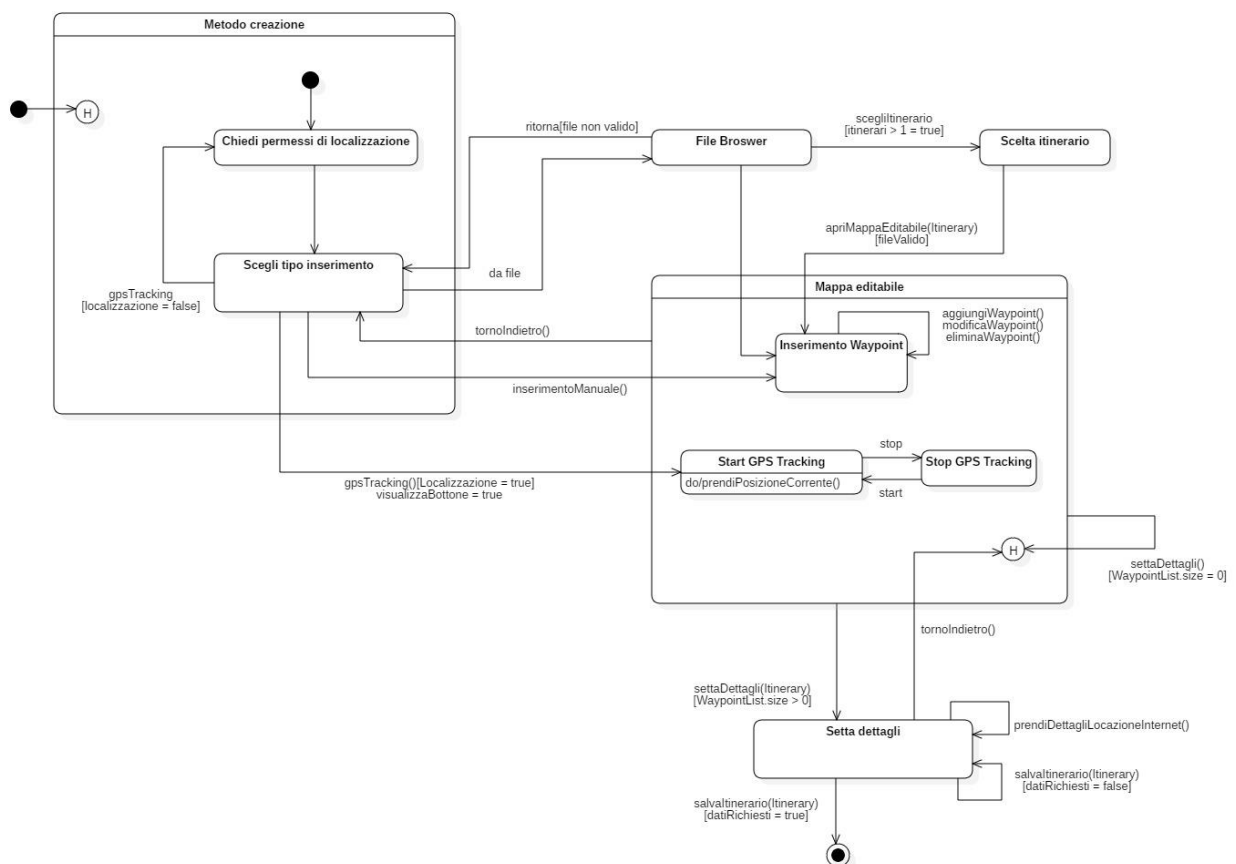
- Itinerary's name:** A text input field.
- Itinerary's place:** A text input field.
- Accessibility for disabled people:** A checkbox that is checked.
- Private itinerary:** A checkbox that is unchecked.
- Difficulty:** A rating system with five stars. The first three stars are filled, and the last two are empty.
- Itinerary's description:** A large text area with a vertical scrollbar.
- Insert points:** A dropdown menu with three options: "Insert from GPX", "Insert from GPS tracking", and "Insert from..." (partially visible).
- NEXT:** A button at the bottom of the form.



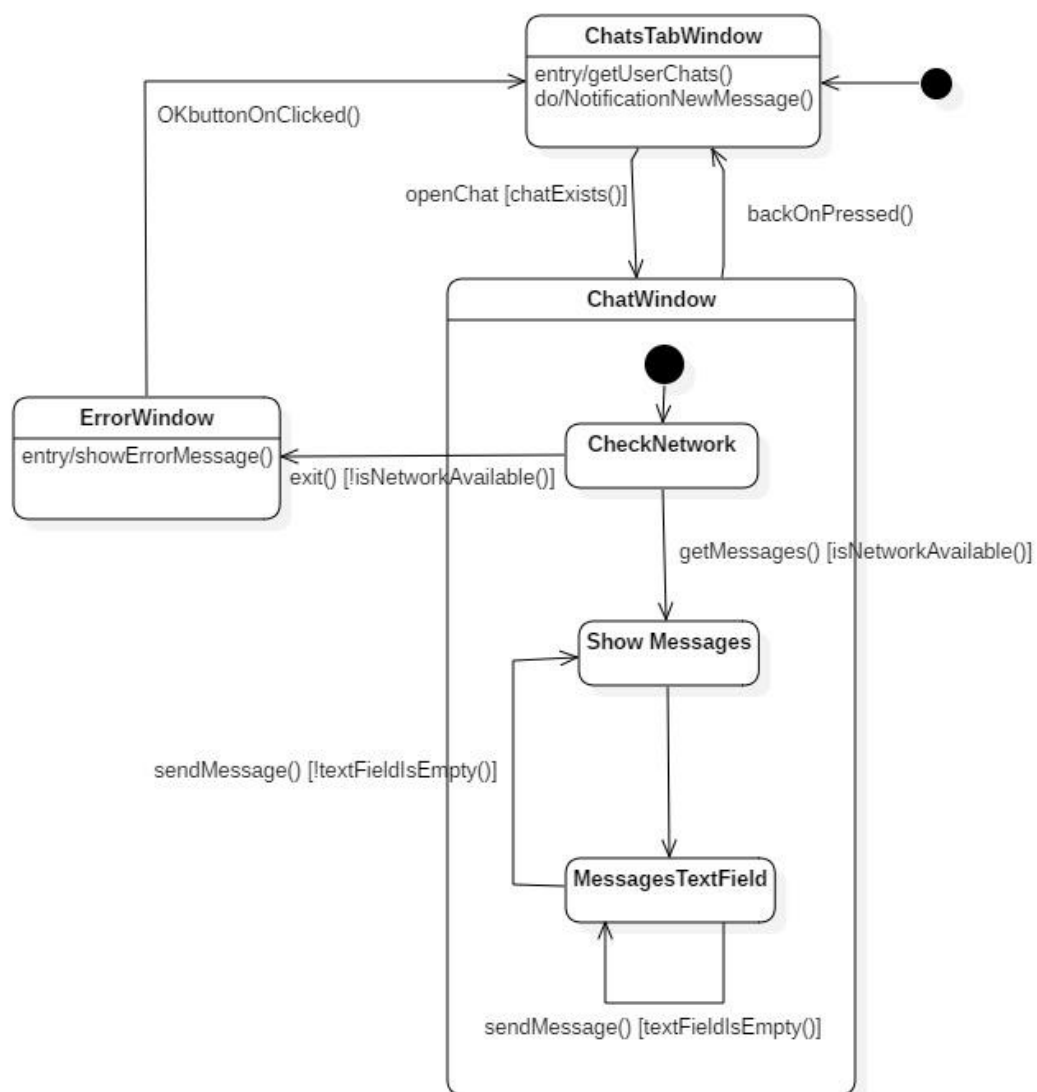
2.8 PROTOTIPAZIONE FUNZIONALE INTERFACCIA GRAFICA

Di seguito verranno riportati gli Statechart Diagram dell'interfaccia grafica riferiti ai casi d'uso precedenti, cioè creazione di un nuovo itinerario e l'invio di un messaggio privato.

2.8.1 STATECHART CREAZIONE ITINERARIO



2.8.2 STATECHART INVIO MESSAGGIO PRIVATO



2.9 GLOSSARIO

	Termine	Descrizione
1	Utente	Con utente si intende un utente che è correttamente registrato a NaTour.
2	GPS	Nelle telecomunicazioni il sistema di posizionamento GPS (acronimo in inglese: Global Positioning System, a sua volta abbreviazione di NAVSTAR GPS, acronimo di NAVigation Satellite Timing And Ranging Global Positioning System o di NAVigation Signal Timing And Ranging Global Position System) è un sistema di posizionamento e navigazione satellitare militare statunitense. Attraverso una rete dedicata di satelliti artificiali in orbita, fornisce a un terminale mobile o ricevitore GPS informazioni sulle sue coordinate geografiche e sul suo orario in ogni condizione meteorologica, ovunque sulla Terra o nelle sue immediate vicinanze dove vi sia un contatto privo di ostacoli con almeno quattro satelliti del sistema.
3	File .gpx	GPX o GPS eXchange Format è uno schema XML progettato per il trasferimento di dati GPS tra applicazioni software. Può essere usato per descrivere waypoint (punti), tracce e percorsi (routes). I suoi tag contengono queste tipologie di informazioni: location (luogo), elevation (elevazione), e time (tempo). Questo fa sì che possa essere utilizzato come formato di interscambio fra ricevitori GPS e pacchetti software.
4	XML	L'XML è un metalinguaggio per la definizione di linguaggi di markup, ovvero un linguaggio basato su un meccanismo sintattico che consente di definire e controllare il significato degli elementi contenuti in un documento o in un testo.



5	AxureRP10	Axure RP Pro è uno strumento utilizzato per fare wireframe e rapidi prototipi di interfacce di software, applicazioni e siti web.
6	Mock-up	Il mock-up è una realizzazione a scopo illustrativo di un sistema. Lo scopo del mock-up è quello di abbozzare l'idea per permettere una corretta implementazione successiva.
7	Lista di preferiti o da visitare	La lista dei preferiti e la lista da visitare sono due collezioni di itinerari che un utente può aggiornare costantemente, aggiungendo o rimuovendo itinerari personali o di altri utenti.
8	Amministratore	Con amministratore si intende un utente che ha dei permessi aggiuntivi, quali modifica e eliminazione di un itinerario e la promozione di altri utenti ad amministratore.
10	Social Network	Il social network è un servizio Internet per la gestione dei rapporti e delle reti sociali, tipicamente fruibile mediante browser o applicazioni mobili appoggiandosi sulla relativa piattaforma, che consente la comunicazione e condivisione per mezzi testuali e multimediali. I servizi di questo tipo, nati come comunità alla fine degli anni novanta e divenuti enormemente popolari nel decennio successivo, permettono agli utenti di creare un proprio profilo, organizzare una lista di contatti, pubblicare un proprio flusso di aggiornamenti e di accedere a quello altrui.



Capitolo 3

Modello di dominio

3.1 INTRODUZIONE

Dopo una descrizione più dettagliata delle funzionalità che il sistema è in grado di offrire, si passa alla definizione dei modelli di dominio, per garantire una base comune di concetti su cui ragionare e definire un vocabolario un po' più specifico del sistema.

3.2 CLASSI DI DOMINIO

Di seguito vengono riportati i Class Diagram relativi alle funzionalità che l'applicazione deve offrire.

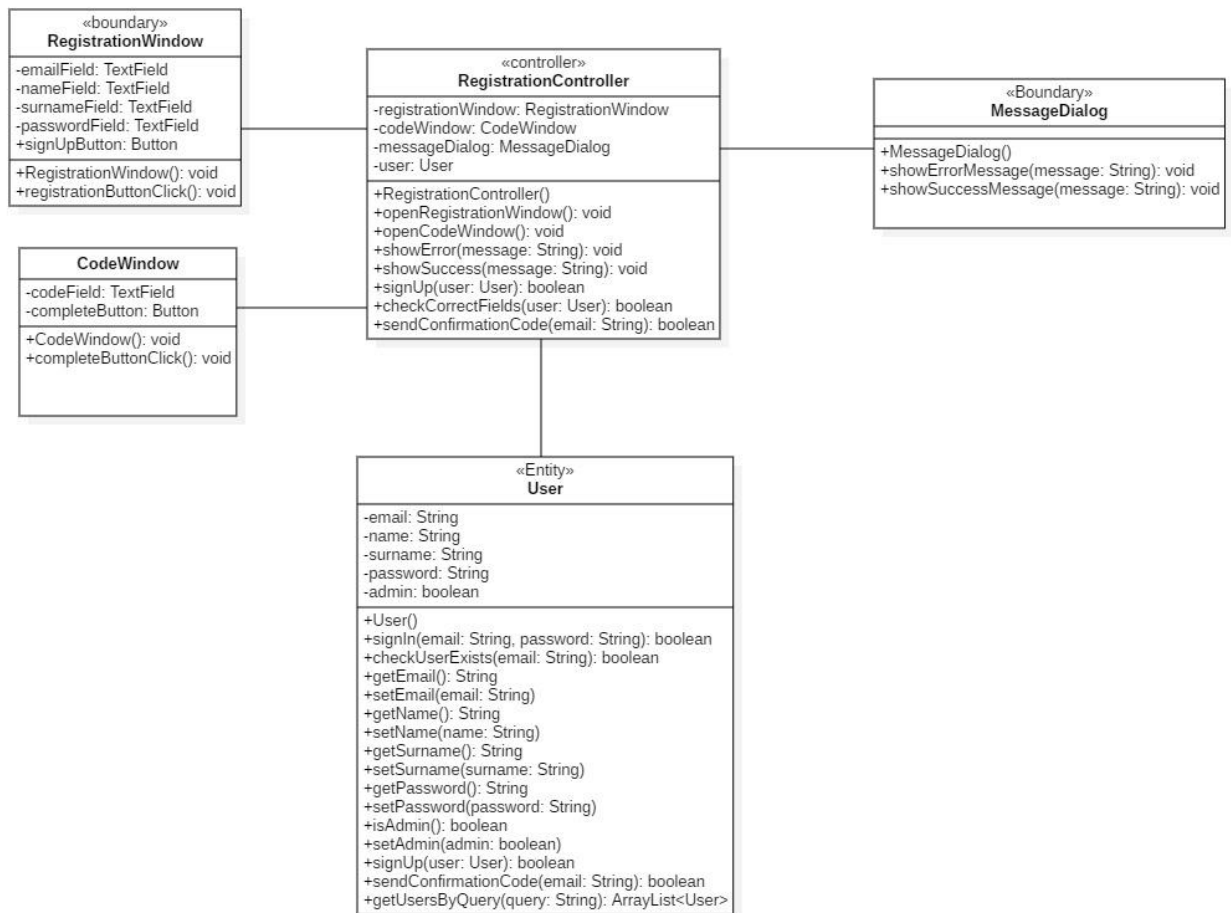
L'individuazione delle classi è guidata dall'euristica Three-Object-Type, dove gli oggetti vengono divisi in:

- **Entity** – Modellano l'informazione persistente.
- **Boundary** – Modellano interazione tra attori e sistema.
- **Control** – Modellano la logica necessaria a svolgere lo use case.

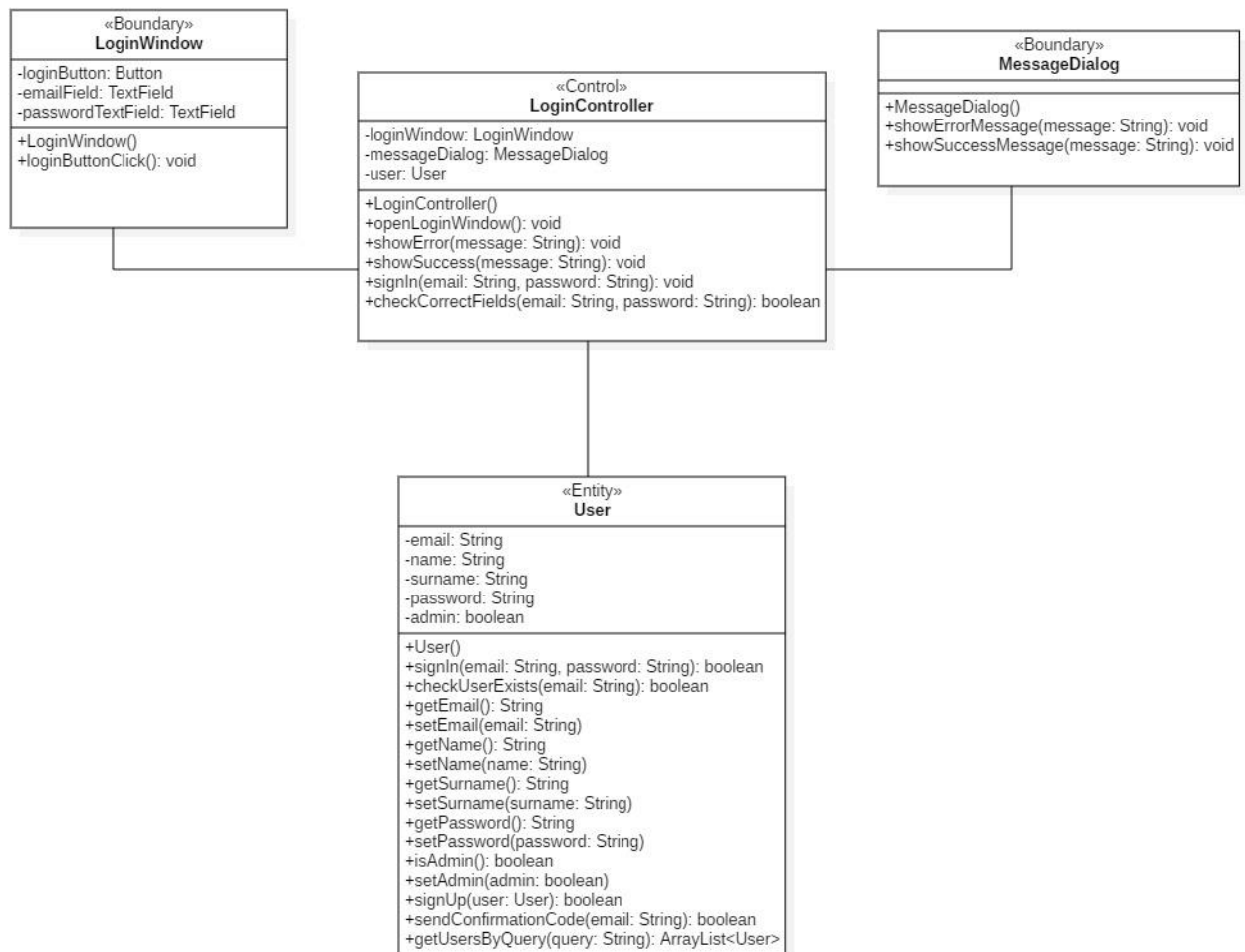
Per garantire la leggibilità, i class diagram sono stati divisi per funzionalità.



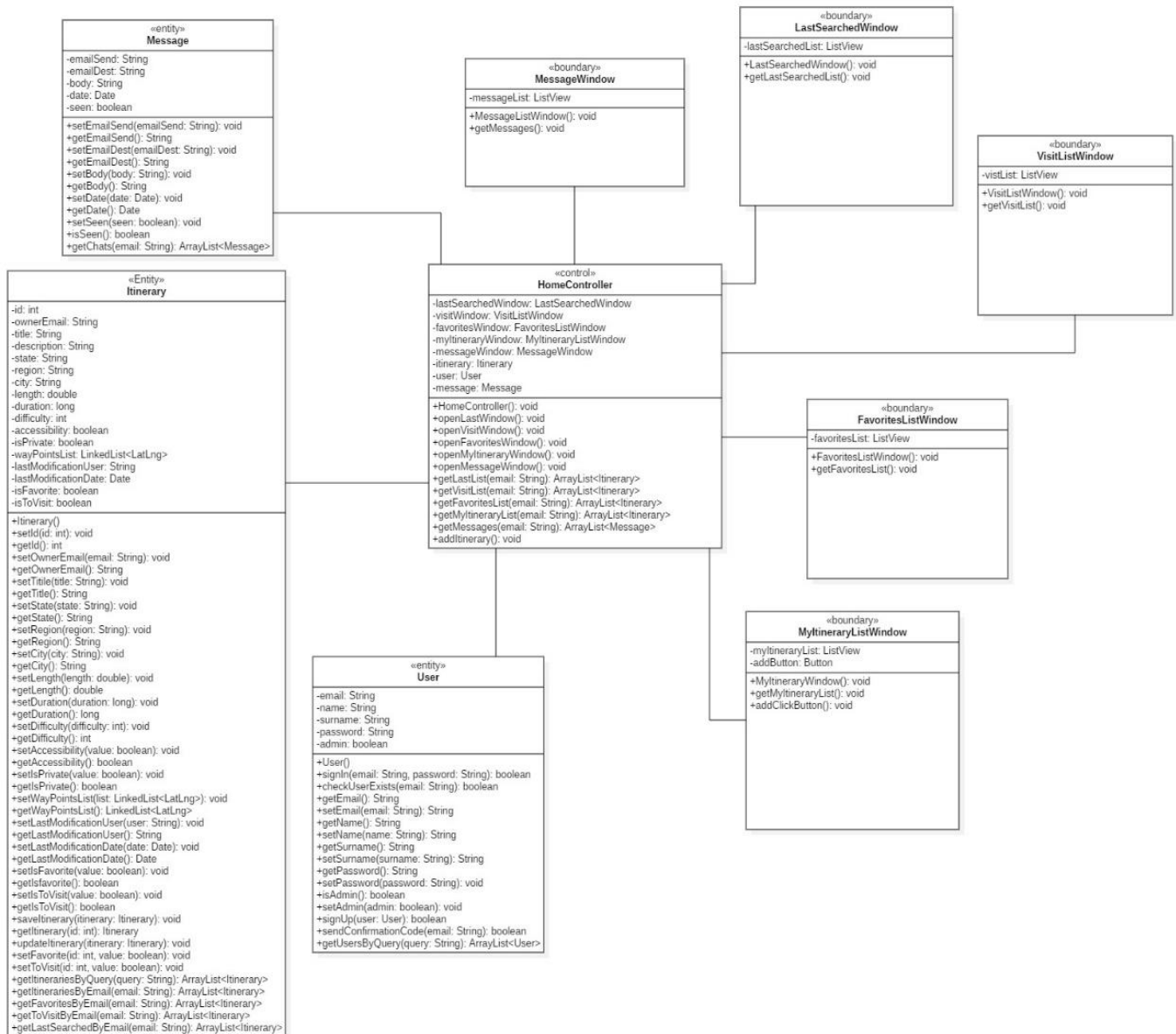
3.2.1 REGISTRAZIONE



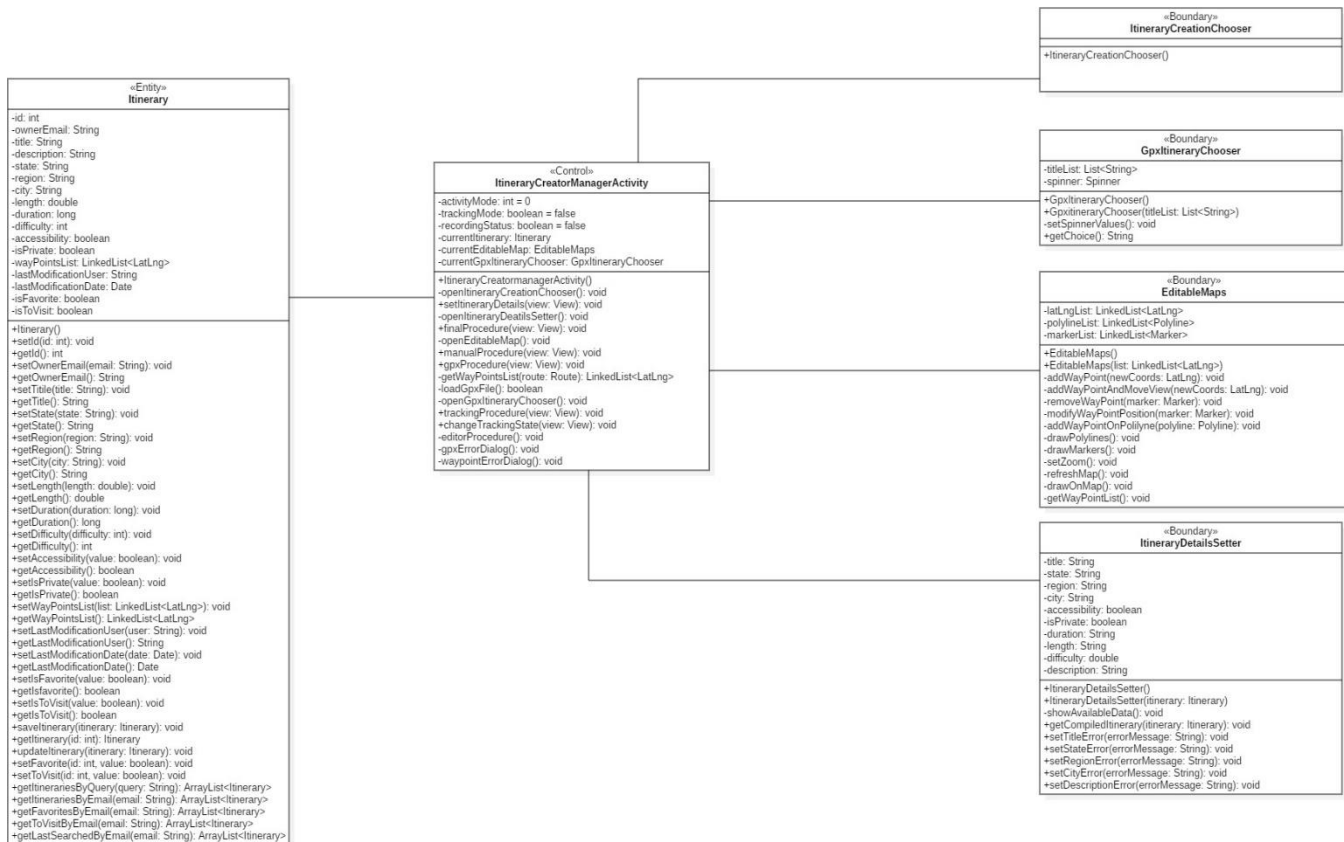
3.2.2 ACCESSO



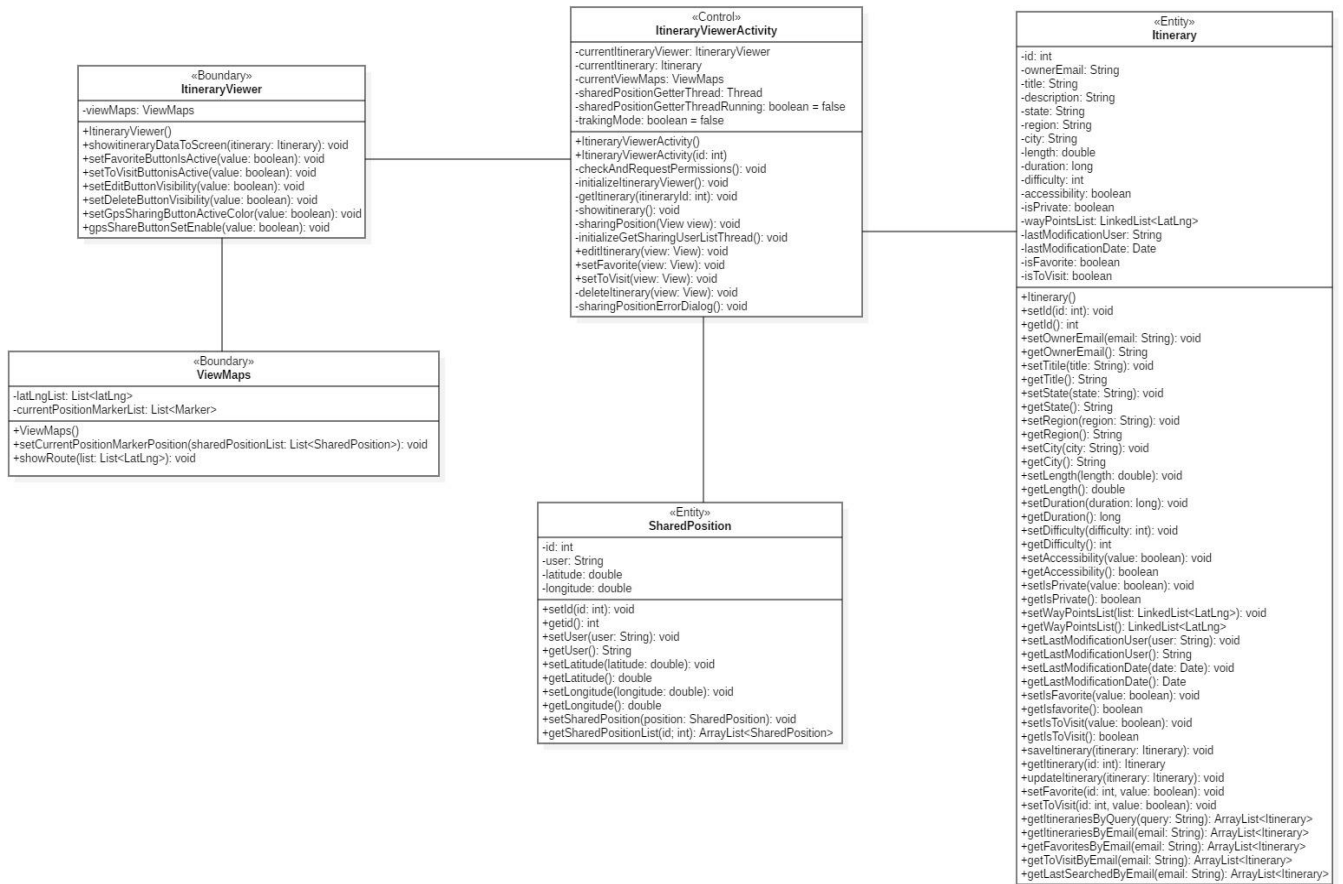
3.2.3 HOME



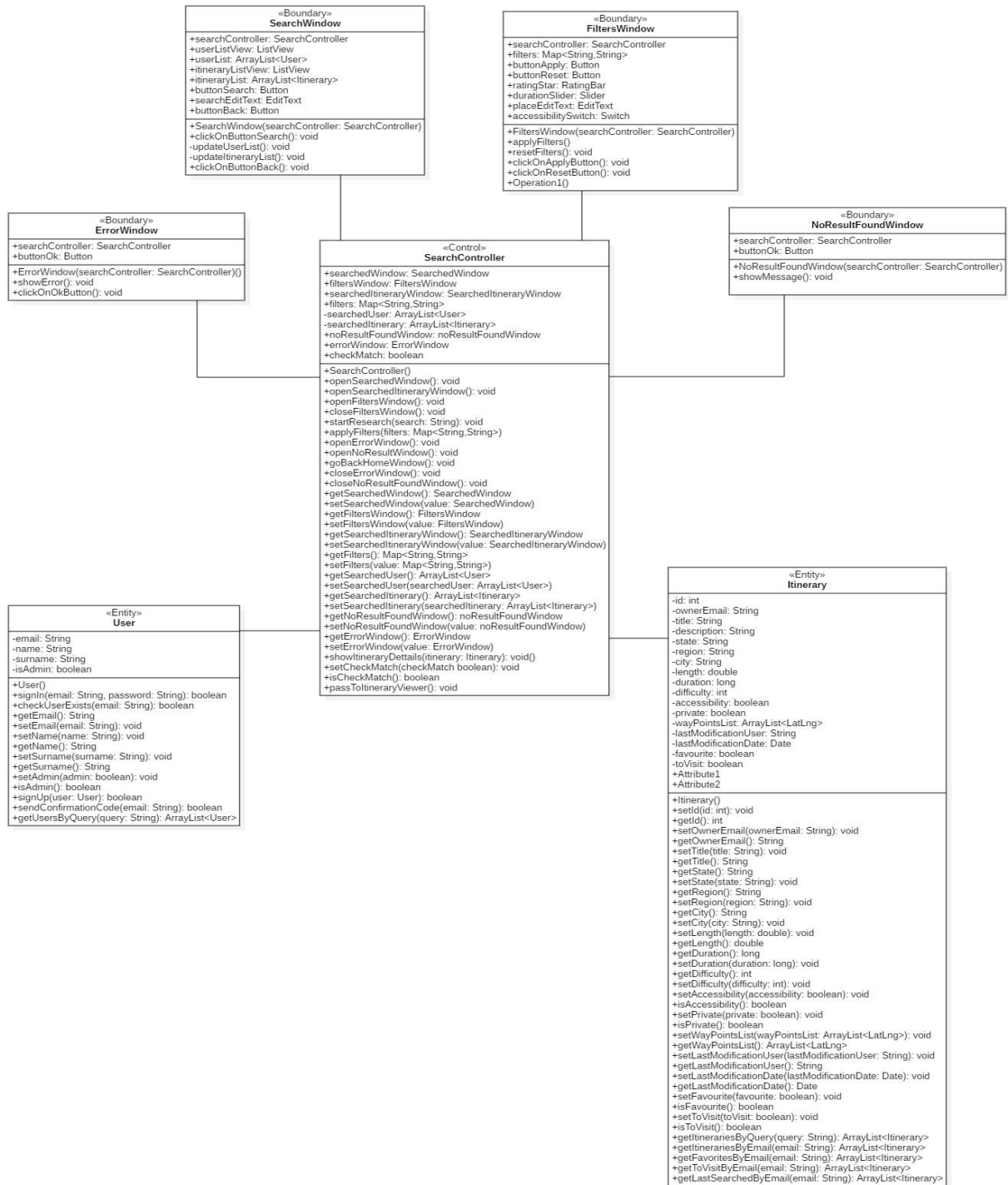
3.2.4 CREAZIONE ITINERARIO



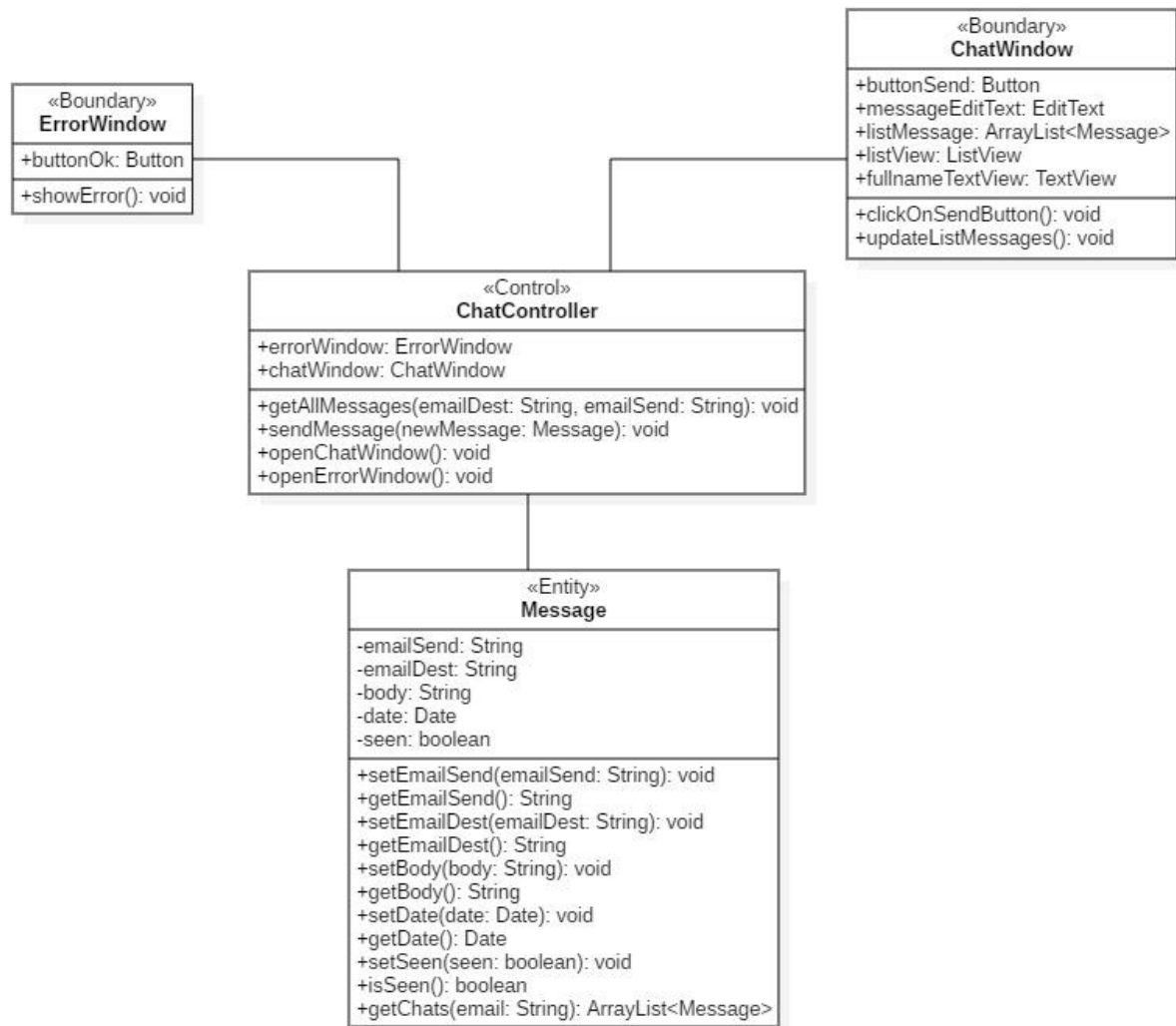
3.2.5 VISUALIZZAZIONE ITINERARIO



3.2.6 RICERCA



3.2.7 MESSAGGISTICA

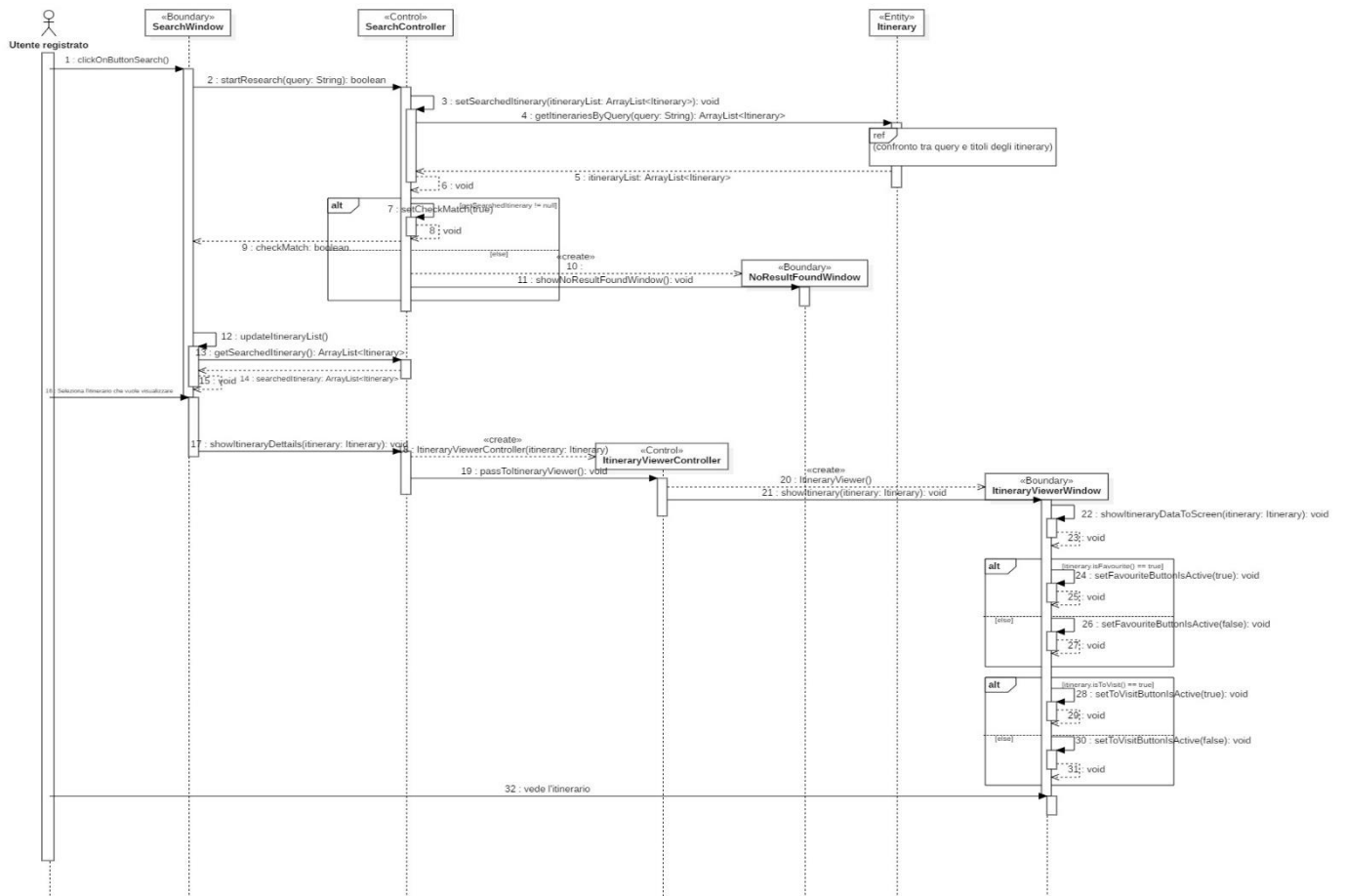


3.3 SEQUENCE DIAGRAM

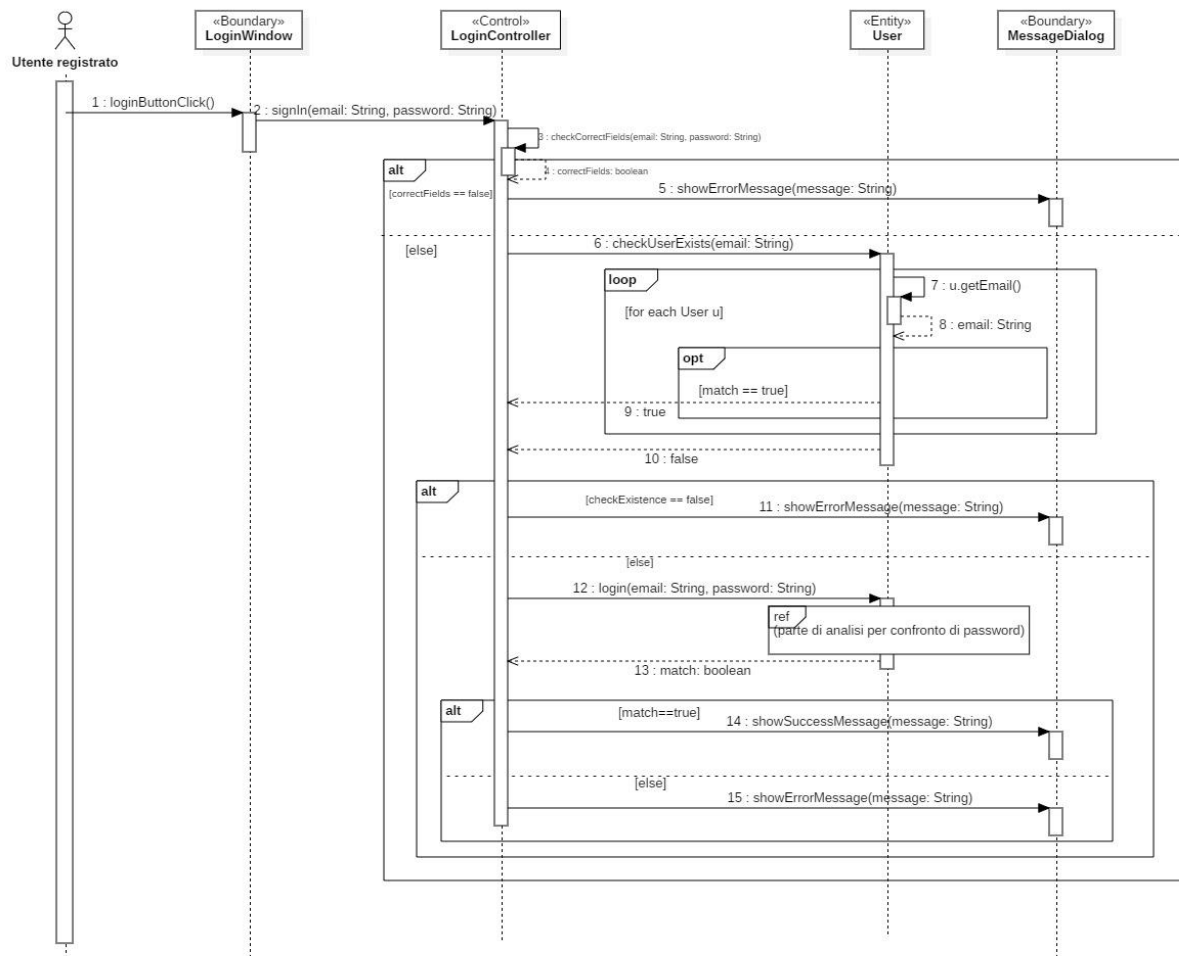
Dopo aver definito le classi vengono elencati di seguito i sequence diagram di due funzionalità.

Le funzionalità scelte sono la ricerca e l'accesso alla piattaforma.

3.3.1 RICERCA



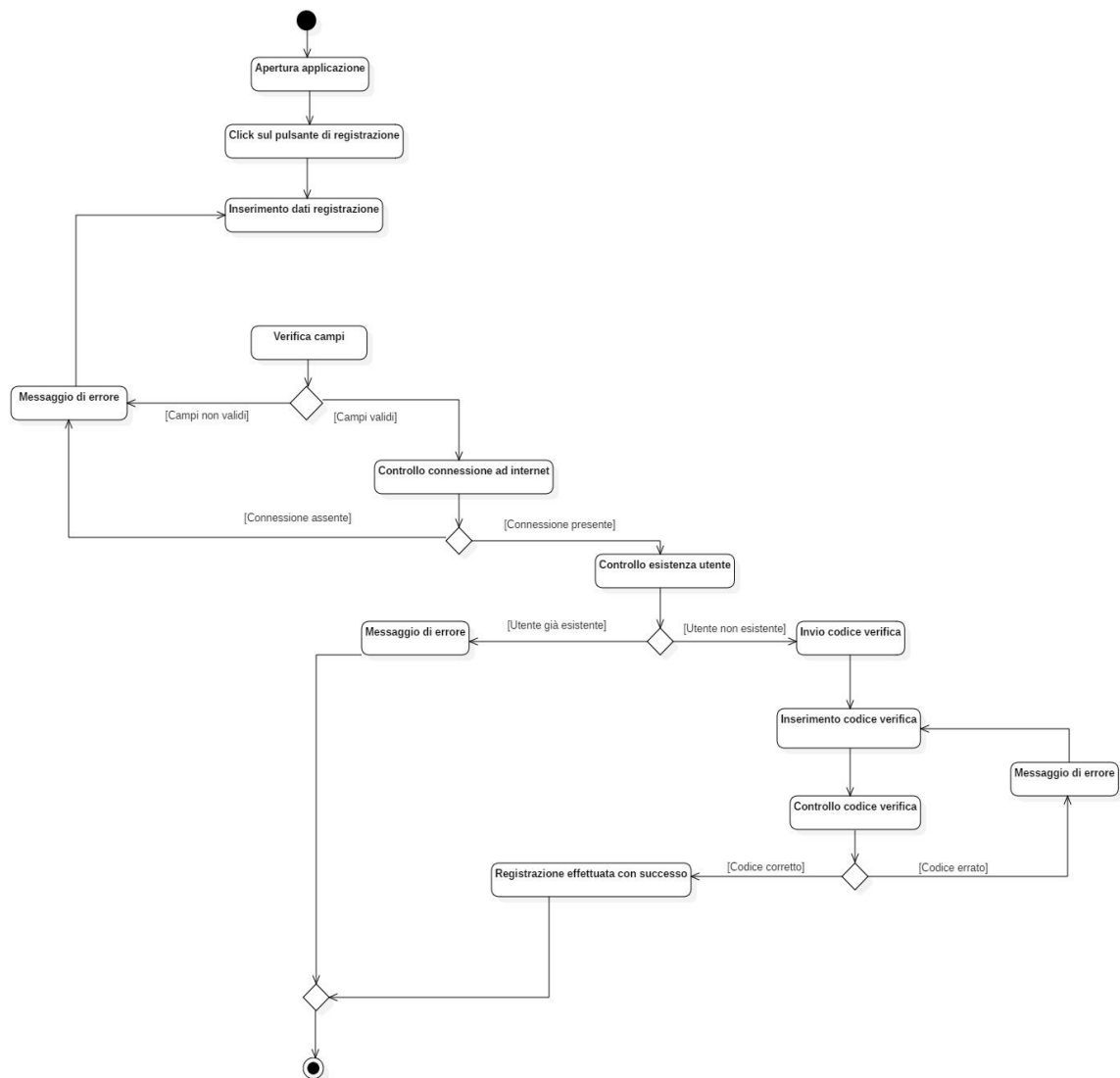
3.3.2 ACCESSO



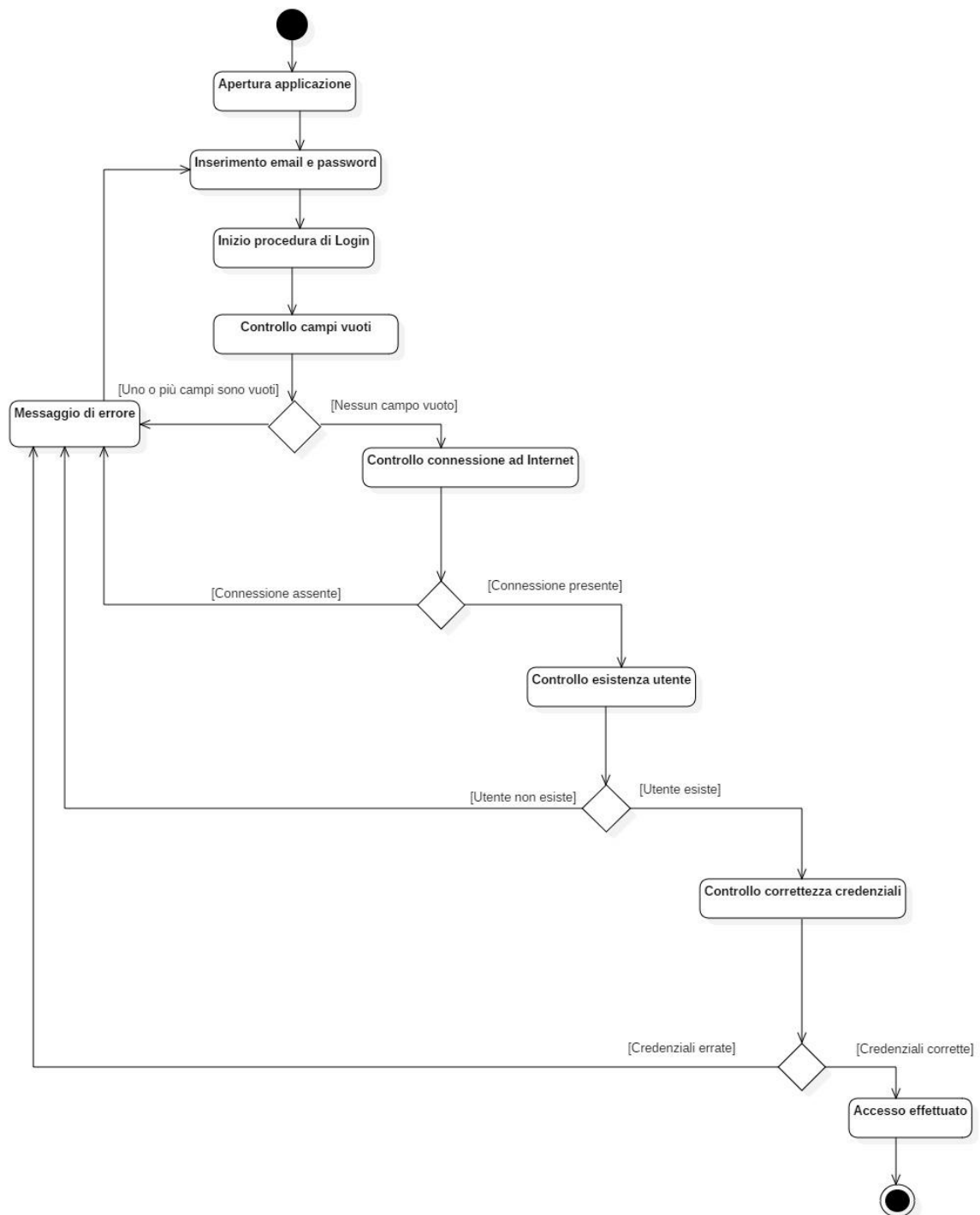
3.4 ACTIVITY DIAGRAM

Dopo i sequence diagram, di seguito vengono riportati gli activity diagram di tutte le funzionalità che NaTour è in grado di offrire.

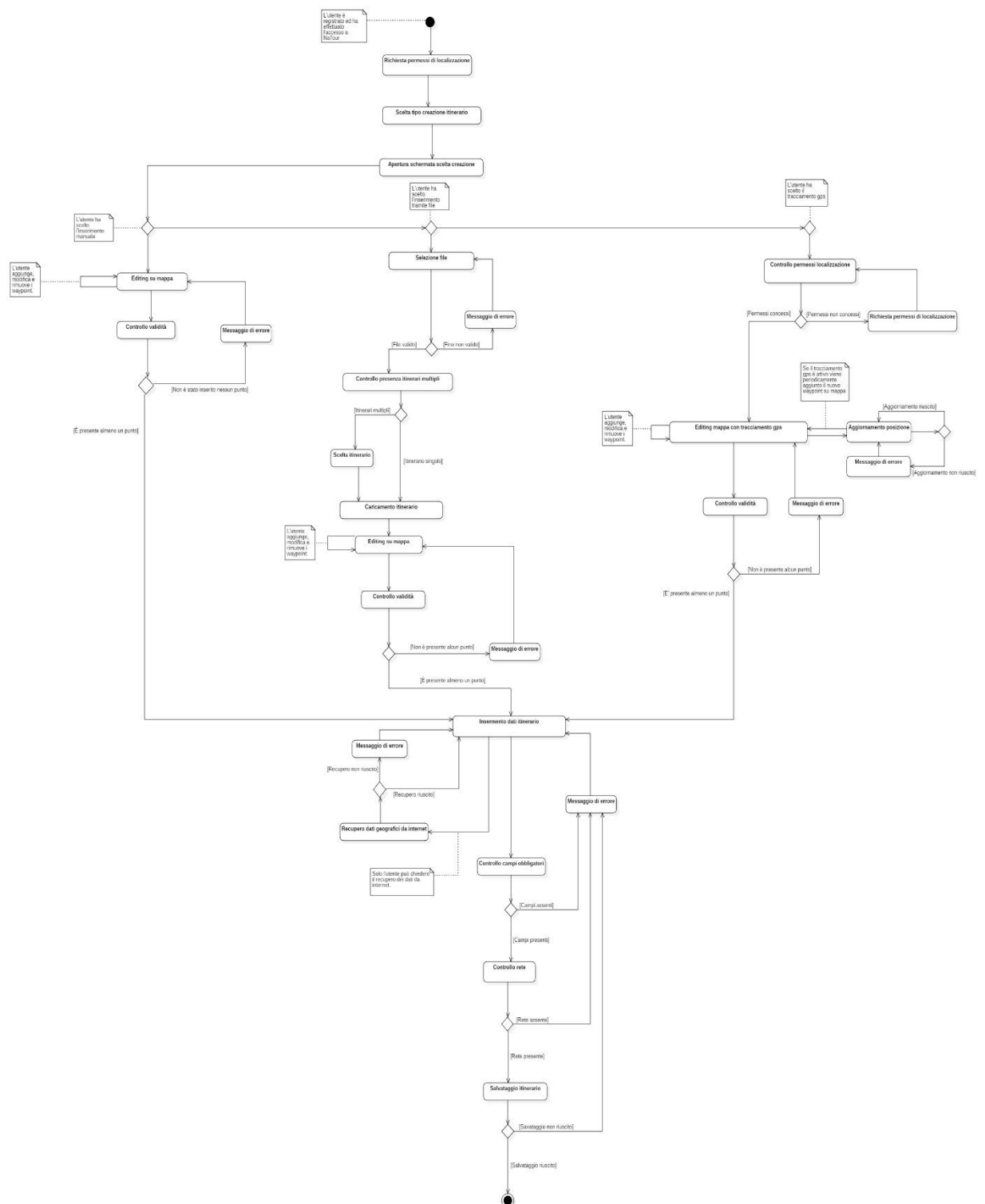
3.4.1 REGISTRAZIONE



3.4.2 ACCESSO



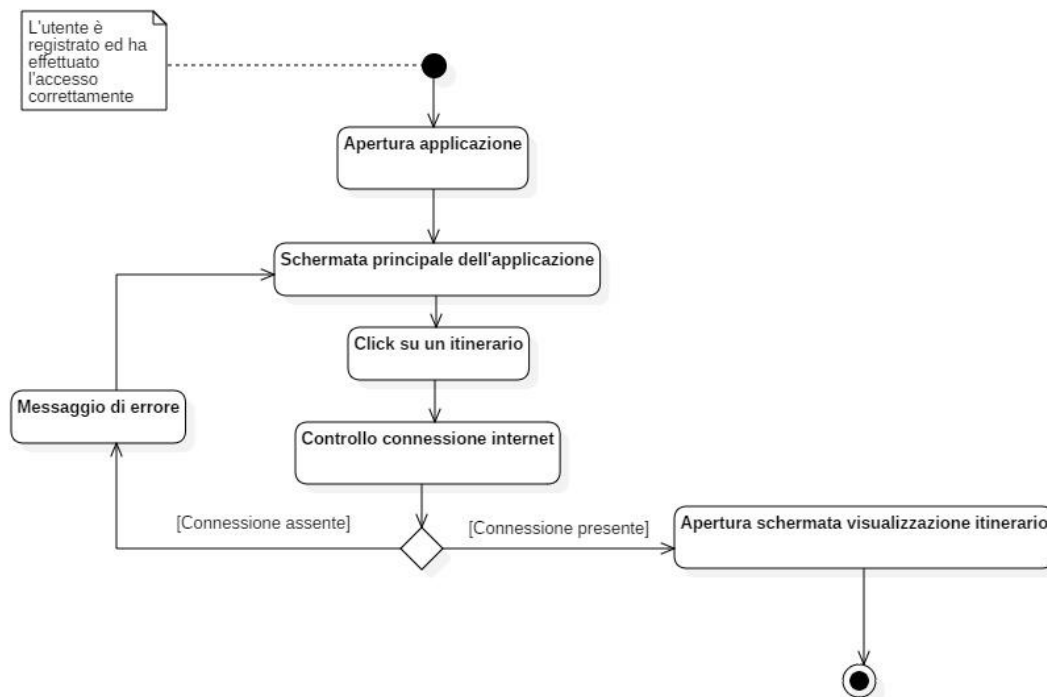
3.4.3 CREAZIONE ITINERARIO



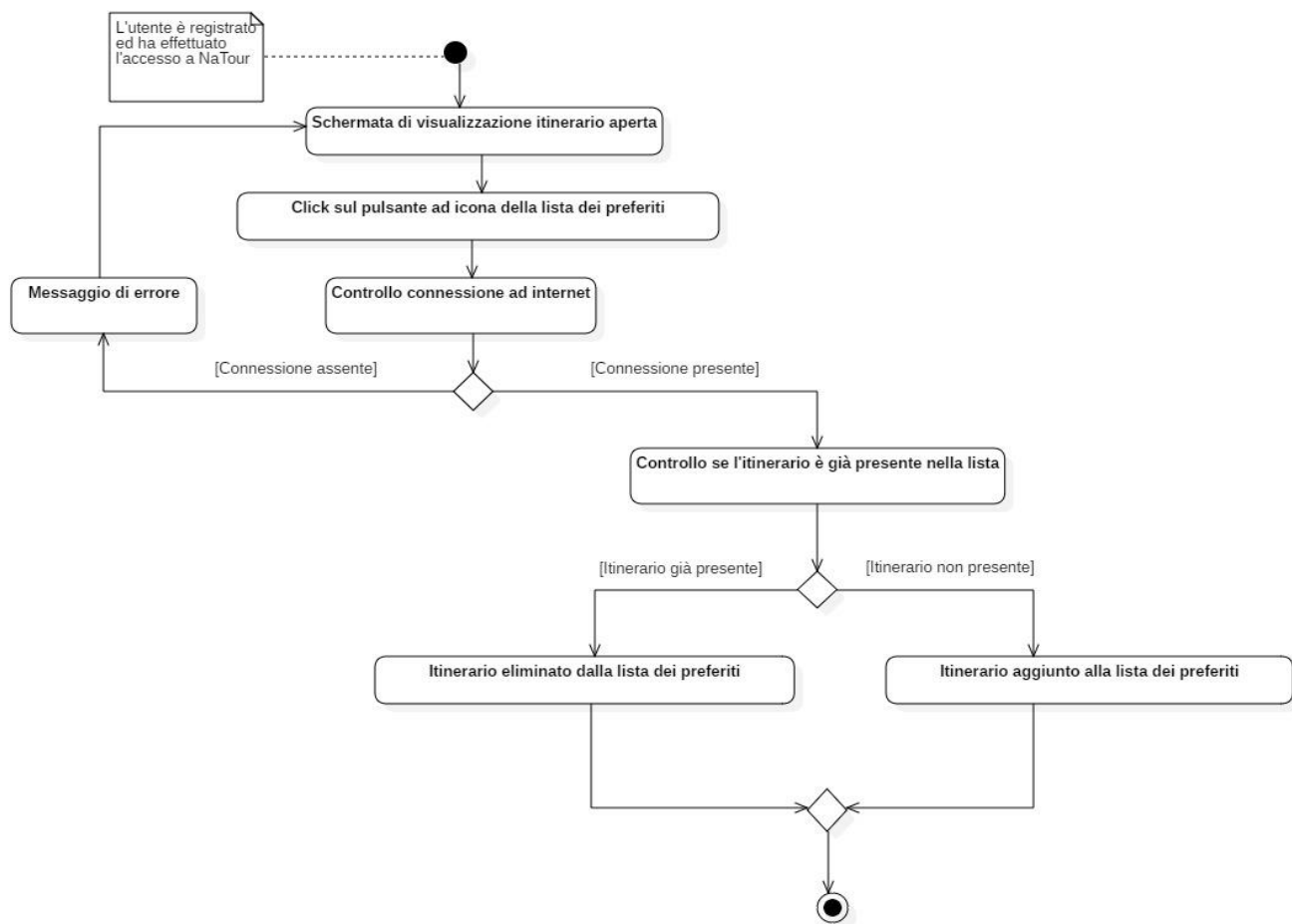
3.4.4 VISUALIZZAZIONE ITINERARIO

Le funzionalità che l'utente è in grado di compiere nella fase di visualizzazione sono molteplici.

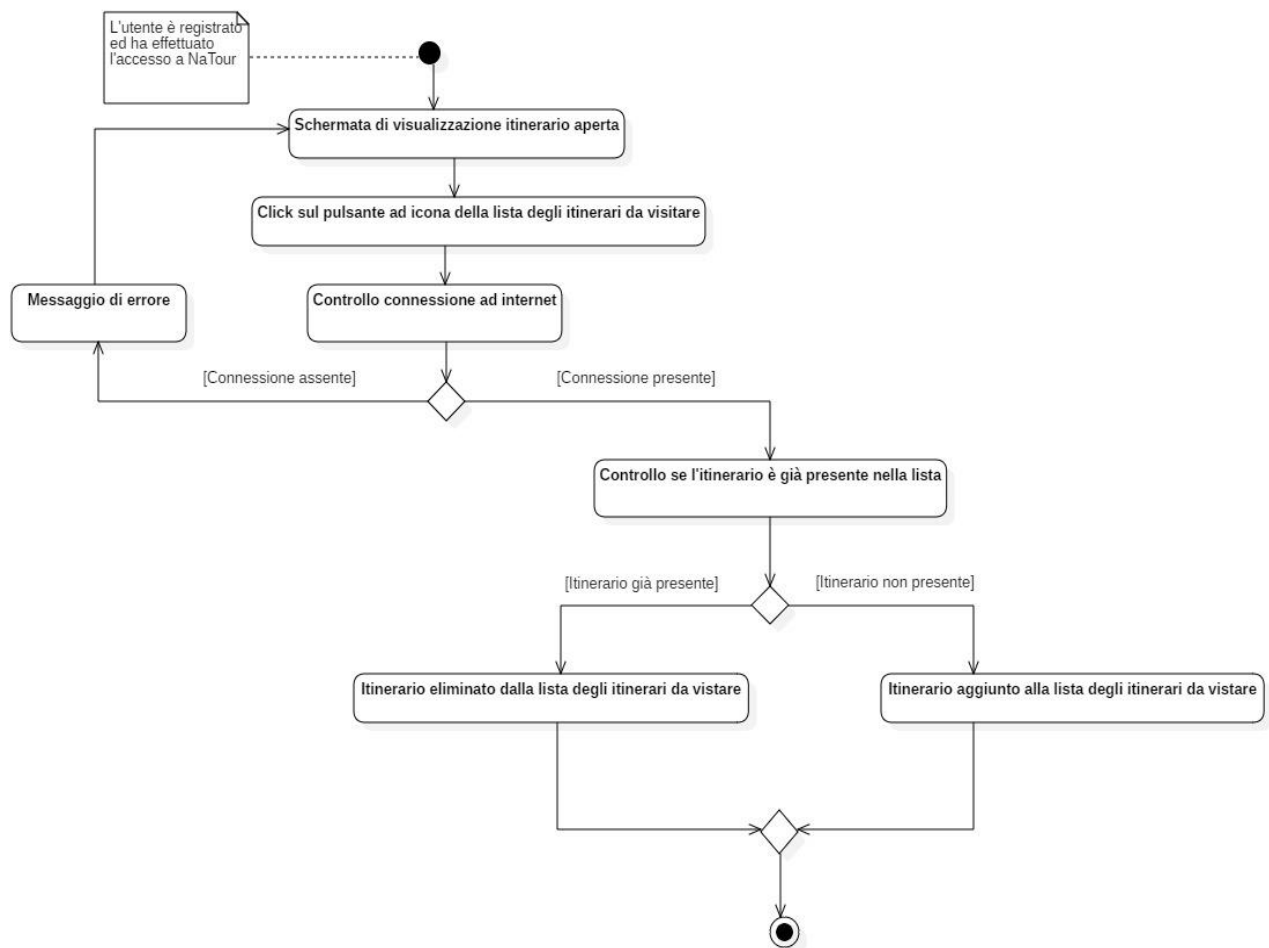
Per garantire una facile leggibilità, quindi, sono stati creati più activity diagram divisi in macro-funzionalità.



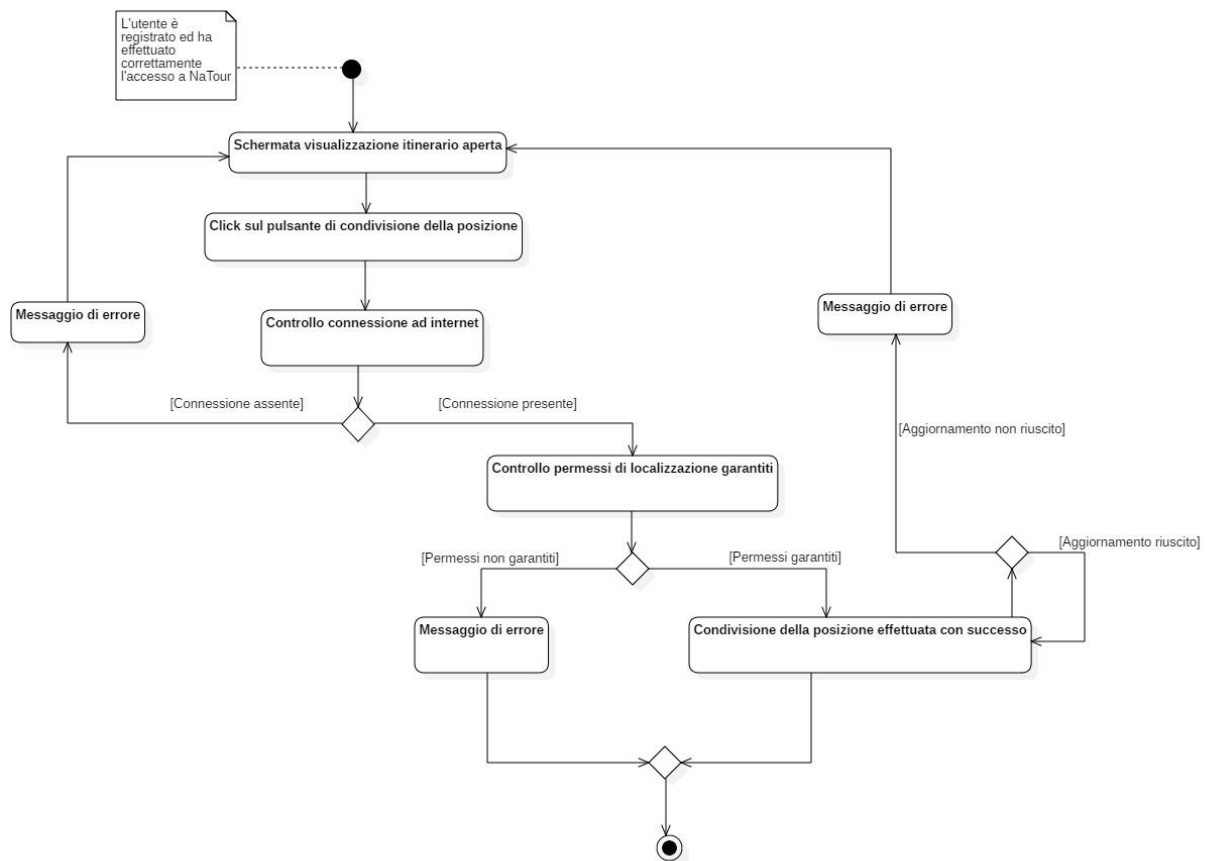
3.4.5 AGGIUNTA LISTA PREFERITI



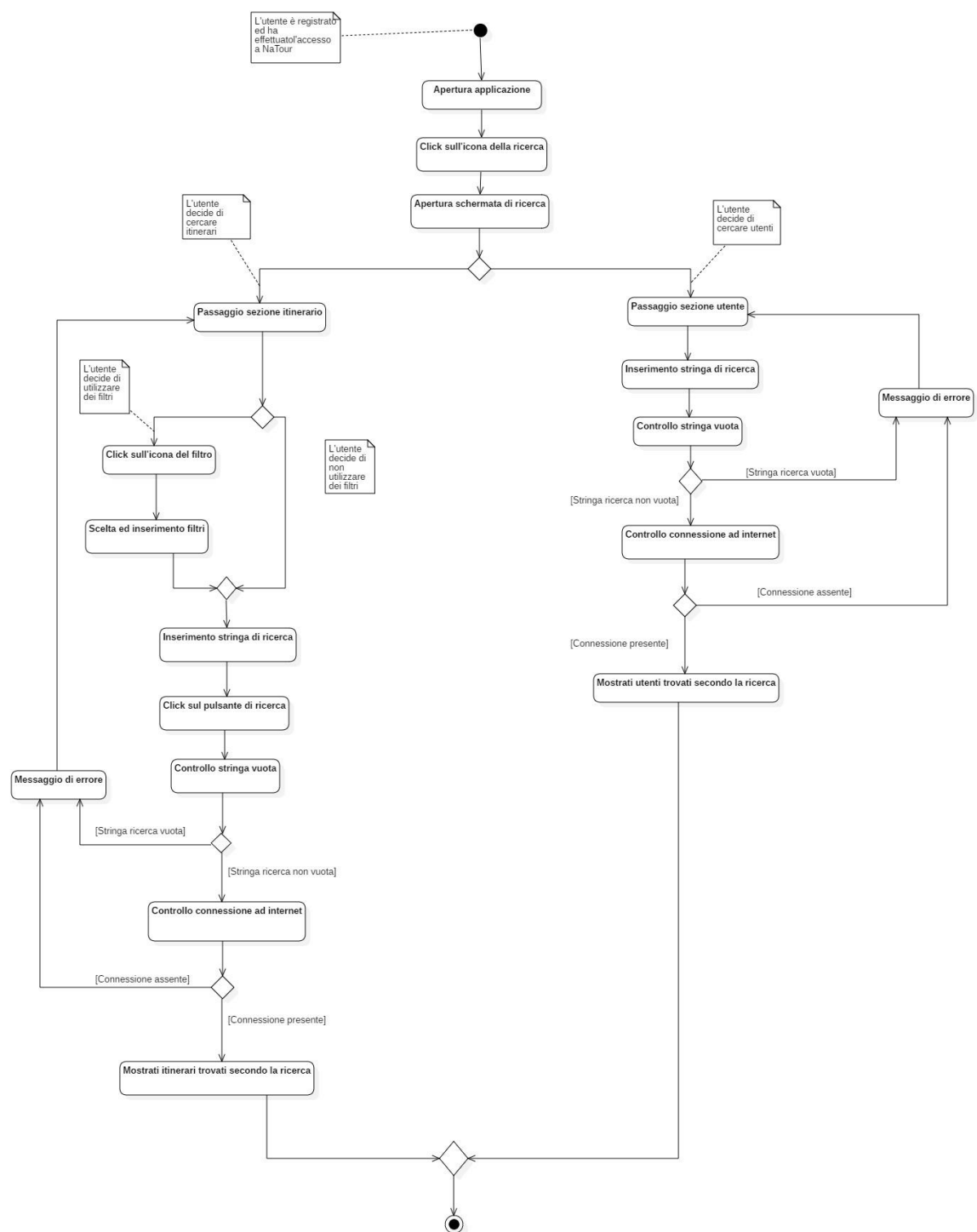
3.4.6 AGGIUNTA LISTA DA VISITARE



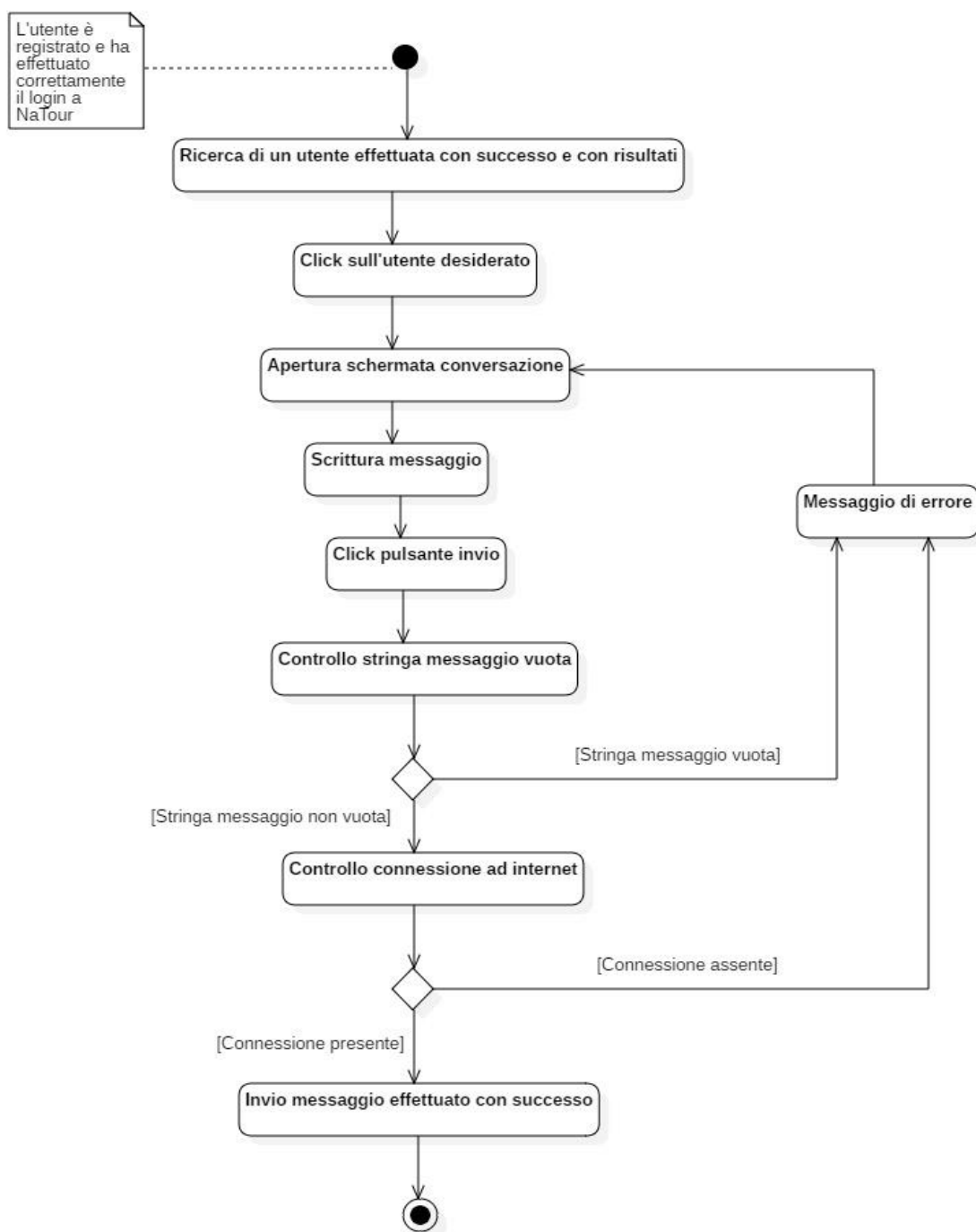
3.4.7 CONDIVISIONE POSIZIONE



3.4.8 RICERCA

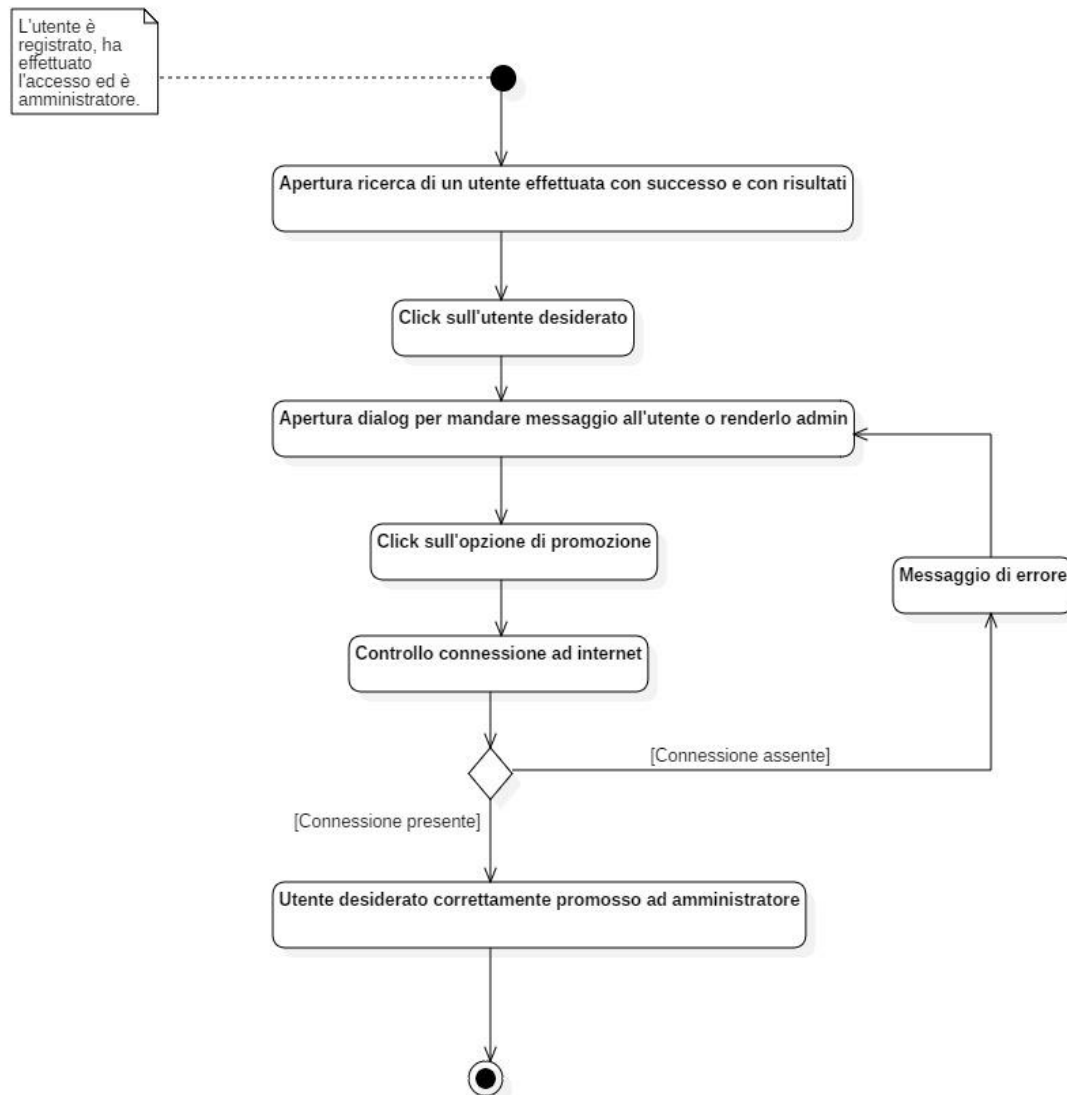


3.4.9 MESSAGGISTICA



3.4.10 PROMOZIONE AD ADMIN

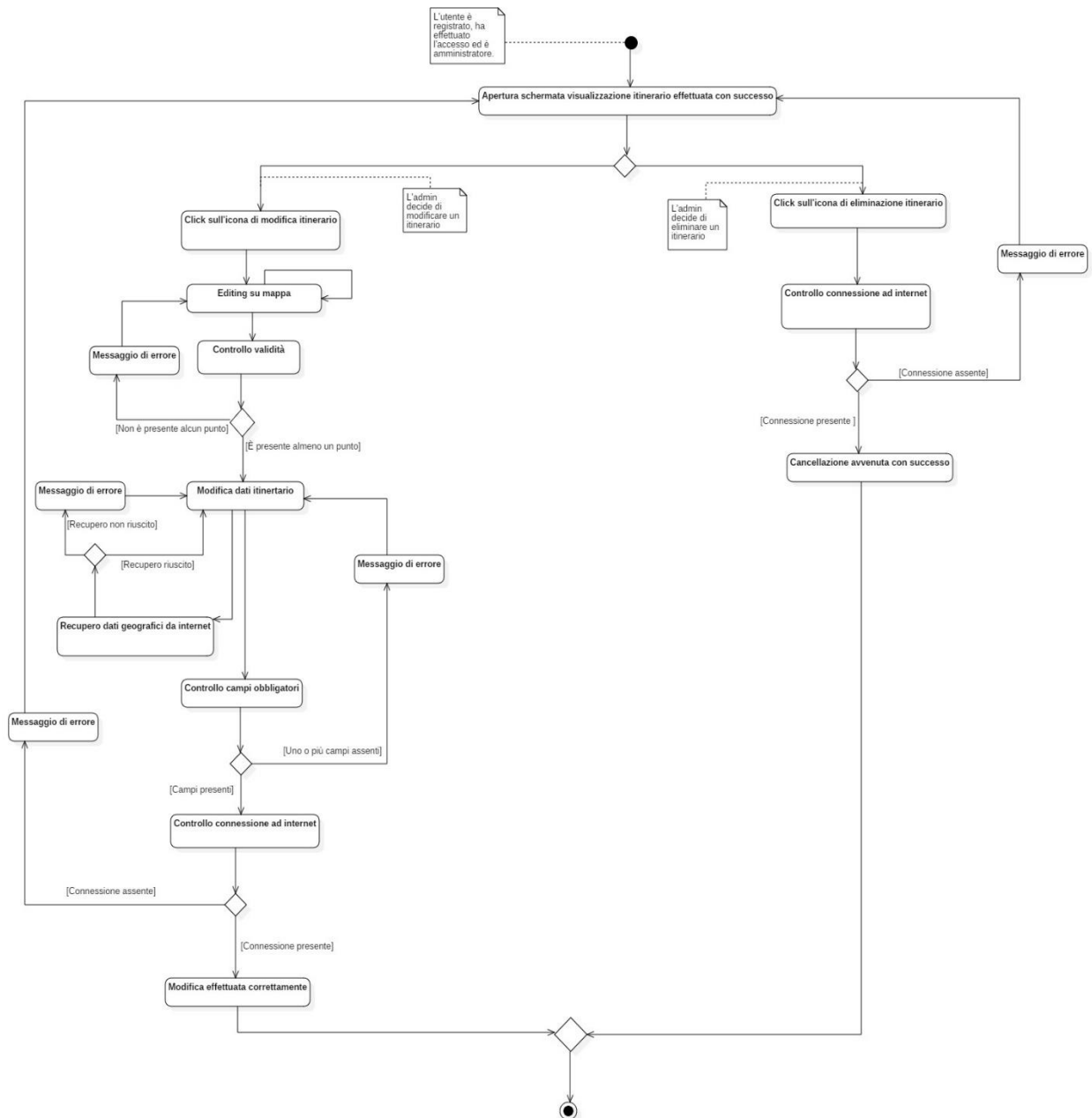
Un amministratore è in grado di promuovere un altro utente ad amministratore.
Questo activity diagram è figlio dell'activity diagram di [ricerca](#).



3.4.11 MODIFICA O CANCELLAZIONE DI UN ITINERARIO

Un amministratore può modificare o cancellare un itinerario.

Questo activity diagram è figlio dell'activity diagram di [visualizzazione](#).



Capitolo 4

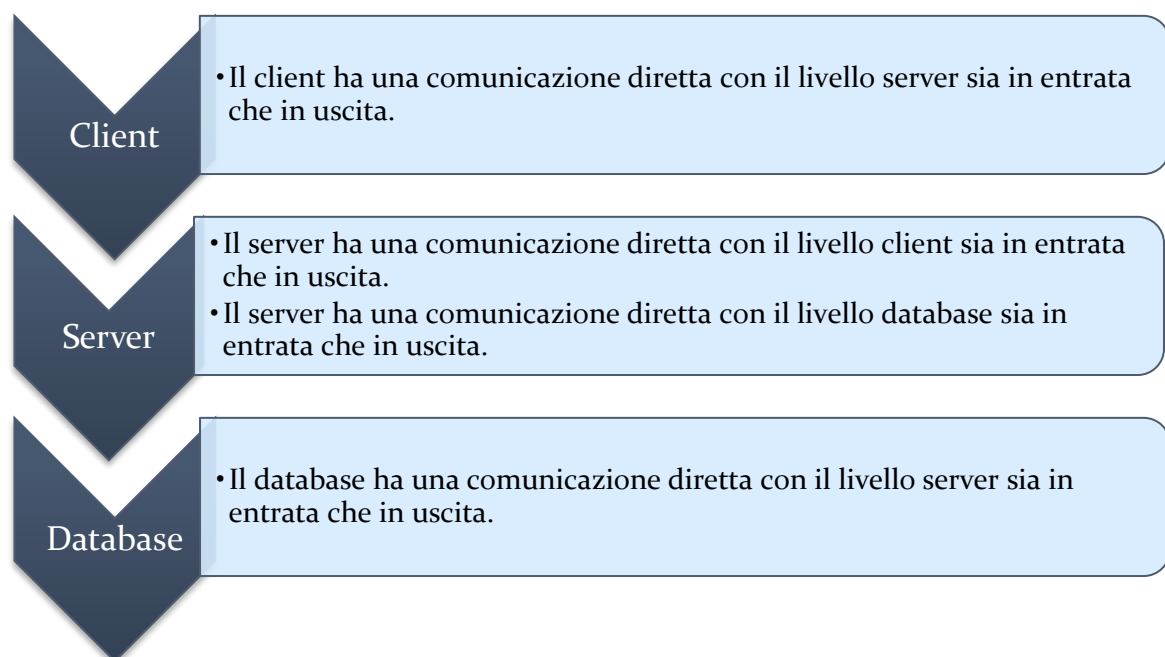
Design del sistema

4.1 INTRODUZIONE

Dopo aver progettato il sistema e aver definito un vocabolario comune, si passa alla descrizione dell'architettura utilizzata durante lo sviluppo.

4.2 ANALISI DELL'ARCHITETTURA

L'architettura del sistema è suddivisa in tre livelli chiusi, cioè ogni livello può accedere solamente al livello immediatamente sottostante.



Il server, quindi, offre una sicurezza aggiuntiva nel recupero dei dati, permettendo al client di non interfacciarsi direttamente con il database ed evitando chiamate rischiose.

4.2.1 ARCHITETTURA SERVER

L'architettura server scelta è di tipo *serverless*.

Il serverless computing è un modello di sviluppo cloud native che consente agli sviluppatori di creare ed eseguire applicazioni senza gestire i server.

Anche se in questo modello i server vengono utilizzati comunque, sono astratti dallo sviluppo delle app.

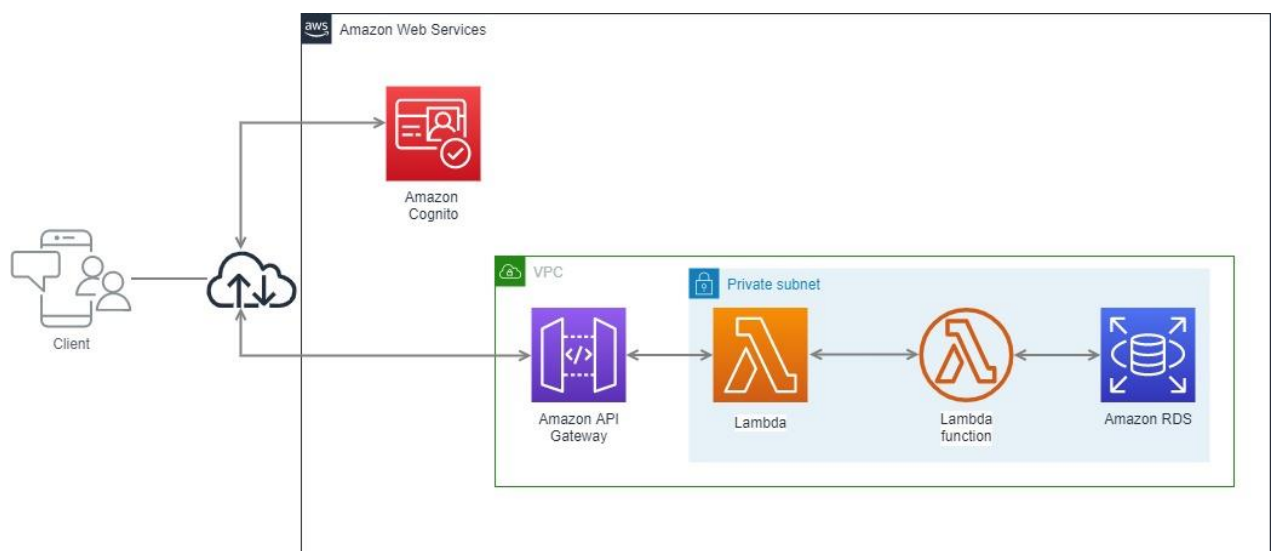
È stato scelto questo tipo di architettura per i suoi numerosi vantaggi, tra i quali:

- Riduzione dei costi di sviluppo e di manutenzione;
- Eliminazione della gestione dell'infrastruttura e del sistema operativo;
- Accessibilità on – demand, risolvendo la necessità di potenze di carichi molto elevati;
- Alta scalabilità;
- Gestione del provisioning delle capacità.

Per realizzare questo tipo di architettura si è scelto di utilizzare *AWS Lambda* in unione ad *Amazon Cognito*.

Questi due servizi di Amazon combinati costruiscono delle API in grado di offrire sicurezza e adattabilità per il database.

In dettaglio l'architettura si presenta così:



Come specificato anche prima, quindi, il client è in grado di comunicare con il database esclusivamente grazie ai due servizi Amazon sopracitati.

Le funzioni utilizzate sono le seguenti:

Tipo	Funzione	Descrizione
GET	<i>getUser</i>	Funzione di GET che recupera i dati di un utente.
	<i>getUserByQuery</i>	Funzione di GET che recupera gli utenti che rispettano la richiesta dell'utente.
	<i>getSharedPosition</i>	Funzione di GET che recupera le posizioni degli utenti in un determinato itinerario.
	<i>downloadItinerary</i>	Funzione di GET che recupera un itinerario.
	<i>getItinerariesByQuery</i>	Funzione di GET che recupera gli itinerari che rispettano la richiesta dell'utente.
	<i>getLists</i>	Funzione di GET che recupera le liste di un utente.
	<i>getRandomItineraries</i>	Funzione di GET che recupera cinque itinerari casuali.
	<i>getMessages</i>	Funzione di GET che recupera i messaggi tra due utenti.
	<i>getUserChats</i>	Funzione di GET che recupera le conversazioni di un utente.
	<i>getNotificationNumber</i>	Funzione di GET che recupera il numero di messaggi non letti da un utente.
POST	<i>setSharedPosition</i>	Funzione di POST per inserire la posizione corrente di un utente nell'itinerario.



	<i>uploadItinerary</i>	Funzione di POST che carica un itinerario.
	<i>setFavorite</i>	Funzione di POST che inserisce l'itinerario nella lista dei preferiti di un utente.
	<i>setToVisit</i>	Funzione di POST che inserisce l'itinerario nella lista da visitare di un utente.
	<i>uploadMessage</i>	Funzione di POST che invia il messaggio.
DELETE	<i>deleteItinerary</i>	Funzione di DELETE che elimina un itinerario.
UPDATE	<i>updateUserAdmin</i>	Funzione di UPDATE utilizzata per promuovere un utente ad amministratore.
	<i>updateItinerary</i>	Funzione di UPDATE che modifica un itinerario.



4.2.2 ARCHITETTURA CLIENT

Il client è stato realizzato in Android con il linguaggio Java.

È stata scelta questa piattaforma per la sua enorme diffusione e per l'altissima documentazione che viene messa a disposizione.

Inoltre, Android è un sistema operativo in continua evoluzione, che si aggiorna costantemente per garantire all'utente la miglior esperienza possibile.

L'architettura è un'architettura chiusa, dove ogni livello comunica solamente con il livello immediatamente sottostante.

Per la realizzazione del client è stato utilizzato il pattern **MVP** (Model – View – Presenter).

Questo pattern è stato preferito al pattern MVC poiché si adatta meglio all'ambiente Android.

Il pattern MVP permette di avere una separazione tra il livello di View e quello di Model, con il Presenter a fare da collante tra i due.

4.3 SERVIZI CLOUD UTILIZZATI

Il sistema utilizza i seguenti servizi offerti da AWS (Amazon Web Services):

- **AWS Lambda** - AWS Lambda è un servizio di calcolo basato su eventi (event driven) serverless .
Lambda esegue il codice su un'infrastruttura di calcolo ad alta disponibilità e gestisce tutta l'amministrazione delle risorse di elaborazione, compresa la manutenzione del server e del sistema operativo, il provisioning della capacità e la scalabilità automatica, il monitoraggio e la registrazione del codice. Con Lambda, è possibile eseguire il codice praticamente per qualsiasi tipo di applicazione o servizio back-end.
Lambda è un servizio indicato per lo sviluppo di API RESTful.
- **Cognito** - Amazon Cognito fornisce autenticazione, autorizzazione e gestione degli utenti per le app Web e per dispositivi mobili. Gli utenti possono accedere direttamente con un nome utente e una password, oppure tramite terze parti, ad esempio Facebook, Amazon, Google o Apple.

I due componenti principali di Amazon Cognito sono i bacini d'utenza e i pool di identità. I bacini d'utenza sono directory utente che forniscono opzioni di registrazione e di accesso agli utenti delle tue app. I pool di identità permettono di concedere agli utenti l'accesso ad altri servizi AWS. È possibile usare i pool di identità e i bacini d'utenza separatamente o insieme.



- **Amazon RDS** - Amazon Relational Database Service (Amazon RDS) è un servizio Web che semplifica la configurazione, l'uso e il dimensionamento di un database relazionale in AWS Cloud. Offre una capacità ridimensionabile a un costo conveniente per un database relazionale standard del settore e gestisce task comuni di amministrazione del database.

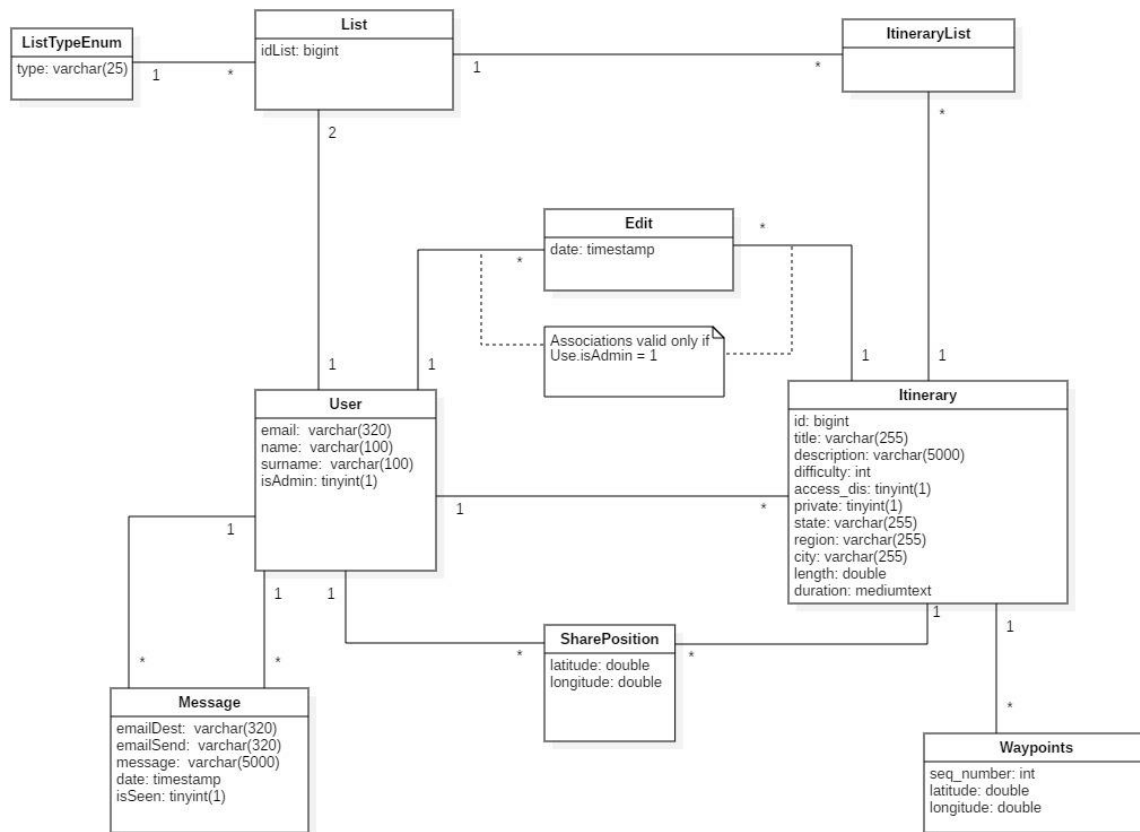
Questo servizio fornisce una capacità ridimensionabile efficiente nei costi, automatizzando al tempo stesso le attività di amministrazione più dispendiose in termini di tempo, quali il provisioning di hardware, l'impostazione di database, l'applicazione di patch e i backup. Consente di concentrarsi sulle proprie applicazioni per fornire le prestazioni ottimali, la disponibilità elevata, la sicurezza e la compatibilità di cui hanno bisogno.

Amazon RDS è disponibile in diversi tipi di istanza database, ottimizzati per la memoria, prestazioni o I/O, e permette di scegliere tra sei motori di database comuni, fra cui Amazon Aurora, PostgreSQL, MySQL, MariaDB, Oracle Database ed SQL Server.



4.4 PROGETTAZIONE DATABASE

Di seguito viene riportato il Class Diagram dell'implementazione del database.



La doppia relazione tra *Message* ed *User* indica l'invio e la ricezione di un messaggio. Si è scelto di proseguire con uno scambio di messaggi asincrono.

Infine, come sottolineato anche nel class diagram, la relazione *User – Edit* e *Edit – Itinerary* è valida solamente quando l'utente è amministratore.



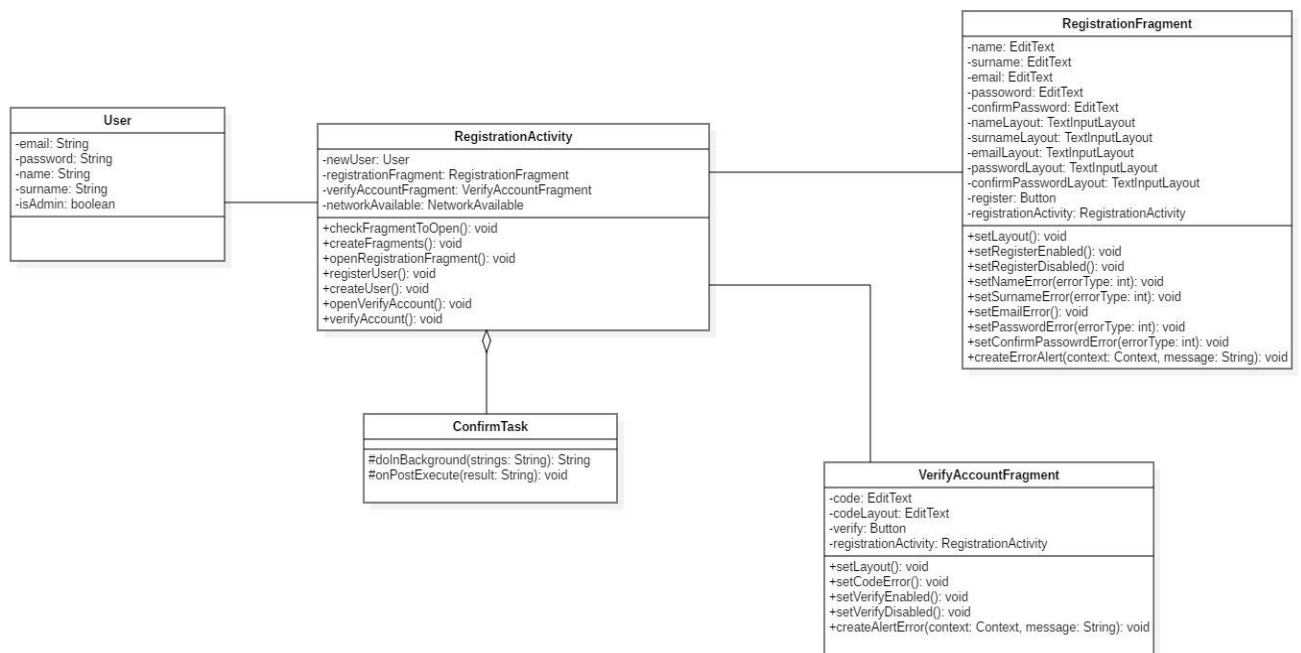
4.5 DIAGRAMMI CLASSI DI DESIGN

Di seguito vengono riportati i class diagram del sistema aggiornati.

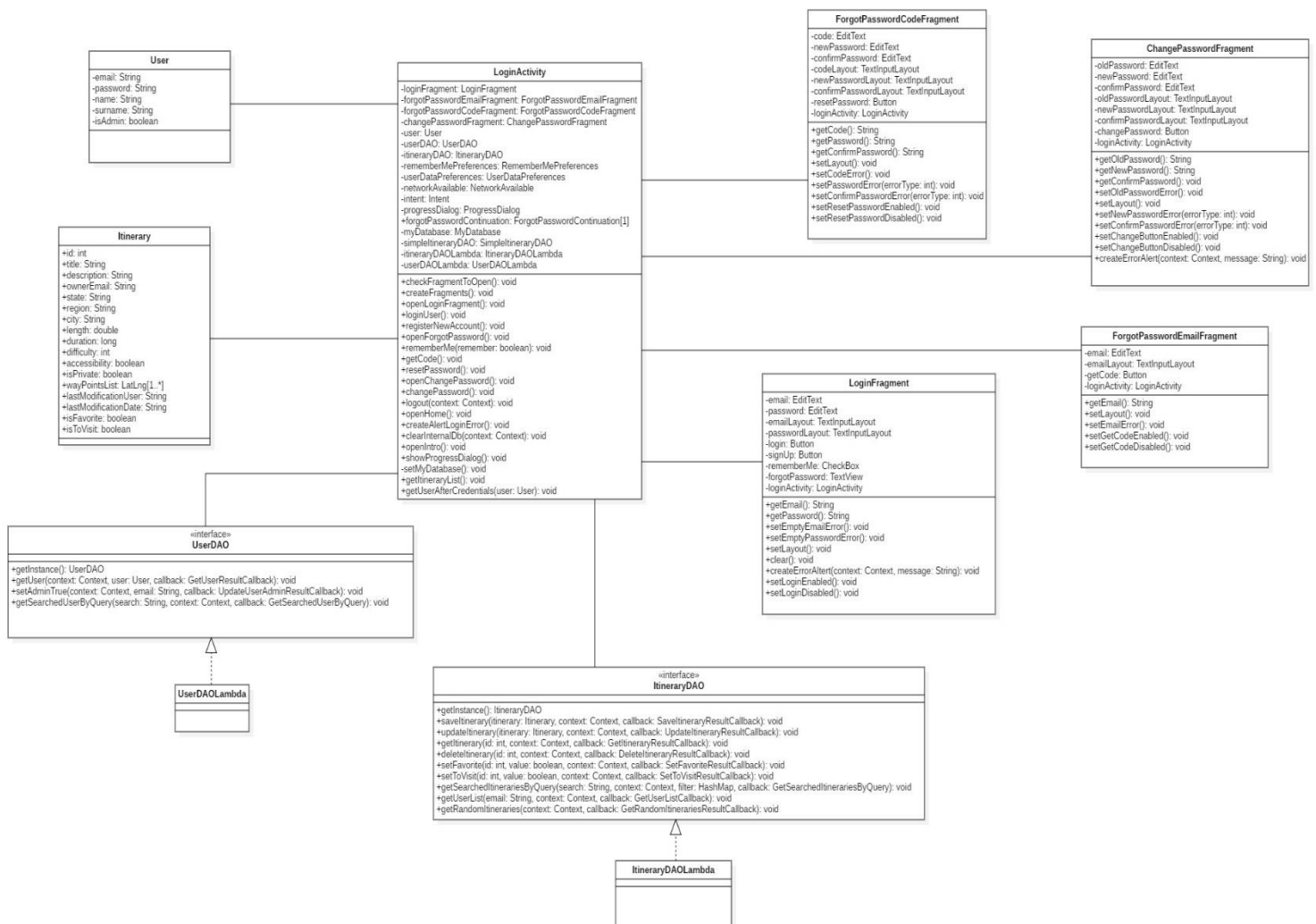
Per garantire la leggibilità, i class diagram sono stati divisi per funzionalità, e sono stati omessi metodi *getter*, *setter* e *override*.

Infine, dove possibile, le classi sono raggruppate per tipo.

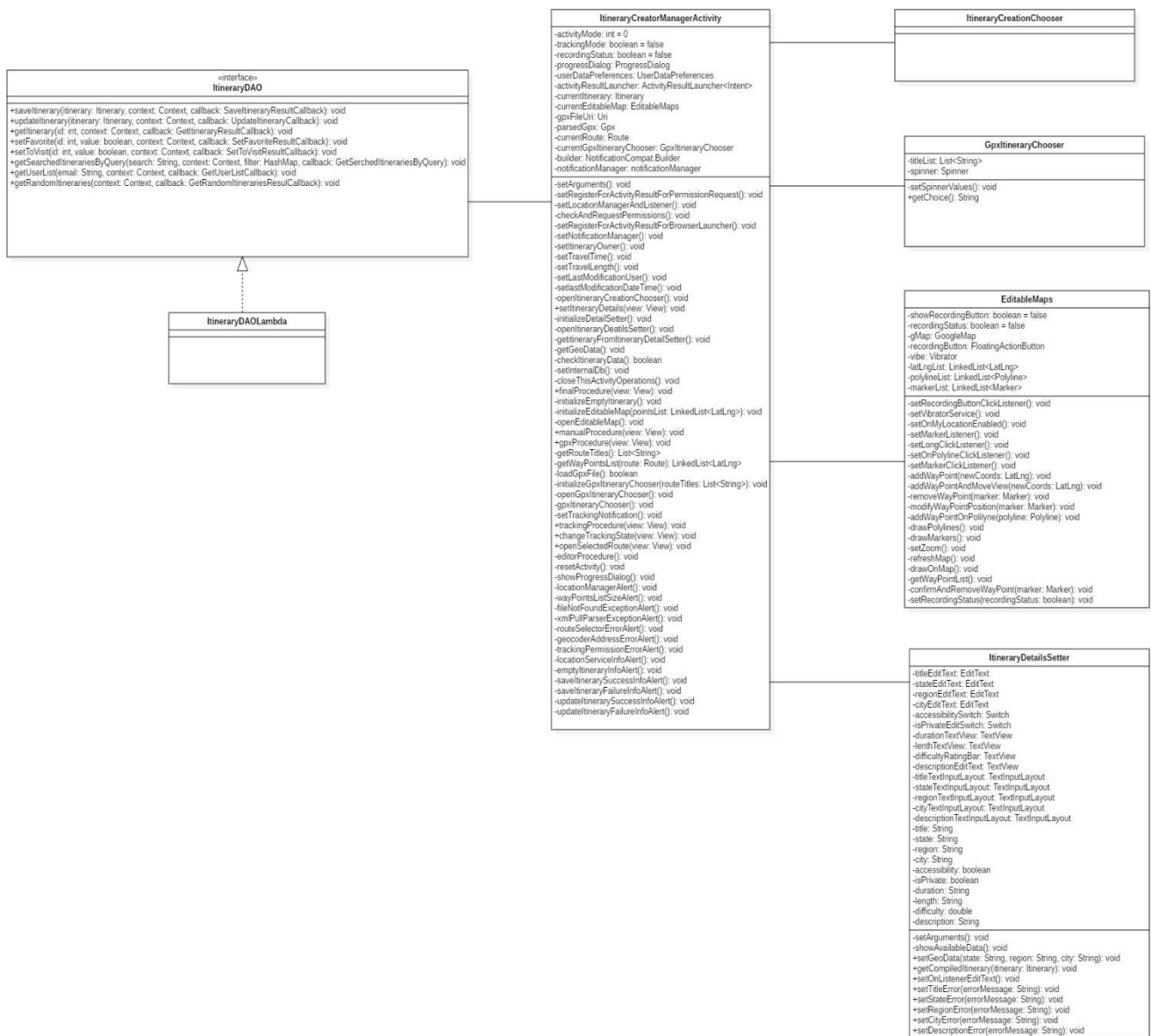
4.5.1 REGISTRAZIONE



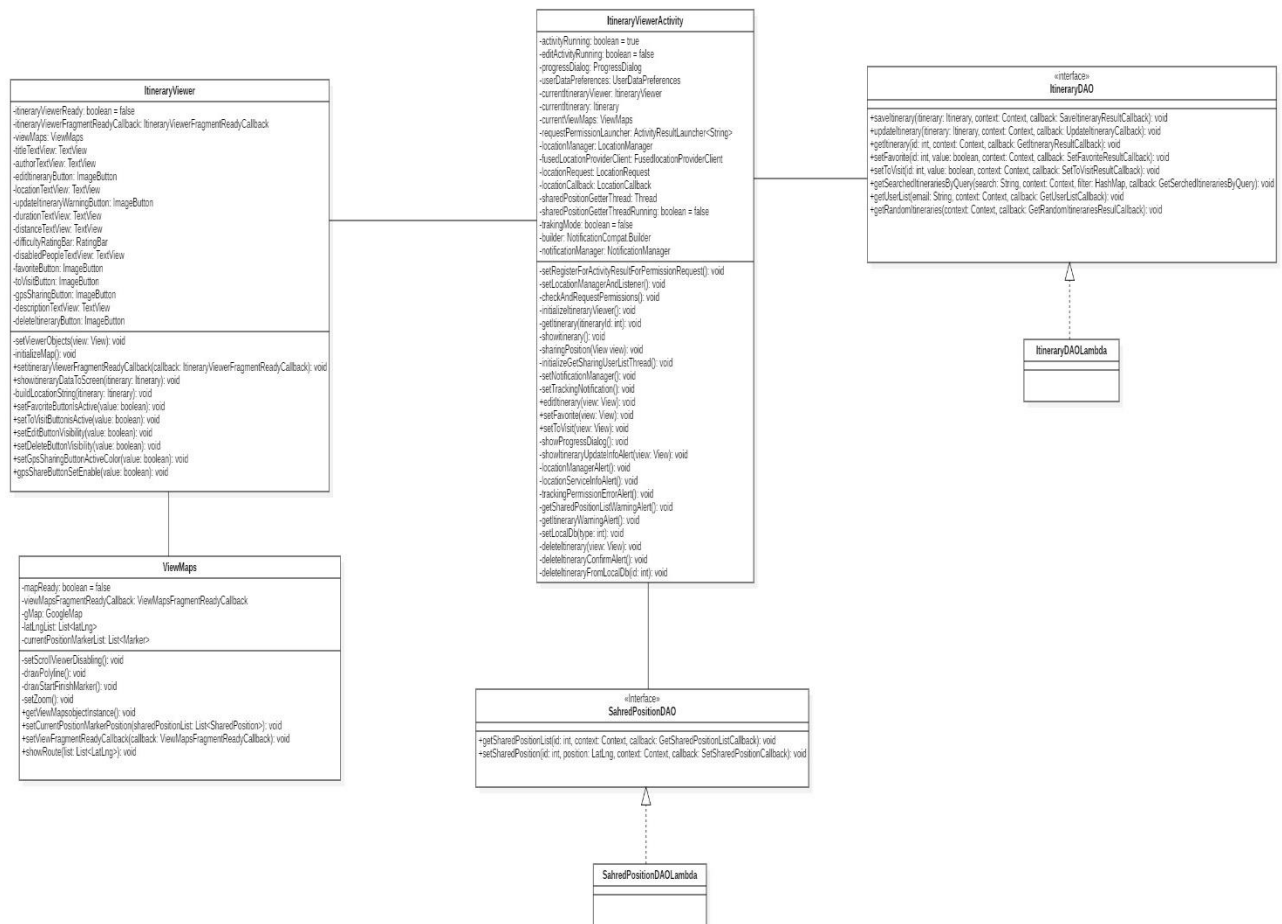
4.5.2 ACCESSO



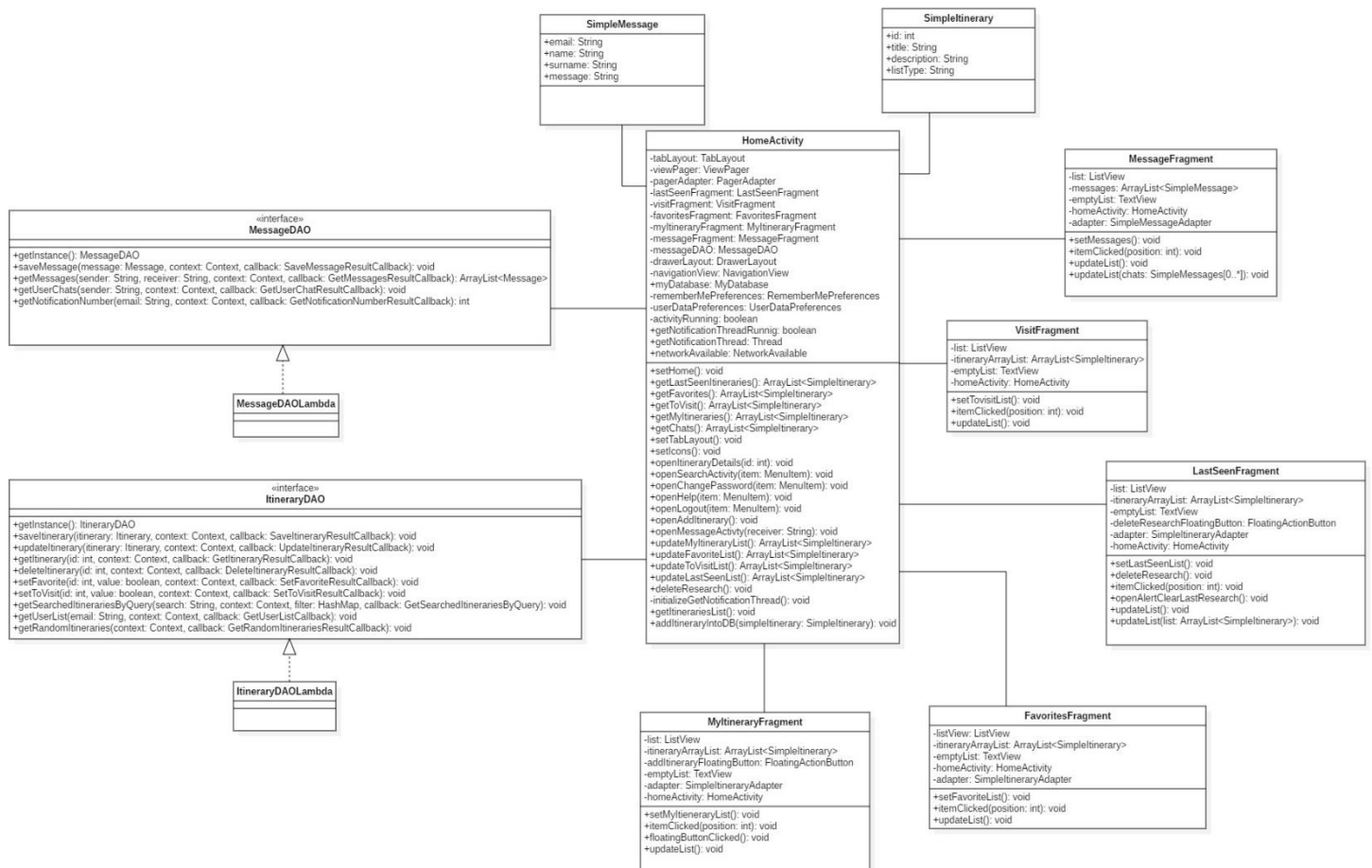
4.5.3 CREAZIONE ITINERARIO



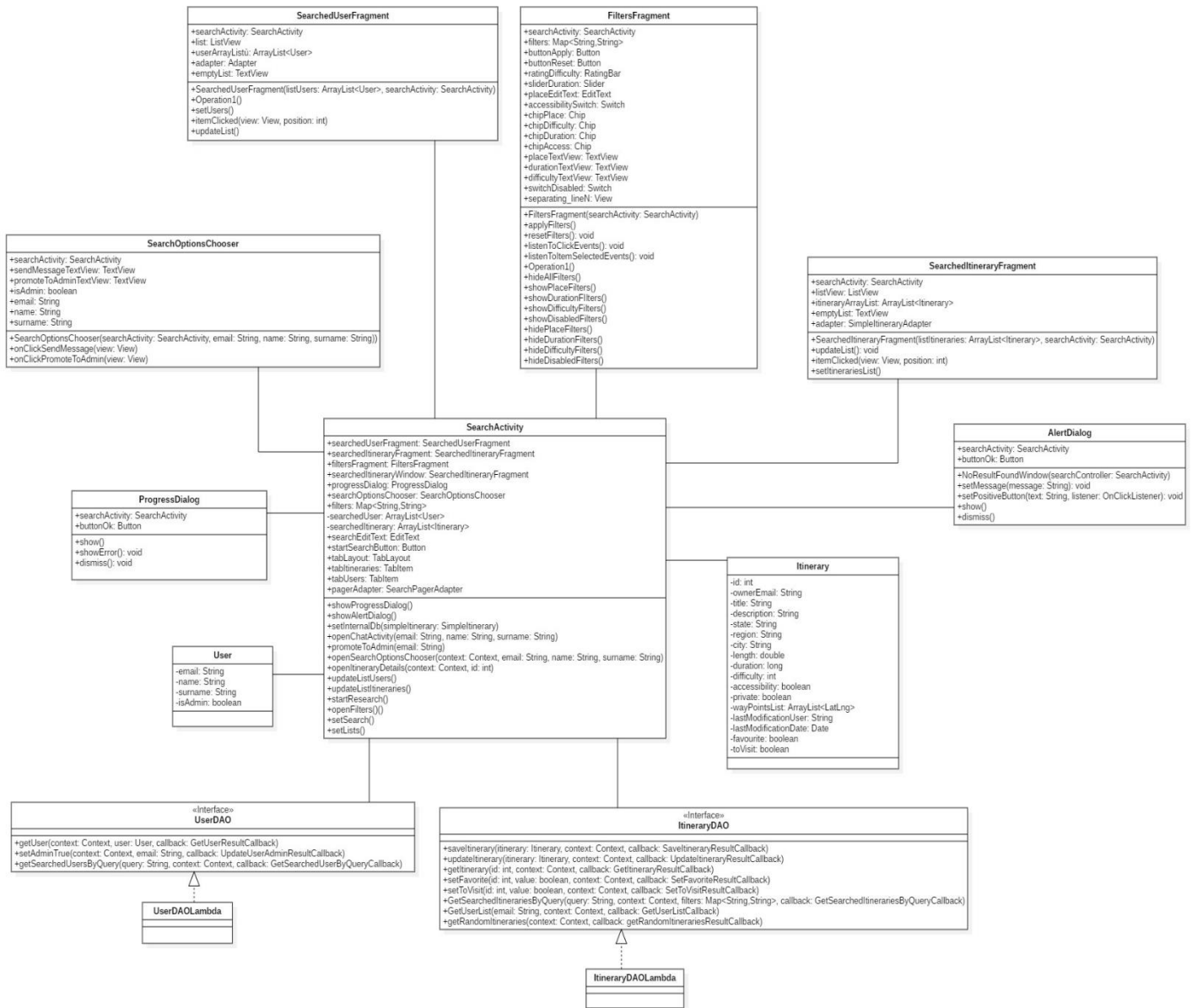
4.5.4 VISUALIZZAZIONE ITINERARIO



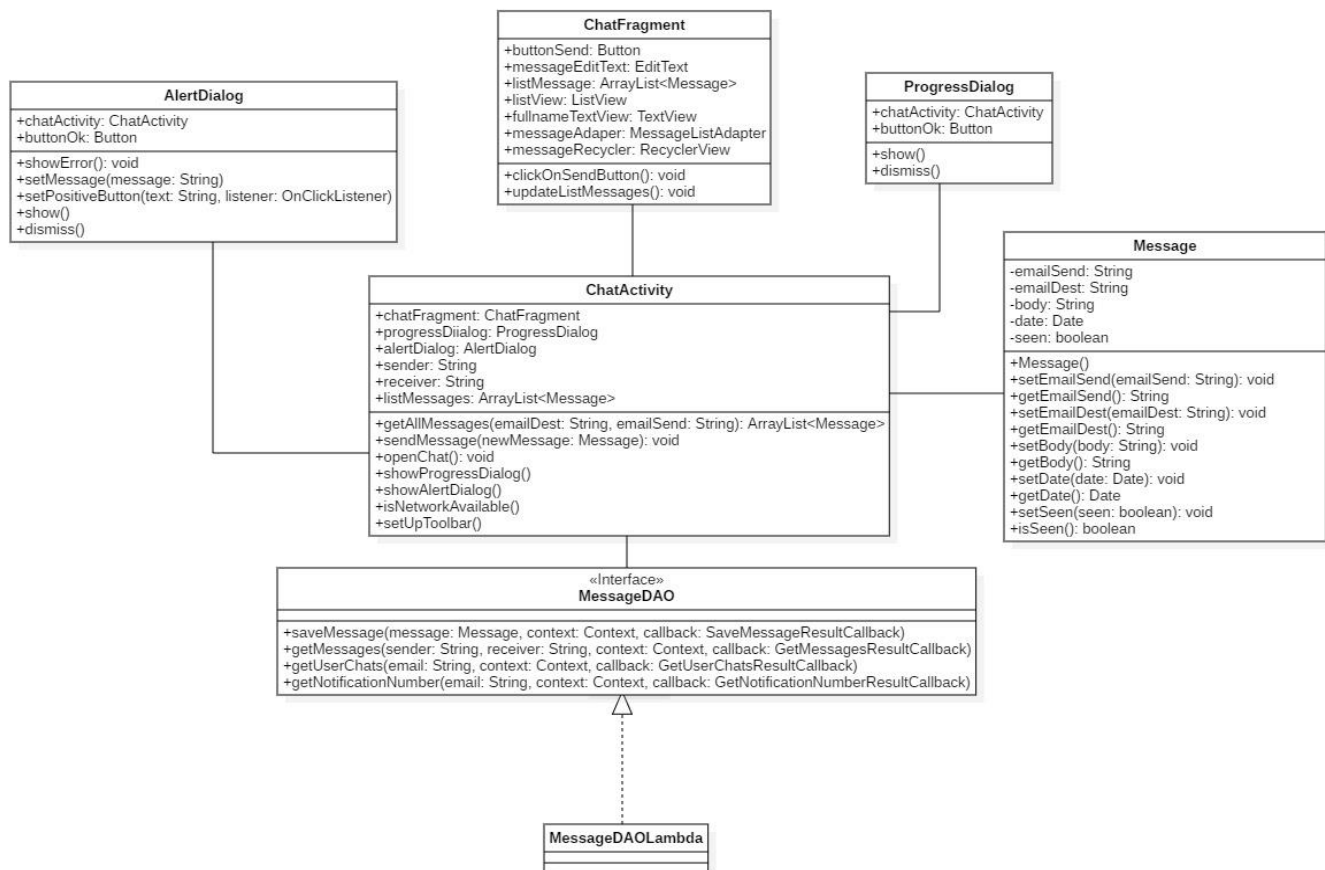
4.5.5 HOME



4.5.6 RICERCA



4.5.7 MESSAGGISTICA

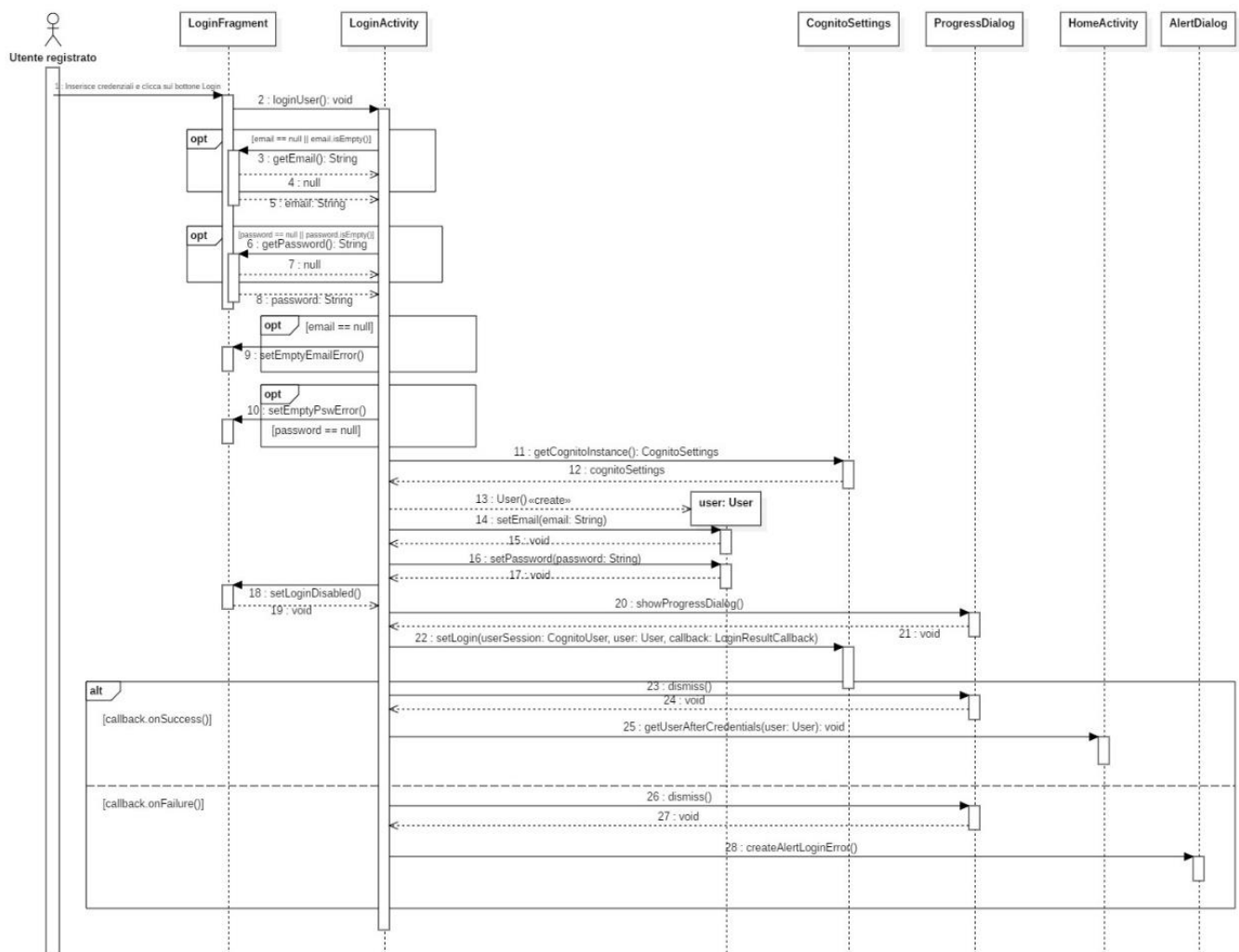


4.6 SEQUENCE DIAGRAM

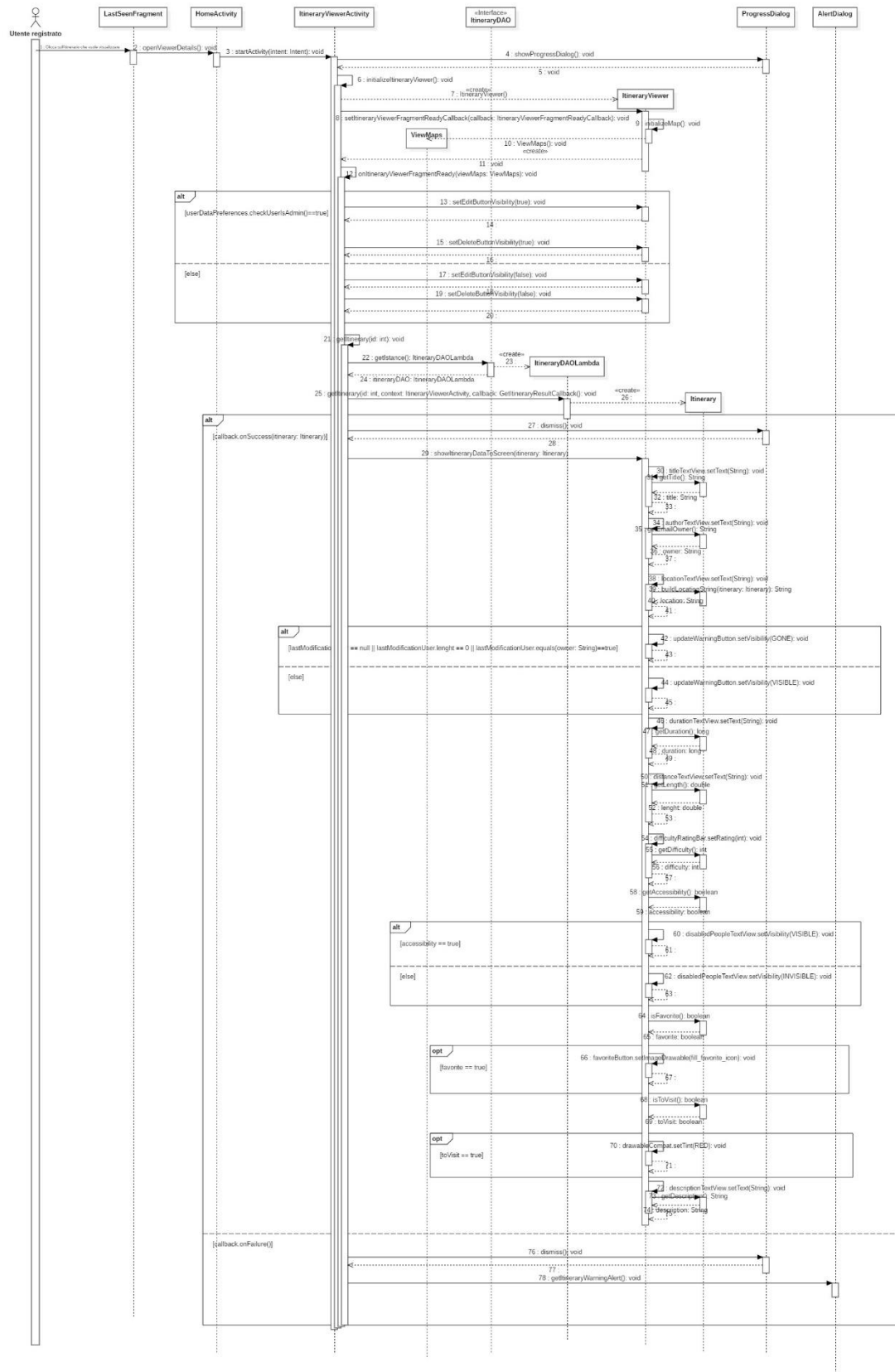
Di seguito vengono riportati i sequence diagram di design di due funzionalità.

Le funzionalità scelte sono la ricerca e l'accesso alla piattaforma.

4.6.1 ACCESSO



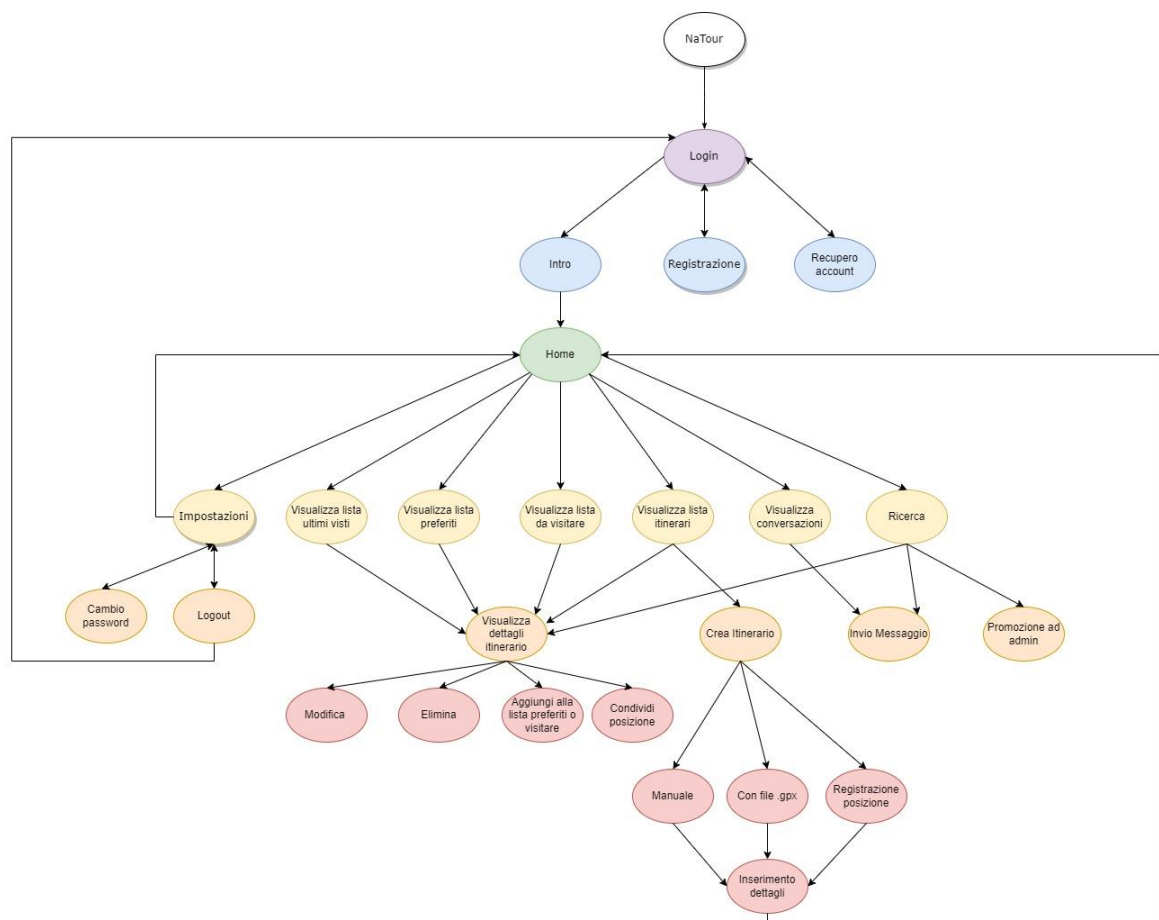
4.6.2 VISUALIZZAZIONE ITINERARIO



4.7 GERARCHIA FUNZIONALE

Di seguito viene riportata la gerarchia funzionale del sistema.

Le funzionalità di modifica ed eliminazione itinerario e quella di promozione ad amministratore sono funzionalità, come già riportato in precedenza, esclusive degli amministratori.



Capitolo 5

Testing

5.1 INTRODUZIONE

Dopo aver definito l'architettura del sistema, si passa al testing. In particolare verranno testati tre metodi dell'applicazione.

Verrà inoltre riportato il codice xUnit dei metodi testati.

5.2 METODO BUILDLOCATIONSTRING

Il metodo `buildLocationString()` si trova nella classe `ItineraryViewer` e viene usato per costruire la stringa di localizzazione nella schermata di visualizzazione dell'itinerario.

Il metodo richiede come parametro:

- `state` – lo stato in cui è situato l'itinerario;
- `region` – la regione in cui è situato l'itinerario;
- `city` – la città in cui è situato l'itinerario.



5.2.1 METODO

```

public String buildLocationString(String state, String region, String city) throws
NullPointerException{
1      if(state == null && region == null && city == null)
2          throw new NullPointerException();

3      String location = "";

4      if(city != null && city.length() > 0){
5          location = location.concat(city);
6          if(region == null & state == null)
7              return location;
8          else{
9              if((region != null && region.length()>0) || (state!=null && state.length() >0))
1             location = location.concat(", ");
11         }
12     }

13     if(region != null && region.length()>0) {
14         location = location.concat(region);
15         if(state == null)
16             return location;
17         else {
18             if(state.length() > 0)
19                 location = location.concat(", ");
20         }
21     }

22     if(state != null && state.length() > 0)
23         location = location.concat(state);

24     return location;
25 }

```



5.2.2 STRATEGIA BLACK BOX

Al fine di testare il metodo con la strategia Black Box, vengono individuate le seguenti classi di equivalenza:

Nome Classe	Parametro	Intervallo	Effetto
<i>CE₁</i>	state region city	{null} {null} {null}	Non valido.
<i>CE₂</i>	state region city	≠{null} ≠{null} ≠{null}	Valido.
<i>CE₃</i>	state region city	≠{null} ≠{null} {null}	Valido.
<i>CE₄</i>	state region city	≠{null} {null} ≠{null}	Valido.
<i>CE₅</i>	state region city	{null} ≠{null} ≠{null}	Valido.
<i>CE₆</i>	state region city	≠{null} {null} {null}	Valido.
<i>CE₇</i>	state region city	{null} ≠{null} {null}	Valido.
<i>CE₈</i>	state region city	{null} {null} ≠{null}	Valido.

È stata adottata la strategia WECT (Weak Equivalence Class Testing) e sono stati implementati e individuati 15 metodi di test.



5.2.3 METODI DI TEST BLACK BOX

```

public class ItineraryViewerTest {

    ItineraryViewer itineraryViewer;
    String state;
    String region;
    String city;

    @Before
    public void setUp() throws Exception {
        state = null;
        region = null;
        city = null;
        itineraryViewer = new ItineraryViewer();
    }

    @Test
    public void buildLocationStringComplete() {
        state = "Italia";
        region = "Campania";
        city = "Napoli";
        String build = itineraryViewer.buildLocationString(state,
region, city);
        assert(build.equals("Napoli, Campania, Italia"));
    }

    @Test
    public void buildLocationStringNoCity() {
        state = "Italia";
        region = "Campania";
        String build = itineraryViewer.buildLocationString(state,
region, city);
        assert(build.equals("Campania, Italia"));
    }

    @Test
    public void buildLocationStringNoRegion() {
        state = "Italia";
        city = "Napoli";
        String build = itineraryViewer.buildLocationString(state,
region, city);
        assert(build.equals("Napoli, Italia"));
    }

    @Test
    public void buildLocationStringNoState() {
        city = "Napoli";
        region = "Campania";
        String build = itineraryViewer.buildLocationString(state,
region, city);
        assert(build.equals("Napoli, Campania"));
    }
}

```



```

@Test
public void buildLocationStringState() {
    state = "Italia";
    String build = itineraryViewer.buildLocationString(state,
region, city);
    assert(build.equals("Italia"));
}

@Test
public void buildLocationStringRegion() {
    region = "Campania";
    String build = itineraryViewer.buildLocationString(state,
region, city);
    assert(build.equals("Campania"));
}

@Test
public void buildLocationStringCity() {
    city = "Napoli";
    String build = itineraryViewer.buildLocationString(state,
region, city);
    assert(build.equals("Napoli"));
}

@Test (expected = NullPointerException.class)
public void buildLocationStringNullStrings() {
    String build = itineraryViewer.buildLocationString(state,
region, city);
    fail();
}

@Test
public void buildLocationStringEmptyStrings() {
    state = "";
    region = "";
    city = "";
    String build = itineraryViewer.buildLocationString(state,
region, city);
    assert(build.equals(""));
}

@Test
public void buildLocationStringEmptyCity() {
    state = "Italia";
    region = "Campania";
    city = "";
    String build = itineraryViewer.buildLocationString(state,
region, city);
    assert(build.equals("Campania, Italia"));
}

@Test
public void buildLocationStringEmptyRegion() {

```



```

        state = "Italia";
        region = "";
        city = "Napoli";
        String build = itineraryViewer.buildLocationString(state,
region, city);
        assert(build.equals("Napoli, Italia"));
    }

    @Test
    public void buildLocationStringEmptyState() {
        state = "";
        region = "Campania";
        city = "Napoli";
        String build = itineraryViewer.buildLocationString(state,
region, city);
        assert(build.equals("Napoli, Campania"));
    }

    @Test
    public void buildLocationStringEmptyStateRegion() {
        state = "";
        region = "";
        city = "Napoli";
        String build = itineraryViewer.buildLocationString(state,
region, city);
        assert(build.equals("Napoli"));
    }

    @Test
    public void buildLocationStringEmptyCityRegion() {
        state = "Italia";
        region = "";
        city = "";
        String build = itineraryViewer.buildLocationString(state,
region, city);
        assert(build.equals("Italia"));
    }

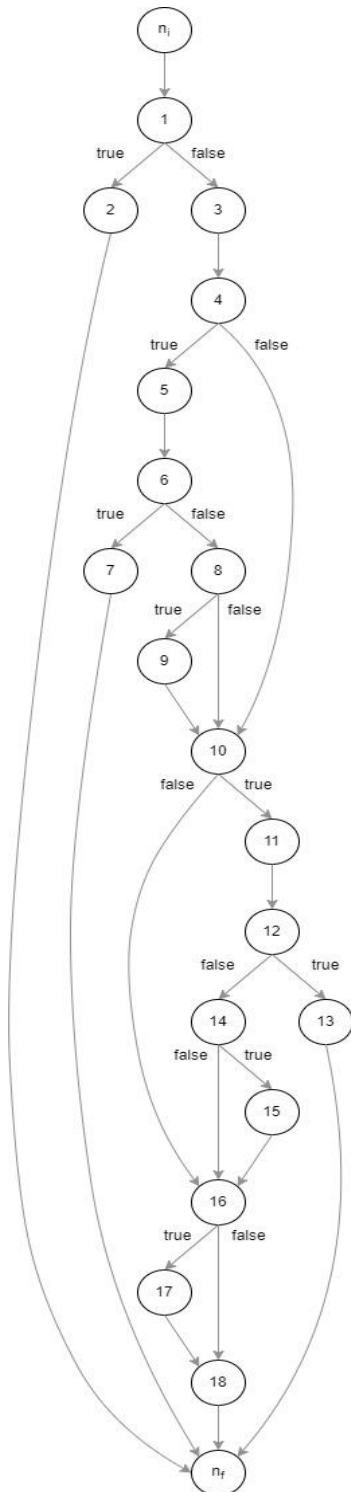
    @Test
    public void buildLocationStringEmptyStateCity() {
        state = "";
        region = "Campania";
        city = "";
        String build = itineraryViewer.buildLocationString(state,
region, city);
        assert(build.equals("Campania"));
    }
}

```



5.2.4 STRATEGIA WHITE BOX

Al fine di testare il metodo con la strategia White Box, è stato disegnato il seguente grafo per il flusso di controllo.



Dal seguente grafico sono stati ricavati i seguenti *metodi*:

- testGenerateBuildStringWhiteBoxPath_NI_1_2_NF ()
- testGenerateBuildStringWhiteBoxPath_NI_1_3_4_5_6_8_9_10_11_12_14_15_16_17_18_NF()
- testGenerateBuildStringWhiteBoxPath_NI_1_3_4_5_6_7_NF()
- testGenerateBuildStringWhiteBoxPath_NI_1_3_4_10_11_12_13_NF()
- testGenerateBuildStringWhiteBoxPath_NI_1_3_4_5_10_16_18_NF()
- testGenerateBuildStringWhiteBoxPath_NI_1_3_4_5_6_8_10_16_18_NF()
- testGenerateBuildStringWhiteBoxPath_NI_1_3_4_10_11_12_14_16_18_NF()

La strategia adottata per i metodi di test è quella della copertura dei cammini.

Per l'imitare l'insieme dei cammini è stato utilizzato il metodo dei *cammini linearmente indipendenti* di McCabe. Inoltre, utilizzando il metodo di McCabe, viene implicata anche la copertura per branch.



5.2.5 METODI DI TEST WHITE BOX

```

public class ItineraryViewerTestWhite {

    ItineraryViewer itineraryViewer;
    String state;
    String region;
    String city;

    @Before
    public void setUp() throws Exception {
        state = null;
        region = null;
        city = null;
        itineraryViewer = new ItineraryViewer();
    }

    @Test (expected = NullPointerException.class)
    public void testGenerateBuildStringWhiteBoxPath_NI_1_2_NF() {
        String build = itineraryViewer.buildLocationString(state,
region, city);
        fail();
    }

    @Test
    public void
testGenerateBuildStringWhiteBoxPath_NI_1_3_4_5_6_8_9_10_11_12_14_15_1
6_17_18_NF() {
        state = "Italia";
        region = "Campania";
        city = "Napoli";
        String build = itineraryViewer.buildLocationString(state,
region, city);
        assert (build.equals("Napoli, Campania, Italia"));
    }

    @Test
    public void
testGenerateBuildStringWhiteBoxPath_NI_1_3_4_5_6_7_NF() {
        city = "Napoli";
        String build = itineraryViewer.buildLocationString(state,
region, city);
        assert (build.equals("Napoli"));
    }

    @Test
    public void
testGenerateBuildStringWhiteBoxPath_NI_1_3_4_10_11_12_13_NF() {
        region = "Campania";
        String build = itineraryViewer.buildLocationString(state,
city, region);
        assert (build.equals("Campania"));
    }
}

```



```
@Test
    public void
testGenerateBuildStringWhiteBoxPath_NI_1_3_4_5_10_16_18_NF() {
    state = "Italia";
    String build = itineraryViewer.buildLocationString(state,
region, city);
    assert (build.equals("Italia"));
}

@Test
    public void
testGenerateBuildStringWhiteBoxPath_NI_1_3_4_5_6_8_10_16_18_NF() {
    city = "Napoli";
    region = "";
    state = "";
    String build = itineraryViewer.buildLocationString(state,
region, city);
    assert (build.equals("Napoli"));
}

@Test
    public void
testGenerateBuildStringWhiteBoxPath_NI_1_3_4_10_11_12_14_16_18_NF() {
    state = "";
    region = "Campania";
    String build = itineraryViewer.buildLocationString(state,
region, city);
    assert (build.equals("Campania"));
}
}
```



5.3 METODO CREATEFILTERQUERY

Il metodo `createFilterQuery()` si trova nella classe `QueryGenerator` e viene utilizzato per generare la query da passare al DAO per la ricerca di un itinerario, secondo i filtri scelti.

Il metodo prendere come parametro:

- `search` – stringa di ricerca dell'utente;
- `filters` – una mappa contenente i filtri e il loro rispettivo valore.



5.3.1 METODO

```

public String createFilterQuery(String search, Map<String, String> filter){

1      if(search == null)
2          throw new NullPointerException();

3      String query = "SELECT I.id, I.title, I.description FROM Itinerary I WHERE
I.private = FALSE AND I.title LIKE '%" + search + "%'";

4      if(filter == null)
5          return query + ";";

6      if(filter.containsKey("Place")) {
7          if (!(filter.get("Place").isEmpty())) {
8              query = query + " AND (I.State LIKE '%" + filter.get("Place") + "%' OR
I.Region LIKE '%" + filter.get("Place") + "%' OR I.City LIKE '%" + filter.get("Place") +
"%')";
          }
      }

9      if(filter.containsKey("Difficulty")) {
10         if ((Integer.valueOf(filter.get("Difficulty")) > 0 &&
(Integer.valueOf(filter.get("Difficulty")) < 6)) {
11             query = query + " AND I.Difficulty <= " + filter.get("Difficulty");
12         } else
13             throw new IllegalArgumentException();
14     }

15     if(filter.containsKey("Duration")) {
16         if ((Integer.valueOf(filter.get("Duration")) > 0 &&
(Integer.valueOf(filter.get("Duration")) < 11)) {
17             int durationInSeconds = Integer.valueOf(filter.get("Duration")) * 3600;
18             query = query + " AND I.Duration <= " + durationInSeconds;
19         } else
20             throw new IllegalArgumentException();
21     }

22     if(filter.containsKey("DisabledAccess")) {
23         query = query + " AND I.access_dis = " + filter.get("DisabledAccess");
24     }

25     query = query + ";";

26     return query;
}

```



5.3.2 STRATEGIA BLACK BOX

Al fine di testare il metodo con la strategia Black Box, vengono individuate le seguenti classi di equivalenza:

Nome Classe	Parametro	Intervallo	Effetto
<i>CE₁</i>	search	{null}	Non valido.
<i>CE₂</i>	search	≠{null}	Valido.
<i>CE₃</i>	filters	{null}	Valido.
<i>CE₄</i>	filters	≠{null}	Valido.
<i>CE₅</i>	filters.get("Duration")	[MIN_INT, 0]	Non valido.
<i>CE₆</i>	filters.get("Duration")	]0, 11[	Valido.
<i>CE₇</i>	filters.get("Duration")	[11, MAX_INT]	Non valido.
<i>CE₈</i>	filters.get("Difficulty")	[MIN_INT, 0[	Non valido.
<i>CE₉</i>	filters.get("Difficulty")	]0, 6[	Valido.
<i>CE₁₀</i>	filters.get("Difficulty")	[6, MAX_INT]	Non valido.

È stata adottata la strategia WECT (Weak Equivalence Class Testing) e sono stati implementati e individuati 23 metodi di test.



5.3.3 METODI DI TEST

```

public class QueryGeneratorTest {

    QueryGenerator queryGenerator;
    String search;
    String place;
    String duration;
    String difficulty;
    String disabledAccess;
    Map<String, String> filters;

    @Before
    public void setUp() throws Exception {
        queryGenerator = new QueryGenerator();
        search = "Diego";
        place = "Napoli";
        duration = "8";
        difficulty = "3";
        disabledAccess = "true";
    }

    @Test
    public void createFilterQueryNoFilter() {
        filters = null;
        assert (queryGenerator.createFilterQuery(search,
filters).equals("SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'"));
    }

    @Test (expected = NullPointerException.class)
    public void createFilterQueryStringNull() {
        search = null;
        filters = null;
        String query = queryGenerator.createFilterQuery(search,
filters);
        fail();
    }

    @Test
    public void createFilterQueryEmptyString() {
        search = "";
        filters = null;
        assert (queryGenerator.createFilterQuery(search,
filters).equals("SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%%'"));
    }

    @Test
    public void createFilterQueryFullFilters() {
        filters = new HashMap<>();
        filters.put("Place", place);
        filters.put("Duration", duration);
    }
}

```



```

        filters.put("Difficulty", difficulty);
        filters.put("DisabledAccess", disabledAccess);
        assert (queryGenerator.createFilterQuery(search,
filters).equals(
            "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
        + " AND (I.State LIKE '%Napoli%' OR I.Region LIKE
'%Napoli%' OR I.City LIKE '%Napoli%') "
        + " AND I.Difficulty <= 3"
        + " AND I.Duration <= 28800"
        + " AND I.access_dis = true;"
        ));
    }

    @Test
    public void createFilterQueryNoPlace() {
        filters = new HashMap<>();
        filters.put("Duration", duration);
        filters.put("Difficulty", difficulty);
        filters.put("DisabledAccess", disabledAccess);
        assert (queryGenerator.createFilterQuery(search,
filters).equals(
            "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
        + " AND I.Difficulty <= 3"
        + " AND I.Duration <= 28800"
        + " AND I.access_dis = true;"
        ));
    }

    @Test
    public void createFilterQueryNoDuration() {
        filters = new HashMap<>();
        filters.put("Place", place);
        filters.put("Difficulty", difficulty);
        filters.put("DisabledAccess", disabledAccess);
        assert (queryGenerator.createFilterQuery(search,
filters).equals(
            "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
        + " AND (I.State LIKE '%Napoli%' OR I.Region
LIKE '%Napoli%' OR I.City LIKE '%Napoli%') "
        + " AND I.Difficulty <= 3"
        + " AND I.access_dis = true;"
        ));
    }

    @Test
    public void createFilterQueryNoDifficulty() {
        filters = new HashMap<>();
        filters.put("Place", place);
        filters.put("Duration", duration);
        filters.put("DisabledAccess", disabledAccess);

```



```

        assert (queryGenerator.createFilterQuery(search,
filters).equals(
        "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
        + " AND (I.State LIKE '%Napoli%' OR I.Region
LIKE '%Napoli%' OR I.City LIKE '%Napoli%') "
        + " AND I.Duration <= 28800"
        + " AND I.access_dis = true;"
    ));
    }

    @Test
    public void createFilterQueryNoDisabledAccess() {
        filters = new HashMap<>();
        filters.put("Place", place);
        filters.put("Duration", duration);
        filters.put("Difficulty", difficulty);
        assert (queryGenerator.createFilterQuery(search,
filters).equals(
        "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
        + " AND (I.State LIKE '%Napoli%' OR I.Region
LIKE '%Napoli%' OR I.City LIKE '%Napoli%') "
        + " AND I.Difficulty <= 3"
        + " AND I.Duration <= 28800;"
    ));
    }

    @Test
    public void createFilterQueryDifficultyAndDisablesAccess() {
        filters = new HashMap<>();

        filters.put("Difficulty", difficulty);
        filters.put("DisabledAccess", disabledAccess);
        assert (queryGenerator.createFilterQuery(search,
filters).equals(
        "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
        + " AND I.Difficulty <= 3"
        + " AND I.access_dis = true;"
    ));
    }

    @Test
    public void createFilterQueryPlaceAndDuration() {
        filters = new HashMap<>();
        filters.put("Place", place);
        filters.put("Duration", duration);
        assert (queryGenerator.createFilterQuery(search,
filters).equals(
        "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
        + " AND (I.State LIKE '%Napoli%' OR I.Region
LIKE '%Napoli%' OR I.City LIKE '%Napoli%') "

```



```

        + " AND I.Duration <= 28800;"
    ));
}

@Test
public void createFilterQueryPlaceAndDisabledAccess() {
    filters = new HashMap<>();
    filters.put("Place", place);
    filters.put("DisabledAccess", disabledAccess);

    assert (queryGenerator.createFilterQuery(search,
filters).equals(
        "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
        + " AND (I.State LIKE '%Napoli%' OR I.Region
LIKE '%Napoli%' OR I.City LIKE '%Napoli%') "
        + " AND I.access_dis = true;"
    ));
}

@Test
public void createFilterQueryPlaceAndDifficulty() {
    filters = new HashMap<>();
    filters.put("Place", place);
    filters.put("Difficulty", difficulty);
    assert (queryGenerator.createFilterQuery(search,
filters).equals(
        "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
        + " AND (I.State LIKE '%Napoli%' OR I.Region
LIKE '%Napoli%' OR I.City LIKE '%Napoli%') "
        + " AND I.Difficulty <= 3;"
    ));
}

@Test
public void createFilterQueryDurationAndDifficulty() {
    filters = new HashMap<>();
    filters.put("Duration", duration);
    filters.put("Difficulty", difficulty);

    assert (queryGenerator.createFilterQuery(search,
filters).equals(
        "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
        + " AND I.Difficulty <= 3"
        + " AND I.Duration <= 28800;"
    ));
}

@Test
public void createFilterQueryDurationAndDisabledAccess() {
    filters = new HashMap<>();
    filters.put("Duration", duration);

```



```

        filters.put("DisabledAccess", disabledAccess);
        assert (queryGenerator.createFilterQuery(search,
filters).equals(
            "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
            + " AND I.Duration <= 28800"
            + " AND I.access_dis = true;"
        ));
    }

    @Test
    public void createFilterQueryPlace() {
        filters = new HashMap<>();
        filters.put("Place", place);
        assert (queryGenerator.createFilterQuery(search,
filters).equals(
            "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
            + " AND (I.State LIKE '%Napoli%' OR I.Region
LIKE '%Napoli%' OR I.City LIKE '%Napoli%');"
        ));
    }

    @Test
    public void createFilterQueryDifficulty() {
        filters = new HashMap<>();
        filters.put("Difficulty", difficulty);
        assert (queryGenerator.createFilterQuery(search,
filters).equals(
            "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
            + " AND I.Difficulty <= 3;"
        ));
    }

    @Test
    public void createFilterQueryDuration() {
        filters = new HashMap<>();
        filters.put("Duration", duration);
        assert (queryGenerator.createFilterQuery(search,
filters).equals(
            "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
            + " AND I.Duration <= 28800;"
        ));
    }

    @Test
    public void createFilterQueryDisabledAccess() {
        filters = new HashMap<>();
        filters.put("DisabledAccess", disabledAccess);
        assert (queryGenerator.createFilterQuery(search,
filters).equals(

```



```

        "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%'
        + " AND I.access_dis = true;"

    ));
}

@Test
public void createFilterQueryEmptyStringPlace() {
    filters = new HashMap<>();
    filters.put("Place", "");

    assert (queryGenerator.createFilterQuery(search,
filters).equals(
        "SELECT I.id, I.title, I.description FROM Itinerary I
WHERE I.private = FALSE AND I.title LIKE '%Diego%';"
    ));
}

@Test (expected = IllegalArgumentException.class)
public void createFilterQueryNotValidLessDuration() {
    filters = new HashMap<>();
    filters.put("Place", place);
    filters.put("Duration", "0");
    filters.put("Difficulty", difficulty);
    filters.put("DisabledAccess", disabledAccess);

    String query = queryGenerator.createFilterQuery(search,
filters);
    fail();
}

@Test (expected = IllegalArgumentException.class)
public void createFilterQueryNotValidLessDifficulty() {
    filters = new HashMap<>();
    filters.put("Place", place);
    filters.put("Duration", duration);
    filters.put("Difficulty", "0");
    filters.put("DisabledAccess", disabledAccess);

    String query = queryGenerator.createFilterQuery(search,
filters);
    fail();
}

@Test (expected = IllegalArgumentException.class)
public void createFilterQueryNotValidMoreDuration() {
    filters = new HashMap<>();
    filters.put("Place", place);
    filters.put("Duration", "11");
    filters.put("Difficulty", difficulty);
    filters.put("DisabledAccess", disabledAccess);

```




```
        String query = queryGenerator.createFilterQuery(search,
filters);
        fail();

    }

    @Test (expected = IllegalArgumentException.class)
    public void createFilterQueryNotValidMoreDifficulty() {
        filters = new HashMap<>();
        filters.put("Place", place);
        filters.put("Duration", duration);
        filters.put("Difficulty", "6");
        filters.put("DisabledAccess", disabledAccess);

        String query = queryGenerator.createFilterQuery(search,
filters);
        fail();

    }

}
```



5.4 METODO CHECKPASSWORD

Il metodo `checkPassword` si trova nella classe `PasswordChecker` e viene utilizzato per effettuare il controllo della validità della password.

Il metodo `checkPassword` richiede due parametri:

- `newPassword` – la nuova password scelta dall'utente;
- `confirmPassword` – la conferma della nuova password.

5.4.1 METODO

```
public boolean checkPassword(String newPassword, String confirmPassword) throws
PasswordNotCorrectException {

1      if(confirmPassword == null || newPassword == null)
2          throw new NullPointerException();

3      final Pattern pattern = Pattern.compile(PASSWORD_PATTERN);

4      Matcher matcher = pattern.matcher(newPassword);
5      if(!matcher.matches()){
6          throw new PasswordNotCorrectException();
7      }

8      matcher = pattern.matcher(confirmPassword);
9      if(!matcher.matches()){
10         throw new PasswordNotCorrectException();
11     }

12     return newPassword.equals(confirmPassword);
13 }
```



5.4.2 STRATEGIA BLACK BOX

Al fine di testare il metodo con la strategia Black Box, vengono individuate le seguenti classi di equivalenza:

(NB: in giallo viene sottolineata la parte mancante del pattern.)

Nome Classe	Parametro	Intervallo	Effetto
CE ₁	newPassword	{null}	Non valida.
CE ₂	newPassword	{^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#&()._-[]{};:',?/*~\$^+=<>]).{8,16}\$}	Non valida.
CE ₃	newPassword	{^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#&()._-[]{};:',?/*~\$^+=<>]).{8,16}\$}	Non valida.
CE ₄	newPassword	{^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#&()._-[]{};:',?/*~\$^+=<>]).{8,16}\$}	Non valida.
CE ₅	newPassword	{^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#&()._-[]{};:',?/*~\$^+=<>]).{8,16}\$}	Non valida.
CE ₆	newPassword	{^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#&()._-[]{};:',?/*~\$^+=<>]).{8,16}\$}	Non valida.
CE ₇	newPassword	{^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#&()._-[]{};:',?/*~\$^+=<>]).{8,16}\$}	Non valida.
CE ₈	confirmPassword	{null}	Non valida.
CE ₉	confirmPassword	{^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#&()._-[]{};:',?/*~\$^+=<>]).{8,16}\$}	Non valida.
CE ₁₀	confirmPassword	{^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#&()._-[]{};:',?/*~\$^+=<>]).{8,16}\$}	Non valida.



CE ₁₁	confirmPassword	{ ^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#&()_~{};:','/*~\$^+=<>]).{8,16}\$}	Non valida.
CE ₁₂	confirmPassword	{ ^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#&()_~{};:','/*~\$^+=<>]).{8,16}\$}	Non valida.
CE ₁₃	confirmPassword	{ ^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#&()_~{};:','/*~\$^+=<>]).{8,16}\$}	Non valida.
CE ₁₄	confirmPassword	{ ^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#&()_~{};:','/*~\$^+=<>]).{8,16}\$}	Non valida.
CE ₁₅	newPassword, confirmPassword	{ ^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!@#&()_~{};:','/*~\$^+=<>]).{8,16}\$}	Valida.

È stata adottata la strategia WECT (Weak Equivalence Class Testing) e sono stati implementati e individuati 13 metodi di test.



5.4.3 METODI DI TEST BLACK BOX

```

public class PasswordCheckerTest {

    PasswordChecker passwordChecker;
    String newPassword;
    String confirmPassword;

    @Before
    public void setUp() throws Exception {
        passwordChecker = new PasswordChecker();
    }

    @Test (expected = NullPointerException.class)
    public void checkPasswordNewPasswordNull() throws
    PasswordNotCorrectException {
        newPassword = null;
        confirmPassword = "CiaoCiao.00";
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }

    @Test (expected = NullPointerException.class)
    public void checkPasswordConfirmPasswordNull() throws
    PasswordNotCorrectException {
        confirmPassword = null;
        newPassword = "CiaoCiao.00";
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }

    @Test (expected = NullPointerException.class)
    public void checkPasswordBothPasswordNull() throws
    PasswordNotCorrectException {
        confirmPassword = null;
        newPassword = null;
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }

    @Test
    public void checkPasswordCorrectPasswordSame() throws
    PasswordNotCorrectException {
        confirmPassword = "CiaoCiao.00";
        newPassword = "CiaoCiao.00";

        assertTrue (passwordChecker.checkPassword(newPassword,
        confirmPassword));
    }

    @Test
    public void checkPasswordCorrectPasswordNotSame() throws
    PasswordNotCorrectException {
        confirmPassword = "CiaoCiao.00";

```



```

        newPassword = "CiaoCiao.99";

        assertFalse (passwordChecker.checkPassword(newPassword,
confirmPassword));
    }

    @Test (expected = PasswordNotCorrectException.class)
    public void checkPasswordNewPasswordNotMatchingPattern() throws
PasswordNotCorrectException {
        confirmPassword = "CiaoCiao.00";
        newPassword = "CiaoCiao00";
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }

    @Test (expected = PasswordNotCorrectException.class)
    public void checkPasswordConfirmPasswordNotMatchingPattern()
throws PasswordNotCorrectException {
        confirmPassword = "CiaoCiao00";
        newPassword = "CiaoCiao.00";
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }

    @Test (expected = PasswordNotCorrectException.class)
    public void checkPasswordBothNotMatchingPattern() throws
PasswordNotCorrectException {
        confirmPassword = "CiaoCiao00";
        newPassword = "CiaoCiao00";
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }

    @Test (expected = PasswordNotCorrectException.class)
    public void checkPasswordNotAcceptedLowercase() throws
PasswordNotCorrectException {
        confirmPassword = "ciaociao.00";
        newPassword = "ciaociao.00";
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }

    @Test (expected = PasswordNotCorrectException.class)
    public void checkPasswordNotAcceptedUppercase() throws
PasswordNotCorrectException {
        confirmPassword = "CIAOCIAO.00";
        newPassword = "CIAOCIAO.00";
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }

    @Test (expected = PasswordNotCorrectException.class)
    public void checkPasswordNoNumber() throws
PasswordNotCorrectException {

```



```
        confirmPassword = "CiaoCiao.";
        newPassword = "CiaoCiao.";
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }

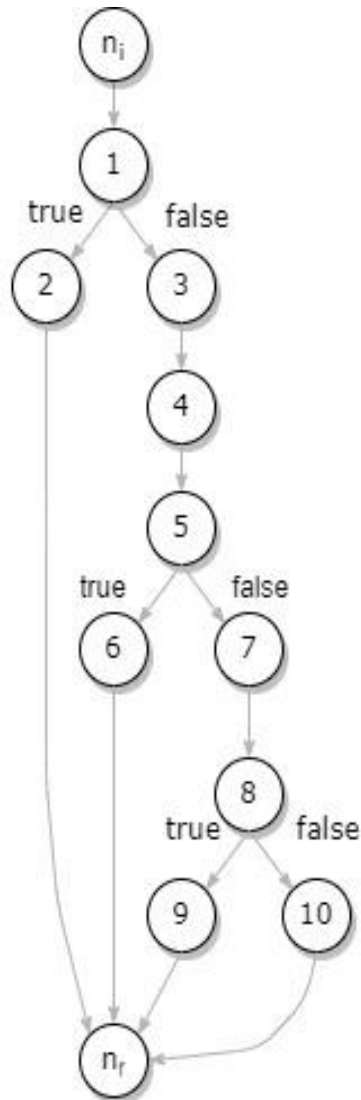
    @Test (expected = PasswordNotCorrectException.class)
    public void checkPasswordShort() throws
    PasswordNotCorrectException {
        confirmPassword = "Ciao.00";
        newPassword = "Ciao.00";
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }

    @Test (expected = PasswordNotCorrectException.class)
    public void checkPasswordLong() throws
    PasswordNotCorrectException {
        confirmPassword = "CiaoCiaoCiao.0000";
        newPassword = "CiaoCiaoCiao.0000";
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }
}
```



5.4.4 STRATEGIA WHITE BOX

Al fine di testare il metodo con la strategia White Box, è stato disegnato il seguente grafo per il flusso di controllo.



Dal seguente grafico sono stati ricavati i seguenti *metodi*:

- testGenerateCheckPasswordWhiteBoxPath_NI_1_2_NF()
- testGenerateCheckPasswordWhiteBoxPath_NI_1_3_4_5_6_NF()
- testGenerateCheckPasswordWhiteBoxPath_NI_1_3_4_5_7_8_9_NF()
- testGenerateCheckPasswordWhiteBoxPath_NI_1_3_4_5_7_8_10_NF()

La strategia adottata per i metodi di test è quella della copertura dei cammini.

Per l'imitare l'insieme dei cammini è stato utilizzato il metodo dei *cammini linearmente indipendenti di McCabe*. Inoltre, utilizzando il metodo di McCabe, viene implicata anche la copertura per branch.



5.4.5 METODI DI TEST WHITE BOX

```

public class PasswordCheckerTestWhite {

    PasswordChecker passwordChecker;
    String newPassword;
    String confirmPassword;

    @Before
    public void setUp() throws Exception {
        passwordChecker = new PasswordChecker();
    }

    @Test (expected = NullPointerException.class)
    public void testGenerateCheckPasswordWhiteBoxPath_NI_1_2_NF()
    throws PasswordNotCorrectException {
        newPassword = null;
        confirmPassword = "CiaoCiao.00";
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }

    @Test (expected = PasswordNotCorrectException.class)
    public void
    testGenerateCheckPasswordWhiteBoxPath_NI_1_3_4_5_6_NF() throws
    PasswordNotCorrectException {
        newPassword = "Ciao.0";
        confirmPassword = "CiaoCiao.00";
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }

    @Test (expected = PasswordNotCorrectException.class)
    public void
    testGenerateCheckPasswordWhiteBoxPath_NI_1_3_4_5_7_8_9_NF() throws
    PasswordNotCorrectException {
        newPassword = "CiaoCiao.00";
        confirmPassword = "Ciao.0";
        passwordChecker.checkPassword(newPassword, confirmPassword);
        fail();
    }

    @Test
    public void
    testGenerateCheckPasswordWhiteBoxPath_NI_1_3_4_5_7_8_10_NF() throws
    PasswordNotCorrectException {
        newPassword = "CiaoCiao.00";
        confirmPassword = "CiaoCiao.00";

        assertTrue (passwordChecker.checkPassword(newPassword,
        confirmPassword));
    }
}

```



5.5 VALUTAZIONE USABILITÀ

Durante lo sviluppo l'applicazione è stata fatta provare ad un ristretto bacino d'utenti, per avere un primo riscontro sulla semplicità del sistema e per raccogliere le opinioni degli utenti.

In particolare, è stato chiesto agli utenti di:

- **TASK #1** - registrarsi;
- **TASK #2** - creare un itinerario;
- **TASK #3** - visualizzarlo;
- **TASK #4** - aggiungerlo alla lista dei preferiti;
- **TASK #5** - effettuare una ricerca;
- **TASK #6** - inviare un messaggio.

	TASK #1	TASK #2	TASK #3	TASK #4	TASK #5	TASK #6
Utente 1	S	F	S	S	P	S
Utente 2	S	F	S	S	P	S
Utente 3	S	F	S	S	S	S
Utente 4	S	P	S	S	S	S

Leggenda: S = Successo, F = Fallimento, P = Successo parziale

Come si può notare dalla tabella, c'è un tasso di successo pari all'83%, risultato vicino a quello desiderato.

La schermata dei messaggi è stata particolarmente apprezzata per i colori; sono stati utilizzati due tipi di verdi che richiamano al tema generale dell'app e il colore del pulsante di invio rimanda al colore dei messaggi inviati.

I punti di criticità, invece, sono stati riscontrati in TASK #2 e TASK #5, cioè creazione di itinerario e ricerca.

Gli utenti hanno riscontrato difficoltà nell'inserimento e cancellazione di punti nella mappa durante la creazione di un itinerario e hanno trovato problemi quando la ricerca effettuata non ha dato risultati, non lasciando nessun messaggio di informazione.

Gli utenti hanno avuto problemi anche nella schermata principale, dove non era presente nessun simbolo che indicasse all'utente in che sezione si trovasse.

Questi problemi sono stati risolti per non aderenza all'euristiche di Nielsen.

In particolare, prima della modifica, i punti sulla mappa venivano aggiunti o cancellati al singolo tocco.



La situazione era la seguente:

```

----- beginning of main
03-19 22:52:17.715 6296 6296 I UI_INTERACTION: Cliccato
pulsante Next.
03-19 22:52:19.094 6296 6296 I UI_INTERACTION: Cliccato
pulsante Next.
03-19 22:52:20.463 6296 6296 I UI_INTERACTION: Cliccato
pulsante Next.
03-19 22:52:20.464 6296 6296 I UI_INTERACTION: Apertura
schermata Home.
03-19 22:52:25.259 6296 6296 I UI_INTERACTION: Premuto sulla
tab 3.
03-19 22:52:30.171 6296 6296 I UI_INTERACTION: Premuto pulsante
per aggiungere itinerario.
03-19 22:52:32.436 6296 6296 I UI_INTERACTION: Premuto il
pulsante "Manually".
03-19 22:52:32.447 6296 6296 I UI_INTERACTION: Aperta schermata
della mappa editabile.
03-19 22:52:39.145 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.160 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.192 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.270 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.271 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.333 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.383 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.428 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.452 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.503 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.506 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.551 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.636 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.641 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.703 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.746 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.786 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.791 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.832 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.916 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:39.936 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.004 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.056 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.076 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.094 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.151 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.207 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.214 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.258 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.320 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.361 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.365 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.417 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.454 6296 6296 I UI_INTERACTION: Movimento mappa.
03-19 22:52:40.509 6296 6296 I UI_INTERACTION: Movimento mappa.

```



03-19 22:52:51.418	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:51.477	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:51.554	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:51.657	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:53.019	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:53.195	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:57.650	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:57.712	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:57.801	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:57.996	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:58.126	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:58.316	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:58.332	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:58.457	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:58.573	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:58.659	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:58.789	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:58.948	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:58.968	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:59.081	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:59.137	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:59.212	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:59.225	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:59.340	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:59.411	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:52:59.478	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:53:04.486	6296	6296	I	UI_INTERACTION: Aggiunto waypoint.
03-19 22:53:06.721	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:53:06.775	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:53:06.863	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:53:06.915	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:53:07.040	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:53:07.111	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:53:07.184	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:53:07.278	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:53:07.350	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:53:07.368	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:53:07.477	6296	6296	I	UI_INTERACTION: Movimento mappa.
03-19 22:53:08.749	6296	6296	I	UI_INTERACTION: Premuto su marker.
03-19 22:53:08.750	6296	6296	I	UI_INTERACTION: Visualizzata dialog eliminazione waypoint.
03-19 22:53:11.218	6296	6296	I	UI_INTERACTION: Premuto pulsante CANCEL.
03-19 22:53:13.082	6296	6296	I	UI_INTERACTION: Aggiunto waypoint.
03-19 22:53:15.435	6296	6296	I	UI_INTERACTION: Aggiunto waypoint.
03-19 22:53:16.656	6296	6296	I	UI_INTERACTION: Aggiunto waypoint.
03-19 22:53:17.963	6296	6296	I	UI_INTERACTION: Aggiunto waypoint.
03-19 22:53:19.507	6296	6296	I	UI_INTERACTION: Premuto su marker.



```

03-19 22:53:19.508 6296 6296 I UI_INTERACTION: Visualizzata
dialog eliminazione waypoint.
03-19 22:53:21.042 6296 6296 I UI_INTERACTION: Premuto pulsante
CANCEL.
03-19 22:53:22.852 6296 6296 I UI_INTERACTION: Aggiunto
waypoint.
03-19 22:53:27.538 6296 6296 I UI_INTERACTION: Premuto su
marker.
03-19 22:53:27.539 6296 6296 I UI_INTERACTION: Visualizzata
dialog eliminazione waypoint.
03-19 22:53:29.158 6296 6296 I UI_INTERACTION: Premuto pulsante
CANCEL.
03-19 22:53:31.469 6296 6296 I UI_INTERACTION: Premuto su
marker.
03-19 22:53:31.470 6296 6296 I UI_INTERACTION: Visualizzata
dialog eliminazione waypoint.
03-19 22:53:32.900 6296 6296 I UI_INTERACTION: Premuto pulsante
CANCEL.
03-19 22:53:36.969 6296 6296 I UI_INTERACTION: Aggiunto
waypoint.
03-19 22:53:40.444 6296 6296 I UI_INTERACTION: Aggiunto
waypoint.
03-19 22:53:42.709 6296 6296 I UI_INTERACTION: Aggiunto
waypoint.
03-19 22:53:43.768 6296 6296 I UI_INTERACTION: Aggiunto
waypoint.
03-19 22:53:45.804 6296 6296 I UI_INTERACTION: Aperta schermata
per il settaggio dei dettagli dell'itinerario
03-19 22:54:09.820 6296 6296 I UI_INTERACTION: Visualizzata
process dialog.
03-19 22:54:16.039 6296 6296 I UI_INTERACTION: Chiusa la
process dialog.
03-19 22:54:16.041 6296 6296 I UI_INTERACTION: Visualizzata
dialog di salvataggio avvenuto con successo.
03-19 22:54:18.185 6296 6296 I UI_INTERACTION: Premuto pulsante
OK.

```

Come si può facilmente notare, l'utente è stato costretto più volte ad eliminare un punto aggiunto per errore, andando a penalizzare la sua esperienza.

Il problema è stato risolto rendendo l'aggiunta dei punti meno casuale grazie al tocco prolungato.

La situazione, dopo la modifica, è la seguente.

```

03-19 23:00:26.858 6695 6695 I UI_INTERACTION: Cliccato
pulsante Next.
03-19 23:00:27.532 6695 6695 I UI_INTERACTION: Cliccato
pulsante Next.
03-19 23:00:28.066 6695 6695 I UI_INTERACTION: Cliccato
pulsante Next.
03-19 23:00:28.066 6695 6695 I UI_INTERACTION: Apertura
schermata Home.
03-19 23:00:31.932 6695 6695 I UI_INTERACTION: Premuto sulla
tab 3.

```



03-19 23:00:33.219 6695 6695 I UI_INTERACTION: Premuto pulsante
 per aggiungere itinerario.
 03-19 23:00:36.157 6695 6695 I UI_INTERACTION: Premuto il
 pulsante "Manually".
 03-19 23:00:36.165 6695 6695 I UI_INTERACTION: Aperta schermata
 della mappa editabile.
 03-19 23:00:42.513 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:42.530 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:42.565 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:42.608 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:42.627 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:42.660 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:42.696 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:42.731 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:42.833 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:42.833 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:42.836 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:42.882 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:42.943 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:42.998 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.054 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.091 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.127 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.176 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.191 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.240 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.283 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.304 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.360 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.406 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.426 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.463 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.486 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.532 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.551 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.579 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.638 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.640 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.680 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.721 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.742 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.778 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.831 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:43.871 6695 6695 I UI_INTERACTION: Movimento mappa.
 03-19 23:00:50.262 6695 6695 I UI_INTERACTION: Aggiunto
 waypoint.
 03-19 23:00:53.292 6695 6695 I UI_INTERACTION: Aggiunto
 waypoint.
 03-19 23:00:54.961 6695 6695 I UI_INTERACTION: Aggiunto
 waypoint.
 03-19 23:00:56.724 6695 6695 I UI_INTERACTION: Aggiunto
 waypoint.
 03-19 23:00:58.619 6695 6695 I UI_INTERACTION: Aggiunto
 waypoint.
 03-19 23:01:00.850 6695 6695 I UI_INTERACTION: Aggiunto
 waypoint.



```

03-19 23:01:02.698 6695 6695 I UI_INTERACTION: Aggiunto
waypoint.
03-19 23:01:09.030 6695 6695 I UI_INTERACTION: Aggiunto
waypoint.
03-19 23:01:11.362 6695 6695 I UI_INTERACTION: Premuto su
marker.
03-19 23:01:11.363 6695 6695 I UI_INTERACTION: Visualizzata
dialog eliminazione waypoint.
03-19 23:01:13.308 6695 6695 I UI_INTERACTION: Premuto pulsante
CONFIRM.
03-19 23:01:13.308 6695 6695 I UI_INTERACTION: Waypoint
cancellato.
03-19 23:01:15.477 6695 6695 I UI_INTERACTION: Aggiunto
waypoint.
03-19 23:01:17.438 6695 6695 I UI_INTERACTION: Aperta schermata
per il settaggio dei dettagli dell'itinerario
03-19 23:01:31.766 6695 6695 I UI_INTERACTION: Premuto il
pulsante "get info from internet".
03-19 23:01:37.882 6695 6695 I UI_INTERACTION: Visualizzata
process dialog.
03-19 23:01:43.982 6695 6695 I UI_INTERACTION: Chiusa la
process dialog.
03-19 23:01:44.001 6695 6695 I UI_INTERACTION: Visualizzata
dialog di salvataggio avvenuto con successo.
03-19 23:01:45.785 6695 6695 I UI_INTERACTION: Premuto pulsante
OK.

```

Sono stati aggiunti anche dei messaggi di risposta all'utente in caso di mancati risultati di ricerca.

Infine, è stata aggiunto un indicatore che segnala all'utente la sezione in cui si trova.

Dopo le modifiche effettuate è stato chiesto ad un nuovo bacino d'utenza di effettuare gli stessi test precedenti.

	TASK #1	TASK #2	TASK #3	TASK #4	TASK #5	TASK #6
Utente 1	S	S	S	S	S	P
Utente 2	S	P	S	S	S	S
Utente 3	P	S	S	P	S	S
Utente 4	S	S	S	S	S	S

Leggenda: S = Successo, F = Fallimento, P = Successo parziale

NaTour ha ottenuto un tasso di successo del 91% , indicando come i cambiamenti effettuati hanno portato ad un miglioramento generale nell'utilizzo dell'applicazione.

